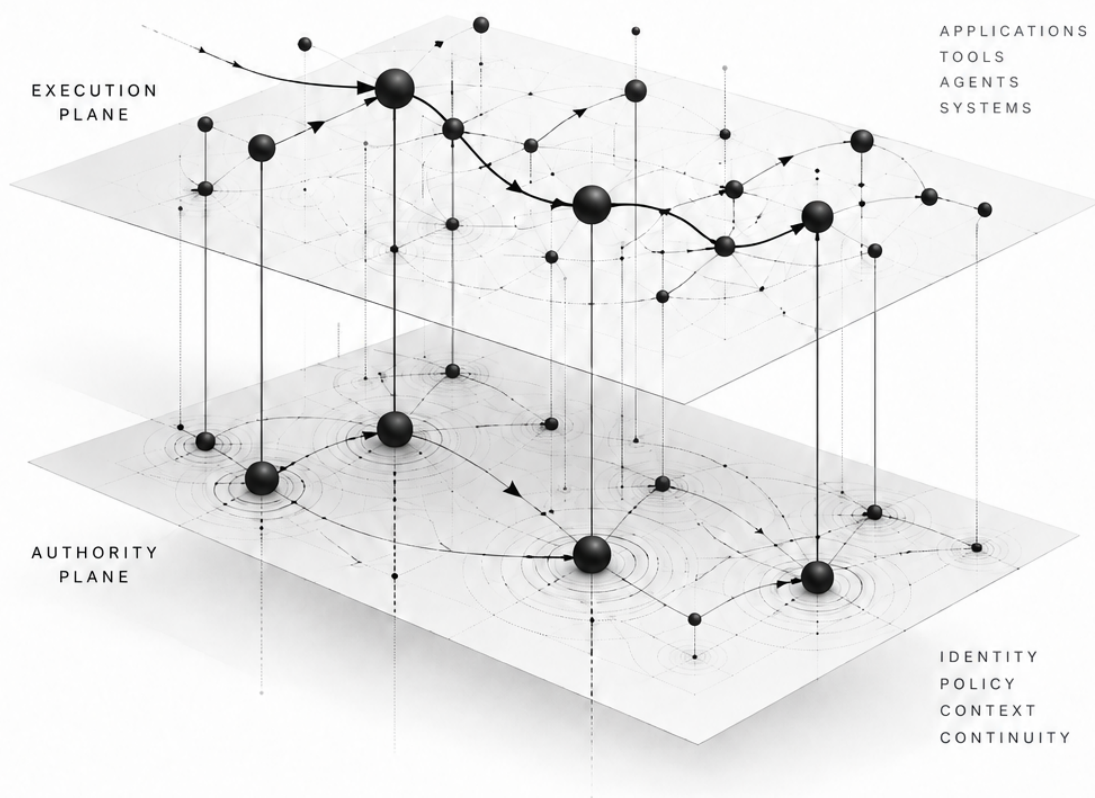


Outcome-Oriented Programming

From Applications to Agentic Execution



BUILD RELIABLE, CONTEXT-AWARE SYSTEMS
THAT TURN INTENT INTO REAL-WORLD OUTCOMES

Nicola Gallo

Outcome-Oriented Programming is not a prediction that one architecture will win. It is a hypothesis that, as execution becomes more agentic, production systems will increasingly need to separate probabilistic intent interpretation from deterministic execution, explicit authority propagation, formal verification, and governance.

Outcome-Oriented Programming

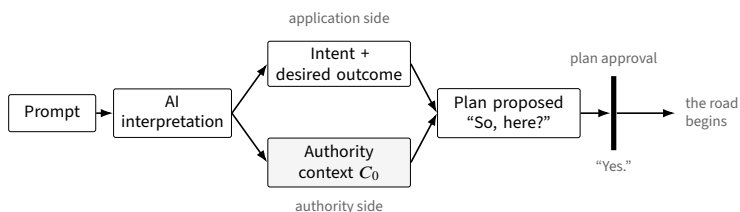
From Applications to Agentic Execution

Why This Book Exists

Organizations of every size, from startups to enterprises, are already bringing **large language models into software development and operations**: they generate code, tests, documentation, and refactorings; they assist debugging; they power chatbots and automate support. This changes the speed of software production, but not yet its basic structure. The organization still builds the application in advance and operates it; the user later uses it, within the behavior that was authored ahead of time. For an enterprise the stakes are higher: it already runs business processes, know-how, services, and controls, and the real value lies in **letting AI agents act on those existing processes**.

Agentic execution is the next step. Instead of asking AI to help build software for a future request, the user asks an agent directly for an outcome. In the theoretical limit, the agent interprets the request, decides what must happen, creates or composes the missing execution logic, calls tools and systems, adapts to runtime evidence, and returns the result. The pre-authored application stops being a necessary intermediate object.

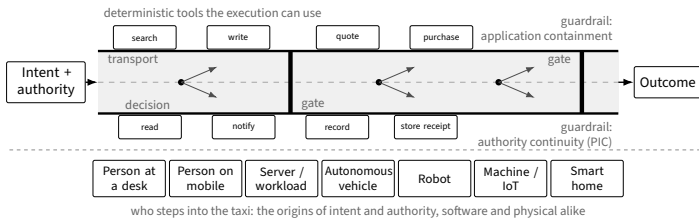
Before any execution starts, the prompt has to become an **intent**. To read the picture, imagine you have just stepped into a taxi. You say where you want to go. The driver works out two things at once: on the **application side**, the destination, that is the intent and the desired outcome; on the **security side**, what you are entitled to ask of them, that is the authority context. Then the driver proposes a route: “so, here, this way?” That proposal is the **plan**. Your “yes” is its **approval**. Only after that does the journey begin.



From prompt to intent and authority: interpretation fixes the destination on the application side and the authority context on the security side; the plan is then proposed (“so, here?”) and approved (“yes”) before execution starts.

Now think of the execution itself as a **road**. Intent and authority are where the road begins; the outcome is where it must arrive. Along the way the AI makes choices at forks and uses the **deterministic tools** that stand on the road edges. **Authority continuity is one dimension of security**: it is verified on every step, and the **guardrails** keep each choice inside the admissible space. If the execution has the authority to use a tool and that tool needs no policy decision, security is enough and the road goes on. **Gates are governance**: explicit boundaries where a control policy requires a decision before the journey may continue. Who steps into the taxi can be a person at a desk, a person on a phone, a server or cloud workload talking machine to machine, an autonomous vehicle, a robot, a machine on a factory floor, a device in a home: **any of them can originate an intent and its authority**, and any of them can be where the

road arrives. This is why the architecture in this book does not distinguish, at its core, between software actors and physical ones: what is today an integration between software and the physical world becomes, on this road, **one execution that crosses both**.



An execution as a road: the road is the transport, the AI chooses at forks and uses the tools on the edges; authority is verified on every step within the guardrails, gates are governance and stand only where a policy requires a decision. Below the line, who can step into the taxi.

TRANSPORT IS NOT THE SECURITY MODEL

The road itself is the **transport**: the protocols, queues, and channels that carry an execution from one step to the next. A transport must be secure in its own right: confidentiality, integrity, authentication, and privacy of what travels on it are requirements of the road, and its properties can legitimately be inputs to security policy. The requirement here is narrower: **transport alone must not be taken as proof that a requested effect is authorized**. When authorization is inferred implicitly from transport trust, as when a trusted channel is taken as proof that a step is allowed, part of the authorization rationale lives outside the authority model; reasoning, composition, and audit become harder, and inconsistencies and vulnerabilities tend to appear in those gaps. The authority model should be able to state, in its own terms and with any explicitly admitted transport properties as inputs, why a continuation is authorized. **Transport moves the execution and protects it in transit; by itself it does not decide what the execution may cause.**

That limit is not a production architecture. Generating everything on every request adds latency, inference and validation cost, and new uncertainty. Correctness, security, governance, compliance, and accountability also move into the execution path. The engineering problem becomes where to use open-ended intelligence and where to reuse deterministic, bounded, and verifiable software.

Outcome-Oriented Programming proposes that separation:

Prompt → Intent + Outcome + Authority → Ontology → IER / Plan
→ Plan approval → Compatible Tools → Execution → Evidence / Outcome

The shift also changes what it means to trust software. Digests, signatures, and supply-chain provenance can support claims about artifact identity, integrity, signer, and release path under a trust model. **They do not, by themselves, establish program behavior**. Tools therefore need explicit machine-readable semantics, effects, contracts, invariants, and stronger assurance where the risk justifies it.

Execution is then constrained in two independent ways. **Application containment** constrains accepted software behavior. **Authority containment** constrains what this execution lineage may cause; PIC (www.pic-protocol.org) provides the temporal authority model for that dimension. Runtime evidence feeds governance gates, which decide on residual risk.

The paradigm shift is from AI that helps write applications to AI that programs outcomes over a constrained, reusable, and increasingly verifiable execution substrate.

Publication and Rights

Outcome-Oriented Programming: From Applications to Agentic Execution

First edition, 6 September 2026.

Author and Original Creator: Nicola Gallo

Publisher / Issuer, Economic Copyright Holder and Licensor: Nitro Agility S.r.l.

Nicola Gallo authored and conceptually developed the original contributions presented in this work. For this publication, he acts in his capacity as director of Nitro Agility S.r.l., in the name and on behalf of the company.

Copyright © 2026 Nitro Agility S.r.l.

Except where otherwise indicated, the original text, original diagrams, and original illustrative pseudocode and listings in this book are licensed by Nitro Agility S.r.l. under the **Creative Commons Attribution 4.0 International License (CC BY 4.0)**. The standard CC BY 4.0 legal code governs that license; this publication does not replace or modify it.

Canonical license: <https://creativecommons.org/licenses/by/4.0/>

Book source and editions: the LaTeX source, errata, and future editions of this book are maintained at <https://github.com/nitroagility/books>, in the directory `outcome-oriented-programming`.

Suggested attribution: *Nicola Gallo (author and original creator), Outcome-Oriented Programming: From Applications to Agentic Execution, © 2026 Nitro Agility S.r.l., CC BY 4.0.*

Third-party works and separately licensed material remain subject to their respective rights and terms. No patent or trademark rights are granted except where expressly stated.

Technical notice. This book presents research, theoretical models, and architectural proposals; it is not a certification or a guarantee that any implementation will be secure, correct, compliant, safe, or suitable for a particular deployment. Mathematical and formal claims apply only to the stated models, properties, assumptions, and artifacts. Technical content is provided without warranty as to accuracy or completeness; formal claims remain limited to their stated models, assumptions, properties, and referenced artifacts. Implementers remain responsible for their own systems, validation, compliance, and third-party rights.

Detailed authorship, AI-assistance, implementation-responsibility, third-party-rights, and attribution notices are collected in **Appendix A**. The methodology used to separate theorem, implementation claim, architectural proposal, and open problem is collected in **Appendix B**. A short, non-normative correspondence to selected enterprise risk frameworks is collected in **Appendix C**.

Contact. To reach the author or the publisher:

nicola.gallo@nitroagility.com

info@nitroagility.com

<https://nitroagility.com/>

Implementation. The way of thinking described in this book is being put into practice in Permguard, an open-source authorization project by Nitro Agility: <https://permguard.com/>.

Typeset in LaTeX. Interior diagrams are monochrome and vector-based.

Preface

The visible AI transition begins with large language models entering software development and operations. This is true for startups and enterprises alike: models help programmers generate code, tests, documentation, refactorings, and debugging hypotheses; they power chatbots and optimize support; and they materially change the speed and economics of software production. But it is still an evolution inside the familiar application-centric model: requirements are turned into an application, the organization deploys and operates it, and users then use it within the behavior it already encodes.

For an established organization the question is larger. An enterprise does not start from a blank page: it already runs business processes, accumulated know-how, services, controls, and obligations that predate any new software. The relevant shift is therefore not only how fast software can be written, but **how AI agents can act directly on those existing processes.**

This book asks what comes next. The more radical step is not AI that writes the application faster, but **an AI agent that receives the desired outcome directly and constructs the execution needed to reach it.** In the theoretical limit, the agent can understand the request, determine what must happen, generate or compose missing execution logic, call the necessary systems, adapt to intermediate results, and return the outcome. A task-specific application would no longer have to exist before the request.

That is a much larger claim than “AI writes code,” and it needs an equally important qualification. The computing substrate does not disappear. Models, runtimes, operating systems, databases, protocols, services, tools, networks, and physical machines remain. **What becomes optional, in the theoretical limit, is the pre-authored application as the mandatory object between a human outcome and its execution.**

Production reality immediately pushes back. Unrestricted runtime generation moves design-like work into the hot path: reasoning, synthesis, validation, compilation, and verification consume time and compute before useful deterministic work begins. It also moves decisions that software engineering traditionally made before deployment into runtime. Correctness, security, authority, governance, compliance, evidence, and accountability therefore have to become part of execution itself.

The economic consequence is equally important. When a stable operation already exists, regenerating it repeatedly discards the advantage software has always obtained from reuse. The production architecture should therefore spend open-ended intelligence where interpretation and planning are genuinely needed, then lower the result into reusable deterministic or bounded execution wherever possible.

This is the tension from which Outcome-Oriented Programming begins. The theoretical limit says that the application can disappear as a required prior composition. The engineering answer is selective: **intelligence interprets and proposes; reusable**

software executes; verifiers and governance boundaries decide which proposals are allowed to become effects.

That change exposes a trust gap that ordinary artifact controls do not close. Modern software supply chains can use digests, signatures, publisher identity, provenance, and release metadata to support claims about which artifact was received and whether expected bytes or release paths were preserved under a trust model. Those mechanisms are important, but they are not semantic proofs of what the program will do. Agentic execution therefore needs an explicit behavioral layer as well: machine-readable semantics, effects, preconditions, postconditions, invariants, and assurance evidence tied to the tools that execution may compose.

A further shift is temporal. Distributed software has always executed across time: a producer acts now, a message waits, and a future consumer may not yet exist. Agentic systems generalize that condition. Not only the next executor but also the concrete next option, evidence, or outcome may be unknown when the intent is expressed. Outcome-Oriented Programming therefore needs an execution model that remains meaningful across asynchronous boundaries and unknown future facts.

The road of the opening pages also dissolves a boundary that software architecture has treated as fixed. An execution may be originated by, and may arrive at, a person acting through a computer or a mobile device, a server or cloud workload acting machine to machine, an autonomous vehicle, a robot, an industrial machine, or a connected object in a home. If the meaning of the execution, its authority, and its governance are attached to the lineage rather than to the application that hosts a step, then **software and physical executors are not two worlds joined by integration. They are positions on the same road.** This book does not predict how fast that convergence will happen; it argues that an architecture for agentic execution should not assume the distinction.

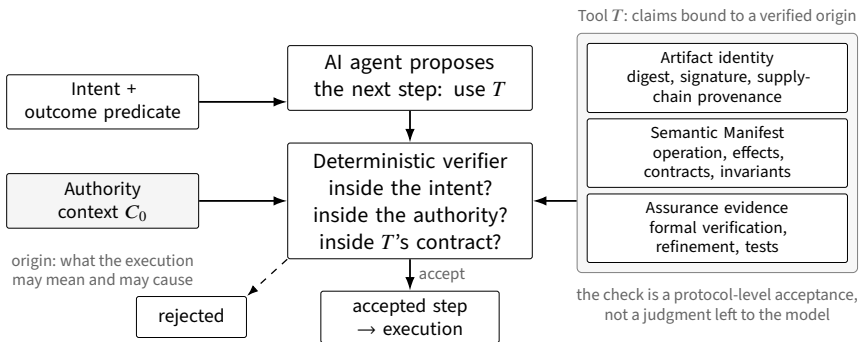
A further shift is about authority. If an execution can cross applications, services, tools, queues, agents, and organizations, the application is no longer a sufficient security boundary. Neither is the transport. Channels, queues, and protocols must be secure in their own right, but the authority of a step cannot be inferred from the channel it arrived on; **a security model must be able to state, in its own terms, why a step is authorized.** A single execution may cross many software stacks, while a single service may process many unrelated executions at once. The meaningful unit becomes the **execution lineage**: the causal chain that begins with an intent and continues through each action that the intent causes.

This is where Provenance Identity Continuity (PIC) becomes central. Public PIC resources are maintained at <https://www.pic-protocol.org>; the source organization is <https://github.com/pic-protocol>; the formal-model portal is <https://www.pic-protocol.org/formal-model>; and the theoretical paper is available at <https://arxiv.org/abs/2607.08906>. PIC models authority as something that must remain causally connected to the execution that originated it. The same execution can be viewed through two parallel planes. The **application plane** asks *what the system is doing*. The **authority plane** asks *what this specific lineage is*

allowed to cause. They describe the same events from different perspectives.

That temporal coordinate also changes the meaning of composition. Two executions can expose the same apparent authority and still represent different authorizations because their authority descends from different causes. An executor that happens to possess two independently valid authorities is not thereby authorized to merge them inside an arbitrary task. If two lineages are intentionally combined, the combination is itself a security decision: policy must authorize the meeting, and the contributing lineages must remain distinguishable. In compact form: **possession does not authorize composition**.

These shifts make the architecture verifier-oriented. AI planning remains probabilistic, but acceptance of critical state transitions should be as deterministic and checkable as the property permits. Tools increasingly need machine-readable Semantic Manifests and contracts describing ontology-level operations, preconditions, effects, postconditions, invariants, and the evidence supporting those claims. Intent and desired outcomes can then be lowered into the same semantic coordinate system before tool binding occurs. The result of that lowering is a compact, machine-oriented plan, the Intent Execution Representation (IER), which the runtime validates instead of natural language. Models can propose semantic interpretations, plans, tools, parameters, and continuations. Deterministic and, where practical, formally verified components decide whether selected semantic contracts, application invariants, authority conditions, and evidence predicates hold before those proposals become execution. Seen from the tool's side and from the origin's side, that check looks like this.



Verifier-oriented execution. A tool carries its artifact identity, its Semantic Manifest, and its assurance evidence, all bound to a verified origin. Before a proposed step becomes execution, a deterministic verifier checks that it is semantically inside the intent, inside the authority context, and inside the tool's contract. That acceptance lives at protocol level, not in the model's confidence.

In compact form, the proposed execution path is:

Prompt → **Intent + Desired Outcome + Authority** → **Ontology** → **IER / Plan**
 → **Plan Approval** → **Semantically Compatible Tools**
 → **Execution** → **Evidence** → **Outcome**

The semantic mapping is not merely a search convenience. It is the bridge between

what the user means and what executable components claim to do. A tool is not selected only because an agent predicts that it is useful. Its Semantic Manifest should state the ontology-level operation it implements, its preconditions, effects, postconditions, invariants, and the assurance evidence supporting those claims. Formal verification, runtime verification, testing, attestation, or other assurance mechanisms can then be attached to specific claims rather than to a vague label of “trusted software.”

This creates two independent containment problems that meet at the same execution event. **Application containment** asks whether the selected tool and transition remain inside the accepted semantic and behavioral contract. **Authority containment** asks whether this specific execution lineage is allowed to cause the transition; PIC addresses that second question through causal continuity and non-expansive authority. Neither property substitutes for the other.

As an architectural acceptance predicate, a critical step can therefore be written schematically as follows.

ACCEPTANCE PREDICATE FOR A CRITICAL STEP

$$\text{Accept}_i = \text{SemanticCorrect}_i \wedge \text{OutcomeCompatible}_i \\ \wedge \text{AuthorityValid}_i \wedge \text{PolicyAcceptable}_i .$$

This expression is a design decomposition, not a claim that all four predicates are automatically decidable or formally proven in every deployment. The point is to make the proof obligations and residual uncertainty explicit.

Execution then produces evidence rather than only side effects: cost, latency, observed effects, policy state, compliance state, assurance state, risk indicators, and progress toward the desired outcome. Those observations can feed governance KPI/KRI and Execution Gates. Where residual risk is acceptable, execution can continue; where it is conditionally acceptable, the runtime can constrain, validate, or escalate; where it is unacceptable, the transition can be denied or redirected to a separately authorized path. *The architecture therefore does not promise zero residual risk.* It tries to convert more of that risk from an implicit hope about model behavior into something that can be **bounded, verified, observed, and governed**.

Human-in-the-loop changes meaning in the same architecture. A human does not need to review every line generated by an AI. Execution can stop at a gate when it reaches an authority, risk, compliance, or policy boundary. That gate may be evaluated by deterministic policy, specialized governance agents, a human, or a combination. Humans remain where an organization deliberately requires human authority, judgment, or accountability.

The thesis of the book can be stated in eight points:

- Outcomes replace pre-authored workflows as the starting point.
- Intent is lowered into explicit semantics.
- Agents propose composition.
- Verified tools perform stable work.
- Application behavior and authority are constrained separately.
- Lineages preserve causality across time.
- Independent authorities meet only through explicit composition.
- Governance controls residual risk and continuation.

The result is not simply “AI that writes software.” It is a move from application-centric software toward outcome-centric, lineage-aware, verifier-oriented execution.

Contents

Preface	6
How to Read This Book	18
I When the Application Stops Being the Unit	19
1 At the Limit, the Application Disappears	20
1.1 From LLM-assisted software to agentic execution	25
1.2 The market restructures	27
1.3 Digital sovereignty as a parallel opportunity	30
1.4 A precise statement of the theoretical limit	33
1.5 Why this is a real ontological break	34
1.6 The contradiction appears immediately	34
1.7 The economic contradiction	35
2 From Possibility to Production	36
2.1 The first boundary: proposal is not execution	36
2.2 Risk determines how much freedom can remain dynamic	36
2.3 The second boundary: intelligence versus deterministic execution	37
2.4 Reuse is an economic consequence, not nostalgia	38
2.5 The blockchain analogy: universality is not economics	38
2.6 The ontological shift	39
3 Outcome-Oriented Programming	41
3.1 From instructions to desired state	41
3.2 What is being programmed?	41
3.3 The desired separation	42
4 Intent, Goal, and Outcome	43
4.1 Outcome	43
4.2 Goal	43
4.3 Intent	43
4.4 Semantic intent is a compiled interpretation	44
II The Execution Architecture	45
5 From Applications to Tools	46
5.1 Tool versus operation	46
5.2 From API description to Semantic Manifest	46

5.3	Relation to semantic service descriptions	47
5.4	Why developers still matter	47
6	Semantic Manifests and Ontological Execution	49
6.1	Artifact identity is not behavior	49
6.2	A Semantic Manifest	49
6.3	From prompt to semantic intent	50
6.4	Semantic tool binding	50
6.5	The two containments	51
7	The Intent Execution Representation	52
7.1	Not necessarily a language	52
7.2	What IER must make explicit	52
7.3	Why machine-oriented code can be better	53
8	Execution Happens in the Future	54
8.1	Causal time is not expiration time	54
9	The N+1 Unknown Executor Problem	56
10	The N+1 Outcome on Unknown Facts	57
11	The N+1 Invalid State Problem	59
11.1	Same apparent authority can encode different authorization	59
12	Execution Lineage	61
12.1	Lineage is not topology	61
12.2	Lineage is not identity propagation	61
12.3	The unit of reasoning changes	62
13	Two Planes, One Execution	63
13.1	A single event, multiple coordinates	63
13.2	Why this matters	63
14	The Application Plane	65
14.1	Planning	65
14.2	Tool matching	65
14.3	Application containment	65
14.4	Outcome validation	66
14.5	Failure is part of the program	66
15	The Authority Plane	67
15.1	Relationship and continuity	67
15.2	Authority to cause	67
15.3	Relation to capabilities, attenuation, and execution history	68
15.4	Authority is not possession, and possession is not composition	68

16 One Lineage, Many Stacks; One Stack, Many Lineages	70
16.1 One lineage can cross many stacks	70
16.2 One stack can process many lineages	70
17 Tools Are Not Authority	72
17.1 Why the distinction matters	72
17.2 Why this avoids capability terminology	73
III Authority and Governance	74
18 PIC: Authority Follows the Lineage	75
18.1 The origin context	75
18.2 Causality is part of authorization	75
18.3 Why this matters for AI agents	75
19 Provenance Is Not Continuity	76
20 Possession Is Not Composition	77
20.1 Same authority, different lineage, different security meaning	77
20.2 A temporal reading of the confused deputy	77
20.3 Explicit execution composition	78
20.4 Composition has its own lifetime and revocation semantics	79
21 From Existing Authority to Continuity	81
21.1 Execution context is a semantic requirement, not necessarily a carried chain	81
22 Execution Gates and the Receiving Boundary	84
22.1 A composition gate is a security boundary	84
22.2 A gate is not a UI dialog	84
22.3 What a gate should receive	85
22.4 Residual risk becomes runtime state	86
23 Beyond Human-in-the-Loop	87
23.1 The evaluator can change	87
23.2 From supervision to governance	87
24 Deferred Authority and New Origination	88
24.1 Deferred decision inside origin-bounded authority	88
24.2 Genuinely new authority	88
25 Governance Intelligence	90
25.1 Specialized governance agents	90
25.2 Governance telemetry: KPI, KRI, and outcome progress	90
25.3 Governance output should be structured	91

25.4	Humans become deliberate roots of authority	91
26	Residual Risk: Where Security Meets Governance	92
26.1	Risk is not necessarily one number	92
26.2	Controls change possibility in different ways	93
26.3	Security invariants and the proof boundary	93
26.4	Residual risk	94
26.5	Governance decides; it does not rewrite the theorem	95
26.6	Accountability is a separate output	95
26.7	Authentication, authorization, continuity, governance	96
26.8	Temporal confused deputy as a residual-risk class	96
26.9	Framework correspondence, not compliance	97
27	Governance Is Execution Too	98
28	Branches, Redelivery, and Uniqueness	99
29	Prompt Injection as an Authority Problem	100
IV	The Verifiable Intent Runtime	101
30	The Runtime Architecture	102
30.1	Responsibilities	102
30.2	The runtime is not the agent	102
31	Formal Methods Become Architecture	105
31.1	From signed bytes to behavioral claims	105
31.2	An assurance ladder	105
31.3	From test cases to proof obligations	106
31.4	Why this matters for hallucination	107
32	The PIC Proof Stack	108
32.1	What the proof stack does not prove	109
33	Formal Tool Contracts and Verifier-Oriented Execution	110
33.1	The new unit of reusable trust	110
33.2	Proof-carrying plans as a direction	111
33.3	New proof obligations created by composition and checkpointing	111
34	What Formal Verification Cannot Rescue	113
35	Execution Economics	114
35.1	Choose the cheapest sufficient mechanism	114
35.2	Budget-aware planning	114
35.3	Inference as an architectural resource	115

36	From Dynamic Plans to Deterministic Tools	116
36.1	A JIT for intentions	116
36.2	The developer's role in crystallization	116
37	Semantic Review	118
37.1	From code diff to behavior diff	118
37.2	Six diffs	118
37.3	Machine-oriented implementation	119
38	The New Role of the Programmer	120
38.1	Work that moves upward	120
38.2	Programming software for software	120
38.3	The highest-value code becomes more foundational	120
38.4	From CRUD gravity to computer-science gravity	121
V	Beyond Applications	122
39	When the Application Boundary Dissolves	123
39.1	Applications become hosts	123
39.2	Temporary applications	123
39.3	The new stack	123
40	Design Principles	125
40.1	Distinguish theoretical universality from production architecture . .	125
40.2	Program outcomes, not procedures	125
40.3	Keep intent distinct from outcome	125
40.4	Use AI for semantic leverage	125
40.5	Lower intent into an execution representation	125
40.6	Use a shared semantic coordinate system	125
40.7	Separate artifact identity from behavioral assurance	125
40.8	Maintain two independent containments	126
40.9	Make tools authority-neutral	126
40.10	Treat physical executors as tools on the same road	126
40.11	Keep transport out of the security model	126
40.12	Make lineage first-class	126
40.13	Model application and authority as parallel planes	126
40.14	Let authority follow the lineage	126
40.15	Make composition explicit	126
40.16	Preserve causal meaning, not necessarily full history	127
40.17	Do not hide authority expansion	127
40.18	Treat gates as runtime primitives	127
40.19	Put intelligence at gates, humans where deliberately required . . .	127
40.20	Crystallize repetition	127
40.21	Review semantics, not only source	127

40.22	Optimize the whole execution economy	127
40.23	Make causal time explicit	127
40.24	Sign or specify predicates for unknown outcomes	128
40.25	Separate continuity from uniqueness and composition	128
40.26	Treat AI output as proposal, not proof	128
40.27	Formalize high-leverage tools	128
40.28	Keep the theorem boundary visible	128
41	A Worked Example	129
41.1	Intent and desired outcome	129
41.2	Semantic interpretation	129
41.3	IER generation	129
41.4	Application plane and application containment	130
41.5	Authority plane	130
41.6	Governance gate	130
41.7	Outcome validation	131
42	The Architectural Shift	132
43	Conclusion: Programming Outcomes	133
A	Legal, Rights, and AI Disclosure	136
A.1	Standard copyright license	136
A.2	Book source repository	136
A.3	Authorship and publication capacity	136
A.4	Artificial-intelligence assistance	136
A.5	Technical scope and implementation responsibility	137
A.6	Third-party rights and implementation	137
A.7	Attribution and rights-review channel	138
B	Source and Claim Discipline	139
B.1	PIC public resources	139
B.2	Formal-artifact snapshot	139
B.3	Claim classes	140
C	Enterprise Risk Framework Correspondence	142
C.1	NIST Cybersecurity Framework 2.0	142
C.2	NIST Risk Management Framework	142
C.3	NIST Zero Trust Architecture	142
C.4	NIS2 and ENISA guidance	143
C.5	DORA	143
C.6	EU AI Act	143
C.7	What the correspondence does not establish	143
	Glossary	145

Contents	17
References	148
About the Author	151

How to Read This Book

The book is organized as an architectural progression rather than as a product manual.

Part I begins at the theoretical agentic limit: an outcome can be requested before any task-specific application exists, and software can be synthesized as part of execution. It then explains why that limit is not a production architecture – governance, security, compliance, correctness, latency, and economics force a new separation between probabilistic proposal and deterministic, verifiable execution – before defining Outcome-Oriented Programming. It also reads the transition economically: the arbitrage windows, the restructuring of the market and of skills, and digital sovereignty as a parallel opportunity. Those passages are the author’s economic reading, not architectural claims.

Part II introduces execution across time: tools, Semantic Manifests, ontology-based semantic binding, the Intent Execution Representation, the three N+1 problems, execution lineages, and the two-plane architecture.

Part III introduces the authority model: PIC, provenance versus continuity, the distinction between possession and composition, the temporal reading of the confused deputy, the high-level PIC-X exchange bridge, receiving boundaries, explicit composition gates, deferred decisions, and Governance Intelligence.

Part IV assembles these concepts into a verifiable Intent Runtime. It separates probabilistic interpretation and planning from semantic binding and deterministic acceptance, introduces an assurance ladder from artifact integrity to formal refinement, and explains the complementary roles of Lean, Tamarin, and ProVerif before returning to execution economics, semantic review, and the changing role of the programmer.

Part V looks beyond today’s application boundary and states the design principles of the architecture in compact form.

Two images from the opening pages recur throughout. The *taxi* stands for turning a prompt into an intent, an authority context, and an approved plan. The *road* stands for the execution itself: the transport that carries it, the guardrails and gates that bound it, and the software and physical executors it meets along the way.

The diagrams deliberately use only boxes, lines, arrows, and grayscale. The architecture should be understandable without depending on a vendor, programming language, cloud, agent framework, or particular implementation technology.

Detailed legal, rights, and AI-assistance notices are collected in Appendix A. Source and claim discipline, including PIC public references and formal-artifact fingerprints, is collected in Appendix B. Appendix C provides a short, non-normative correspondence to selected enterprise risk frameworks so the main argument remains architectural rather than compliance-oriented.

Part I

When the Application Stops Being the Unit

1 At the Limit, the Application Disappears

The visible AI transition starts with LLM-assisted software engineering. A model helps a developer generate code, tests, documentation, refactorings, or debugging hypotheses, and it answers users through chatbots and automated support. Development becomes faster and support more efficient for the user, but the architecture is still familiar:

Requirement → Developer + LLM → Code → Application → User

This is an important shift in productivity and in economic savings, but not yet an ontological one. The organization still constructs and deploys an application before the user can obtain the business outcome through it.

Drawn on a time axis, the gain is a set of shorter segments: requirement to code, and the support that follows the user.

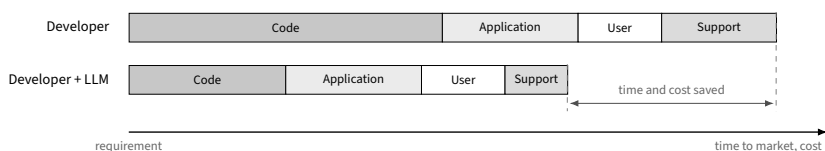


Figure 1.1: Same architecture, shorter segments: the LLM shrinks the distance from requirement to code, and the support that follows the user. Not using it is not a neutral choice; it is a loss of time and cost against those who do.

There is also an economic reading of this first step. What follows in the economic passages of this chapter is the author's reading of market patterns that the industry has already lived through; it is not an architectural claim, and Appendix B classifies it separately. As Developer + LLM becomes widely available, a capability that everyone has may cease to be a durable differentiator: **the value tends to move elsewhere**. Where it moves is not yet settled. Cheaper production may simply mean more software in total, so that programmers remain as necessary as before. Or the frontier may shift to a different kind of software, in which case the race is to create that new kind rather than to optimize the production of the old one; and if that is the case, the first question is who is able to write it. The people with the skills for it might be few; or they might be the very people let go in the cost-cutting phase, whom others will pick up early and at a discount. The competition for scarce research talent around the models themselves illustrates how a complementary capability can become strategically valuable. Other outcomes are possible and not yet visible. The strategic argument advanced here is that treating the commodity itself as the advantage, as if faster generation of the same applications had settled the question, misreads the transition.

These passages are scenario analysis, not assertions that the described market, employment, organizational, or technology outcomes are certain to occur. Alternative

outcomes are possible, and the timing, magnitude, and direction of these effects may differ across industries, jurisdictions, organizations, and technological conditions.

The saving may be temporary as a source of differential margin, and this is the point on which the rest of the chapter turns. As long as few have the capability, the lower cost of development stays with whoever has it, as margin. As the capability diffuses, competitive pressure tends to transfer part of that cost reduction to customers, through lower prices, faster delivery, higher quality, or more output: someone lowers prices to win customers, and the others have to respond. This is what competition has usually done to cost reductions that everyone can obtain. How much of the gain remains as margin depends on market structure, demand, differentiation, contracts, and organizational choices; but where competition works, the saving stops being margin and becomes **the new price of software**. Seen from the customer's side, the same fact reads as the end of a premium. Prices in an industry tend to settle on its production process: the customer pays for the steps the process contains. The price of software as written code carried a premium for one of those steps, scarce labor writing code by hand. When a step disappears from the process, the price of that step tends to disappear with it, and few will pay afterwards for a piece of work that no longer exists, unless the product itself has changed. That premium was itself an arbitrage in the loose sense used here, between what code cost to produce and what customers were used to paying for it, and it narrows as the difference becomes visible and actionable. An organization that waits for that moment to fund its next step may find that the money it was counting on has already gone to price, and may face real innovation, the one described in the next section, with the same budget constraints as before and a market that has already moved. The window in which the saving can be spent is the window in which not everyone has it yet. Organizations may consume, distribute, retain, or reinvest that gain. **The hypothesis advanced here is that reinvesting part of it while it exists improves preparedness for agentic execution**; an organization that only passes it through to price gives up the money and, if the transition materializes, also falls behind those who used it to build what comes next.

KEY IDEA

In plain terms: if LLM-assisted development reduces the cost of delivering some software, an organization may temporarily capture part of that gain as margin. The gain can be passed through to customers, retained, allocated elsewhere, or reinvested. This book argues for treating at least part of it as the opportunity to build the capabilities that a more agentic execution model could require; those who keep it only as margin may see it competed away, with nothing built in exchange.

A board may reasonably read this phase, from its own vantage point, primarily as a cost-saving item: the same output with fewer people, or with cheaper people. That reading is not the board's error. The information it needs is technical, and it can only come from the technical leadership, which can add a second perspective: that part of the saving may be temporary, that a further change in the execution model could require investment before its economic value is visible, and that the margin of this phase may be the funding of the next. **Where that second perspective is**

not communicated, the responsibility sits with the technical leadership, and the organization may underinvest in capabilities that later prove important; a board that was told only about the saving will ask, with reason, why the people whose job it was to see the change coming did not say so. This is a strategic risk identified here, not a prediction about every board or every technical leader.

THE ARBITRAGE WINDOW

A commodity is not a durable advantage, but not everyone has it yet. While adoption is uneven, those who use LLM-assisted development and those who do not run at **two different speeds and two different cost baselines** (Figure 1.2). This looks like competition, but it behaves like arbitrage, and the word is used by analogy rather than in the strict financial sense: the gain comes from a price difference on the same capability, and it lasts only as long as the difference does. An adoption advantage tends to narrow as the capability diffuses, although later differences in cost, quality, regulation, access, or complementary assets can create new differentials. Large organizations, slowed by regulation, compliance, and procedure, may enter late and lose ground to smaller players, for whom the window is also a chance to catch up with competitors that have more resources. Each window tends to narrow once, on one capability; when everyone has that capability, it is a commodity in the full sense. The next window, if it comes, opens on a different capability, as the next section argues. The useful strategic point is that uneven adoption can create a temporary advantage; its duration and magnitude are empirical rather than predetermined. We appear to be in that phase now: adoption is still uneven, and the market is segmenting.

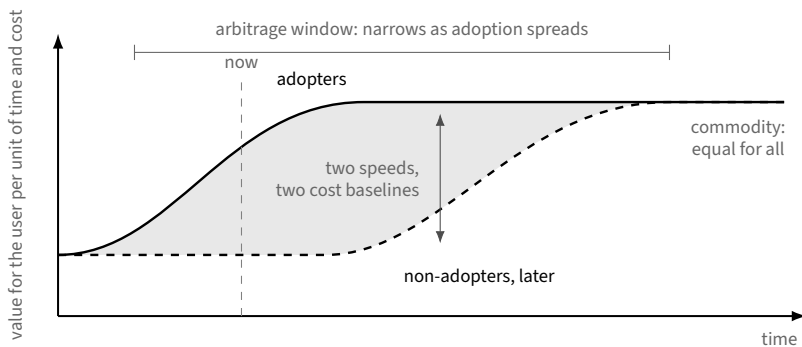


Figure 1.2: The arbitrage window: while adoption is uneven, adopters deliver more value to the user per unit of time and cost than non-adopters. The gap is a temporal and economic competition; it tends to narrow as adoption becomes general and the capability becomes a commodity.

The market remains unsettled, and this book reads the confusion as a symptom of the same phase: attention goes to the small effect, cheaper code, more than to the large one, the change of unit that the rest of this chapter describes. Frontier-model development may remain concentrated among relatively few actors because of its unusually demanding requirements. Around them, current strategies range from

optimizing models one did not build, to API-based products presented as an AI strategy, to the tooling of the moment. Many of these are positions of the phase rather than of the market that follows it: they exist because the layer is not yet a commodity, and they may thin out when it is, as businesses built on reselling bandwidth or on hosting a single server thinned out when connectivity and hosting became commodities. This book focuses on a different possible source of durable value: the substrate on which agentic execution can run, and the people able to build it.

THE ACCESS ARBITRAGE

A second arbitrage could run between those who have access to a sufficiently capable model and those who do not. If access is scarce or priced asymmetrically, organizations with materially lower access costs may obtain a temporary advantage; its duration depends on supply, competition, alternatives, infrastructure investment, open models, regulation, and technological change. It is also the arbitrage that has been most widely understood, which is why it is unlikely to be left open: much of the industry is competing for the same inputs, GPUs, the raw materials and energy behind them, the data centers that host them, and the models they serve. The race exists precisely because access is an advantage few expect to hold uncontested. If those who hold that access chose not to grant it, not to sell it to those who lack it, or to sell it at a price the others cannot bear, the advantage would be theirs for as long as it took the others to build their own, which is one reason the others are building. The mechanism is worth stating: whoever builds services on top of a model pays for the underlying capability, and if that cost differs enough between competitors, the difference alone can price the disadvantaged one out of the market, whatever the quality of what is built on top. It mirrors the argument of Section 1.3: the pressure toward local and sovereign capacity is one way the market may close this window before it can be exploited.

COMPRESSION STARTS WITH CONSULTING

One plausible early pressure point is repetitive application work in consulting firms and internal software teams. As code generation and integration become cheaper, clients may become less willing to pay historical prices for a CRUD screen or a repetitive integration; that creates price and staffing pressure, first where such work is sold explicitly and then where it is done in house. In both cases what remains is whoever carries value, and **the value may not stay in writing CRUD or repetitive tasks**: some forms of that work may become less differentiated and may be unlikely to return to their previous pricing or scarcity if automation becomes broadly available. Whether the total number of programmers falls is the open question of the previous paragraph, because lower production cost may also increase the total amount of software produced; which work stays differentiated is the more defensible conclusion. Greater value may accrue to process knowledge, testing, security, governance, control, gates, approvals, distributed systems, formal semantics, and the design of reusable execution infrastructure. The same programmers may still be needed, or fewer, or more, but for other work.

Two readings look short-sighted from here. A client strategy focused only on lowering the cost of existing CRUD delivery may miss the possibility that the larger opportunity lies in changing the execution model itself. The question is not only how to pay less for the old software but **how to accelerate toward agentic execution**; the market for undifferentiated CRUD is likely to restructure in any case. Likewise, AI reskilling alone may not be sufficient for a consulting firm if the scarce capabilities shift toward process design, computer science proper, distributed systems, security, semantics, and execution infrastructure. Section 1.2 develops this.

The saving has a concrete form on the balance sheet. The same software now costs less to produce, and the difference becomes **margin**. From inside the organization this looks like good news of a simple kind: development is cheaper, so development matters less, and the money it freed can go elsewhere. That reading is the trap. The margin is not a dividend of the transition; it is the capital the transition asks for (Figure 1.3), and Section 1.1 returns to it.

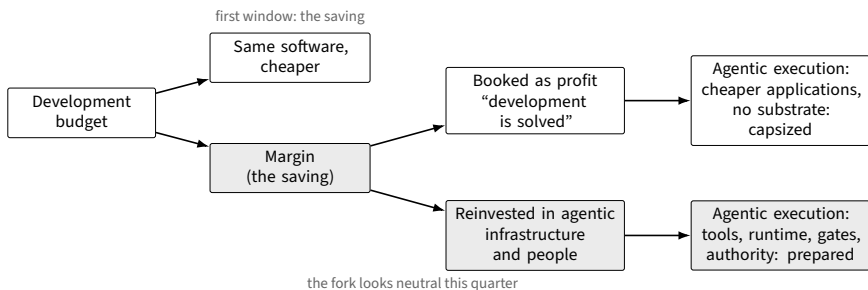


Figure 1.3: The saving becomes margin, and the margin is a fork. Booked as profit, it leaves the organization with cheaper applications at the moment applications stop being the unit. Reinvested, it buys the infrastructure and the people that agentic execution requires.

Agentic execution introduces the more radical possibility. Instead of using AI only to

help build the software that will later serve the request, the user can give an agent the desired outcome directly. The agent can interpret the request, construct or select the required execution, invoke tools and systems, observe intermediate state, adapt, and return the result. **At the theoretical limit, the task-specific application no longer has to pre-exist the request.**

Business processes existed before software. Organizations sold, approved, reconciled, transported, scheduled, paid, and audited before those activities were encoded in applications. Software was introduced to digitalize, automate, and scale those processes, while making their execution reproducible in technical systems. In doing so, it also made the application the normal container in which process logic, user interaction, security controls, and governance were assembled ahead of time.

For decades the sequence has therefore looked roughly like Figure 1.4.

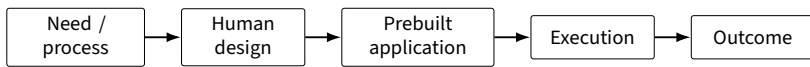


Figure 1.4: Traditional software precomputes a reusable application before the concrete execution exists.

An application is not literally the business process. It is a persistent software artifact that encodes a family of expected future executions. A travel application exists because we expect many future journeys. An expense system exists because we expect many future reimbursements. A CRM exists because we expect many future customer-management workflows. The application is, in this sense, a preassembled answer to a class of future requests.

1.1 From LLM-assisted software to agentic execution

The distinction is about *when composition happens*. LLM-assisted development accelerates the construction of a persistent application before execution. Agentic execution moves part of application-level composition after the request arrives.

At that limit, the user states what should become true. An agent interprets the request, determines what must be done, constructs the required execution, calls services or tools, observes results, adapts, and returns an outcome.

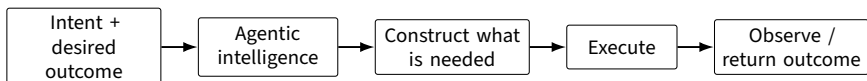


Figure 1.5: The theoretical agentic limit moves composition from design time to execution time.

At that limit, the workflow does not have to exist as a prebuilt application before the request arrives. The application can become an ephemeral product of execution rather than a prerequisite for execution.

On the same time axis (Figure 1.6), the change is no longer a shorter first segment. It is a shorter whole.

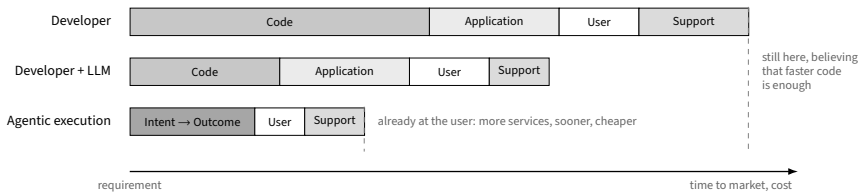


Figure 1.6: Agentic execution shortens the whole distance from requirement to user and support. If it can safely reduce that distance, early adopters may gain advantages in delivery speed, service breadth, or cost; the diagram illustrates that hypothesis, not an inevitable market sequence.

The arbitrage window of the previous pages could therefore repeat. The first window rewards LLM-assisted development and appears to be closing. A second one could emerge, at a point in time that is not fixed, if agentic execution becomes technically, economically, and organizationally practicable at scale; it would reward those who are prepared for it: tools with explicit contracts, authority that follows the execution, governance built into the runtime. **This book's bet is that we are living the first arbitrage and that a second window could emerge** if agentic execution becomes technically, economically, and organizationally practicable; it argues that organizations should prepare for that possibility before its timing or magnitude is known (Figure 1.7).

Nothing guarantees that the two windows are sequential. Agentic execution may become practicable while LLM-assisted development is still unevenly adopted; in that case the windows overlap, and an organization has to place both bets at once (Figure 1.8). **Reading market timing is therefore part of the strategy:** not only whether the second window opens, but when, and how much of the first is still open when it does.

This is where the margin of the first window matters (Figure 1.3). The saving from LLM-assisted development can be booked as profit, on the reasoning that development is now cheap and settled. Or it can be reinvested in what a second window would require: tools with explicit contracts, a runtime with gates and verifiers, authority that follows the execution, people who can build that infrastructure. The two choices look the same on this quarter's accounts and opposite on the next window's, if it opens. An organization that treats current productivity gains solely as permanent savings may arrive at agentic execution with cheaper applications and nothing else, at the moment when applications stop being the unit; **it may find itself underprepared precisely because it saved.** An organization that reinvests arrives with the infrastructure and the people, and is positioned for the second window if it comes.

Two reactions accompany this phase, and both are reactions to the first window only. One is alarm: budgets for development shrink, the volume of hand-written code falls, and it is read as the profession emptying out. The other is enthusiasm: the

saving is real and immediate, and it is read as the problem solved. Neither reaction prepares for a second window. Stated precisely: the alarm misreads a reallocation as a disappearance, and the enthusiasm misreads a possibly temporary margin as a permanent gain. Organizations and people who do not adapt, whether out of alarm or out of enthusiasm, may bear the cost if a second window opens, and they would bear it in the same form: without the substrate, the skills, and the infrastructure that agentic execution requires. The consequence would fall on those who neither adapt nor understand, regardless of which of the two reactions they had.

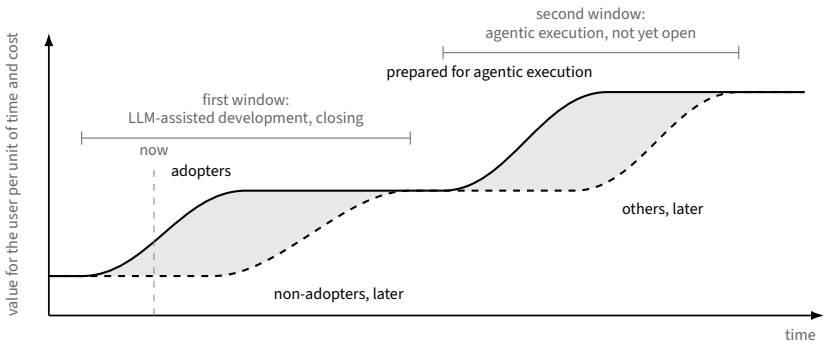


Figure 1.7: Two arbitrage windows. The first, open now, rewards LLM-assisted development and is closing as adoption spreads. A second could open if agentic execution becomes practicable, and it would reward those who are prepared for it. We are living the first; this book’s bet is that a second could emerge if agentic execution becomes practicable.

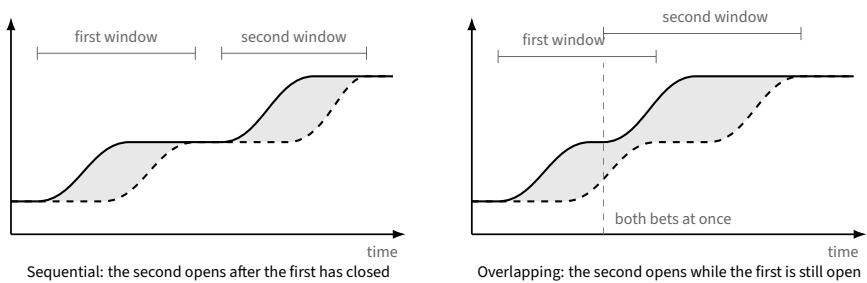


Figure 1.8: The two windows need not be sequential. If agentic execution becomes practicable while the first window is still open, the windows overlap and both bets must be placed at the same time. Market timing is part of the strategy.

1.2 The market restructures

The compression described above changes who the industry employs, and for what. It helps to look back. In the early decades of computing, programmers were few and had to know the whole machine: hardware, operating system, network, algorithm, and the problem itself. Arguably, even the weaker among them were strong by today’s

standards, because the work did not allow otherwise. As software spread, the industry filled with repetitive work: screens, forms, data mapping, integration, maintenance. Companies filled with people for that work; headcount rose while the depth of skill required fell. Agentic execution may reverse the curve. Repetitive work is the first that a model absorbs, and the hypothesis advanced here is that **the industry could move back toward its origins: fewer people, more deeply skilled** (Figure 1.9). Whether headcount actually falls also depends on how much more software is produced when producing it becomes cheaper.

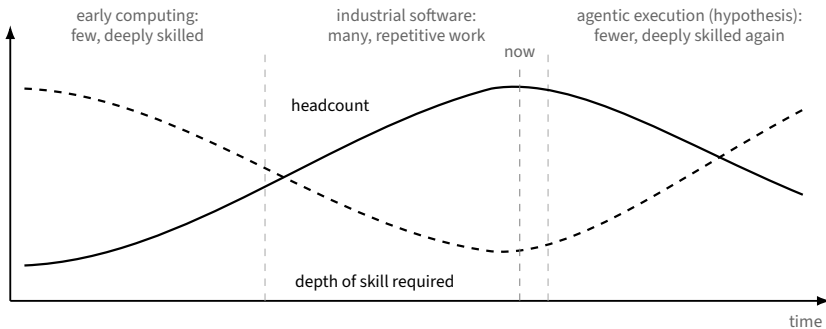


Figure 1.9: A stylized view of headcount and depth of skill over the history of the industry. Repetitive work raised headcount and lowered the skill required; if agentic execution absorbs that work first, the curves could turn back toward their origins. The right-hand segment is the hypothesis, not a measurement.

The same history explains why the model works at all. For decades the industry wrote frameworks, libraries, tools, and abstractions to simplify development. Each of them removed a hard problem and left a repetitive one in its place; that is what filled companies with repetitive work. It is now a double-edged sword, because the repetitive part is what a model absorbs first. But those same abstractions are also what makes a model productive. Imagine an LLM generating everything from scratch, without frameworks: it would produce far more errors, far more to modify and test, and it would lack the settled knowledge of how a requirement becomes working code. The distance between a requirement and its implementation would be enormous. Frameworks close that gap, and that is why generation is fast. **Agentic execution now needs the same kind of infrastructure, and it does not exist yet:** tools with explicit contracts, runtimes, gates, verifiers, and the protocols between them have to be written, by people who know how to write infrastructure.

This changes what reskilling means, or at least it opens a possibility that the reasoning above makes credible and that deserves to be weighed rather than assumed. Reskilling on AI alone is relative, because AI is a means. For routine work, a person who never wrote a CRUD screen can increasingly produce one with a model as well as a person who wrote them for ten years: on that work, **the model tends to erase the difference between them, and then the work itself**. If this reading is right, what the market needs is reskilling on foundations: processes, computer science proper, distributed

systems, security, verification. Repetitive work would still exist, but with a different value and a different price, and it would no longer sustain the large number of people, and the large demand for them, that it has sustained until now.

Figure 1.10 puts the two worlds side by side under this reading. Those who did repetitive work fall outside the new set, and **not because they cannot use AI**: they fall outside because the agent does that work. Those who stay are in the intersection because they have skills the agent does not have; if the agent had them, they would not be needed either. That paradox is what makes “reskill on AI agents” a weak slogan: the skill that keeps a person inside is not the skill of using the tool. Companies should therefore invest in processes, theory, and computer science, and use AI to accelerate that learning. The same holds for universities: the useful training is not AI as a subject but theory, and the creative, human part of the work, at least until that too is reached, if it ever is. Whether it will is outside the scope of this book. If it does, the deeply skilled may be reached as well, and the question would no longer be one of reskilling; until then, the foundations are what keeps a person on the right side of the line.

None of this is a reason to fear the profession’s future. It is an alarm about time. There is still time to update, for companies, for the people who work in them, and for students, and this book says so plainly. But it is time to use, not time to rest on. **Companies, employees, and students all have to work on this now**, and nobody who has adopted a coding assistant should conclude that the work is done.

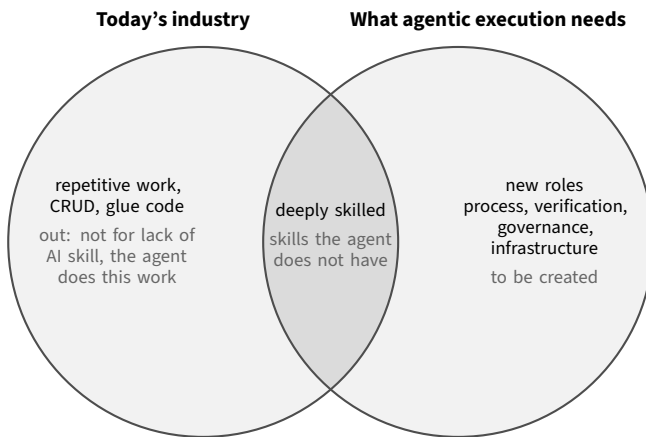


Figure 1.10: Two worlds and their intersection, as a possibility to be weighed rather than a certainty. Repetitive work falls outside the new set because the agent does it, not because the people who did it cannot use AI. The people who stay are those with skills the agent lacks; if the agent had them, they would not be needed either. Reskilling therefore has to target foundations, not the tool.

The scarce work is new. If this transition materializes, software development processes would have to be redesigned around agents; AI operations, approval processes, and monitoring tools would have to be built; whole classes of tools that do not exist yet

would have to be written, because the code of this transition is **code for machines that talk to other machines**. Consulting firms may have to make this turn first, if they are compressed first; internal teams would follow for the same reason. One plausible outcome is a smaller industry in headcount, although lower production costs could also increase total software demand; in either case the industry is likely to keep needing deeply skilled people. Demand for some of those forms of repetitive work may not return at the same scale or economic value if they become increasingly automated; the skills that served it would then need to move toward other forms of valuable work. That is not the end of their working life. Everyone carries a value; what changes is where that value is positioned, and the task is to find where it belongs now and to be ready to move there. For many, this could be the opportunity to try what they never allowed themselves to try: new ideas, a company of their own, work that gives room to their creativity. It may also happen that many who never had the courage to start a business, but who absorbed how companies work from inside, will start one because they have no alternative, and will build something better than what they left: they will fix the things they saw for years and were never allowed to fix under the old model. Moments like this are when good things are born from people. There is a reason to expect it. What the model absorbs is the repetitive part of the work, which was the part with the least human value in it; what is left to people is the creative part, and creativity is not the privilege of a few. Everyone has it, in different fields and in different shades, and a transition that removes repetition brings the human value out rather than in. It is unlikely to happen by itself, and the passage may need the support of governments to be crossed without leaving people behind: time, training on foundations, and room for those who start something new. Everything is open; we are only beginning to understand the change. This is not an end. It may be the start of something new, another revolution, and it could bring many new things with it. Be positive, understand your own value, and keep building.

1.3 Digital sovereignty as a parallel opportunity

A second opportunity runs in parallel with frontier models. The demand for digital sovereignty appears to be growing: under this scenario, organizations and states may need to install models locally, operate them, monitor them, and build independent stacks around them. That is a business in itself, and it does not compete with frontier models; it complements them. The likely shape is a mix: frontier models where breadth of capability matters and a network round trip is acceptable, local models where data cannot leave, where latency matters, or where the decision has to be taken on the spot.

The physical world makes the case unavoidable. An autonomous agent, a robot, or a vehicle that has to take a decision cannot always go to the cloud to take it. It needs intelligence on board, models sized for the device, and interoperability between the local model and the frontier model it may consult. The road of the opening pages already assumed this: the executors it meets are software and physical alike, and some of them will have to think locally. Investment could therefore move toward

local inference as well, and a whole layer of software that does not exist yet, model interoperability, local operations, sovereign stacks, would have to be built.

The economics of this layer follow a pattern the industry has seen before. When a layer becomes a commodity, value does not disappear; it moves to the adjacent layers, to what the commodity needs in order to run, and to whoever controls how it is operated. Operating systems, databases, and web servers went a long way down this path: open implementations took large parts of the market, forced the proprietary ones to adjust, and the profit moved to hosting, operations, and the services built on top. The personal computer showed the other direction of the same law: when the machine became modular, the profit moved to the components that were not yet commodities, the processor and the operating system. Under this reading, models may follow a similar trajectory. **This book expects open models to shape the future**, not because they are better, but because providers that publish capable open models may gain adoption and put competitive pressure on others to broaden access, reduce prices, or differentiate elsewhere; the broadly available model could become increasingly commoditized. Demand then shifts to what runs it, and the market competes on hardware as well, on selling it and on controlling it. Both the non-frontier model and the hardware may end up as commodities, and **the value moves to the two ends**: to the frontier, where few compete on capability, and to control, that is, who operates the infrastructure, who governs what runs on it, who holds the data and the authority it exercises.

Control is not lost in this move, but it has to be defended by speed. Existing infrastructure operators may strengthen their position by adopting the new layer before alternative sovereign stacks mature around them; slower adoption could create room for other providers to capture parts of that operational layer. For a company, internal digital sovereignty is therefore also a way to keep some of the work that would otherwise leave: operating models, monitoring them, integrating them with the processes the company already runs. For countries that do not produce frontier models, it is the way to take part in the new equilibrium rather than import it: the model may come from elsewhere, but the infrastructure, the operations, the governance, and the data can stay. None of this is a prediction about who wins. It is an analysis, and not every analysis turns out to be correct. It rests on observed foundations, the pattern the market has repeated every time a layer of computing became a commodity, and the reader should weigh whether that pattern holds here as it did before.

The equilibrium that results is a familiar one (Figure 1.11). Cloud already has this shape: a few hyperscalers lead and set the pace, the infrastructure underneath is a commodity built on shared standards, and many providers, small and local ones included, operate it close to their customers. One plausible trajectory is that models follow a similar pattern. Under this scenario, frontier models, private, may remain the differentiated edge of a few leaders with high margins; open or broadly available models may move toward commoditization, with value leaving that layer; below them a wide base of players would operate and control the infrastructure, where value is real but divided among many, unless someone consolidates it by controlling data and authority. In this reading, value does not move to the base alone: it moves to both ends

and empties the middle.

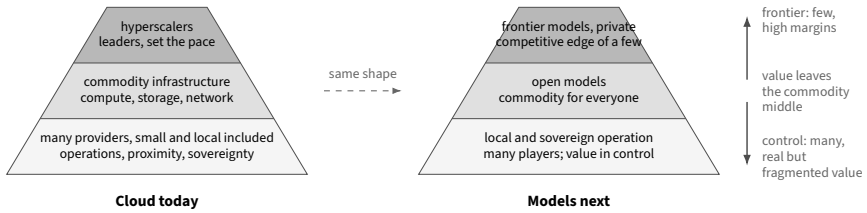


Figure 1.11: One plausible trajectory: the market for models settles the way cloud did. Under this scenario a few frontier providers lead and keep a competitive edge at the top; open or broadly available models move toward commoditization; below them, many players, small and local ones included, operate and control the infrastructure. As in cloud, value leaves the commodity middle for both ends: concentrated at the top, fragmented at the base.

None of this requires a new theory. Economics has described these movements more than once, and this book applies those descriptions as interpretation, not as law. In Stan Shih’s framework, the *smiling curve*, value in a value chain concentrates at the two ends, research and intellectual property on one side, brand, marketing, and service on the other, while the middle, manufacturing and assembly, captures little once it becomes standard [12]. This book adapts the right end to operation and control, which is where service sits in infrastructure markets. Clayton Christensen’s *law of conservation of attractive profits* suggests the dynamic version: when one layer becomes commodity and modular, attractive profit tends to move to adjacent layers that are not yet commodities [13]. Figure 1.12 draws the first; the second describes the movement that produces its shape. Applied here as an analogy to models, frontier research may keep value at one end, sovereign operation and control may keep it at the other, and the open model in the middle may become cheap.

The conclusion is not that software ends. **The application-centric model may cease to dominate every class of software work:** applications assembled ahead of time by large teams of moderately skilled people stop being the default. A new model is born, more demanding and higher in abstraction, and it may reach where the old one did not. Some forms of repetitive CRUD and integration work may become substantially less differentiated and less labor-intensive, while total software demand and the resulting employment effects remain open questions; AI skill alone would not restore that work, because AI removes the need for that skill as such rather than replacing the person who had it with a person who has it plus AI.

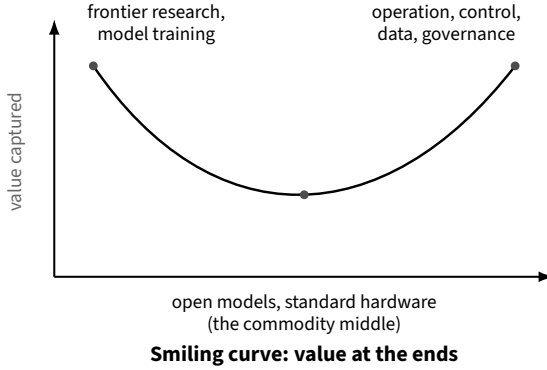


Figure 1.12: The smiling curve: value concentrates at the two ends of the value chain, research on one side and operation, control, and data on the other, while the commodity middle captures little. The law of conservation of attractive profits describes the movement that produces this shape: as the middle becomes a commodity, profit migrates to the adjacent layers.

1.4 A precise statement of the theoretical limit

The claim needs to be stated carefully. It is a conditional thought experiment, not a claim about the capabilities of current AI systems.

Let i be an intent, e an execution environment, and Φ_i a predicate describing an acceptable outcome for that intent. Call a pair (i, e) *feasible* when some executable program run in e can produce a state satisfying Φ_i ; feasibility is a property of the pair, defined independently of any synthesizer. Assume, only for the purpose of the thought experiment, a synthesis mechanism S with the following property: for every feasible pair (i, e) , it can construct an executable program $p_{i,e}$ such that the following holds.

THEORETICAL SYNTHESIS LIMIT

$$p_{i,e} = S(i, e), \quad w = \text{Exec}(p_{i,e}, e), \quad \Phi_i(w) = \top.$$

Under that assumption, the task-specific program $p_{i,e}$ does not have to exist before the request. It can be created after the intent is known. Therefore a predeployed, task-specific application is not logically necessary as an intermediate artifact for that execution.

That conclusion is deliberately narrower than saying “software disappears.” The synthesizer, model, runtime, operating system, compilers, protocols, data stores, services, networks, and physical machines are themselves software and infrastructure. What can disappear in the theoretical limit is the requirement that a particular *application composition* must have been designed and deployed in advance for each class of user outcome.

💡 KEY IDEA

The theoretical endpoint is not a world without computation. It is a world in which the application no longer has to be the pre-authored unit between intent and execution.

1.5 Why this is a real ontological break

This changes the object around which programming is organized.

In the traditional model, the persistent artifact is the application and executions are instances of that artifact. In the agentic limit, the persistent assets can instead be general execution infrastructure, reusable operations, policies, contracts, and verifiers, while the concrete workflow is synthesized as a transient execution.

	Application-centric	Agentic limit
Primary persistent object	Application / workflow	Runtime, tools, contracts, policy
When composition happens	Before deployment	At execution time
User asks for	Function exposed by an application	Outcome within constraints
Concrete workflow	Pre-authored	Synthesized / selected dynamically

This is the conceptual jump on which the rest of the book is built.

1.6 The contradiction appears immediately

The same move that makes the application theoretically unnecessary also removes assumptions on which production software has relied.

Security and governance have traditionally been constructed around software and processes that are sufficiently known ahead of execution to be tested, reviewed, approved, deployed, monitored, and assigned to responsible owners. If the concrete process is created at runtime, part of that design decision moves past the release boundary.

The result is not that governance becomes optional. The opposite is true. Controls that were previously attached to a prebuilt application must either move into runtime or be expressed as constraints that the runtime cannot bypass.

A production system therefore has to answer at least four questions that the theoretical synthesis equation does not answer:

1. **Correctness:** why should a probabilistic planner's proposed execution be accepted?
2. **Security and authority:** which effects is this particular execution allowed to cause?

3. **Governance and compliance:** under which policies, evidence, approvals, and accountability conditions may it continue?
4. **Economics:** why should the system pay open-ended inference, synthesis, and verification costs for work that deterministic software can already perform?

These are not secondary operational concerns. Once workflow construction moves to runtime, they become part of the programming model itself. Nor is the list closed: as the change crystallizes and real systems are built on it, other questions will probably surface beside these four. They are the ones visible today.

1.7 The economic contradiction

Full generation also moves design-like work into the request path. A useful accounting decomposition of request latency and request cost is the following.

REQUEST-PATH ACCOUNTING

$$T_{request} = T_{plan} + T_{synthesis} + T_{verification} + T_{execution} + T_{external}.$$

$$C_{request} = C_{inference} + C_{synthesis} + C_{validation} + C_{compute} + C_{external}.$$

These are accounting identities, not empirical performance laws. Their purpose is simply to expose where the work is paid. If a stable deterministic implementation already exists, repeatedly paying avoidable synthesis and validation cost moves useful work behind unnecessary generative work.

The theoretical limit therefore creates the book's central engineering problem:

If an agent can generate almost anything, **what should still be generated, what should instead be reused, and what must be verified** before any proposal becomes an effect?

Design principle. Universal generation is a useful theoretical limit. It is not, by itself, a production architecture, at least not with the technology that exists today.

2 From Possibility to Production

A production architecture should preserve the semantic freedom of the agentic limit without paying its full cost or accepting its full uncertainty on every execution.

The design problem is therefore not “static software or AI.” It is **how to divide responsibility between probabilistic intelligence and deterministic, governable execution.**

2.1 The first boundary: proposal is not execution

An AI model is valuable precisely because it can operate under ambiguity. That same property makes it a poor final authority for critical state transitions.

The architecture in this book uses an asymmetric rule:

Intelligence may propose broadly. Acceptance must be narrower.

A planner may propose a plan, a tool invocation, a parameter, a candidate outcome, or a continuation. The runtime decides whether that proposal satisfies the conditions required to become execution.

At a high level, an execution proposal x should be admissible only when the relevant independent predicates hold. A book-level abstraction is the following.

ADMISSIBILITY OF AN EXECUTION PROPOSAL

$$\text{Admissible}(x) = V(x) \wedge A(x) \wedge P(x) \wedge G(x),$$

where V denotes technical validation, A authority, P applicable policy or compliance predicates, and G any governance decision required by the organization. This equation is an architectural interface, not a theorem of PIC and not a claim that all risk can be reduced to four Boolean functions.

2.2 Risk determines how much freedom can remain dynamic

A useful design rule follows directly from the distinction between security and governance. **Security** reduces the probability and impact of unacceptable events. **Governance** determines whether the residual risk, under stated conditions and responsibility, is acceptable. **Compliance** may introduce hard constraints that cannot be traded for economic benefit.

That gives a practical architecture spectrum:

Risk / policy posture	Architectural consequence
Not acceptable	Constrain the process; move critical logic into deterministic, testable, and where justified formally verified software.
Acceptable only under conditions	Permit agentic planning inside explicit bounds; require validation, evidence, limits, and approval or execution gates.
Acceptable within policy	Delegate more planning freedom to the agent while retaining observable and enforceable boundaries.

This avoids a common category error. A decision can be economically attractive and still inadmissible because a legal, regulatory, contractual, safety, or authority predicate fails. *“Risk” is therefore not assumed to be one scalar price that can always be exchanged for value.*

Design principle. The amount of agentic freedom is an architectural decision derived from admissible residual risk, not a property of the model’s confidence.

The architecture therefore reduces risk in layers before governance decides whether the remainder is acceptable. Semantic contracts and formal application properties can remove selected behavior classes from the set of possible accepted executions. PIC can remove selected authority/causality mismatches from valid execution. Deterministic policy can reject known inadmissible states. Governance then operates on what remains: evidence quality, environment, compliance interpretation, economic exposure, irreversible effects, and other residual conditions that are not proven away by the lower layers.

2.3 The second boundary: intelligence versus deterministic execution

Once the planner has understood what should happen, most of the physical work should not require open-ended reasoning.

Sorting records, evaluating a typed predicate, signing a structure, querying an indexed database, validating a schema, calculating a checksum, applying an approved transformation, or calling a well-defined service are deterministic or bounded operations. Re-generating their implementation on every request would discard the central economic advantage software has always had: **reuse**.

The production architecture therefore changes the generated artifact. Instead of generating an entire application, the agent generates a small execution representation that composes existing, machine-legible tools.

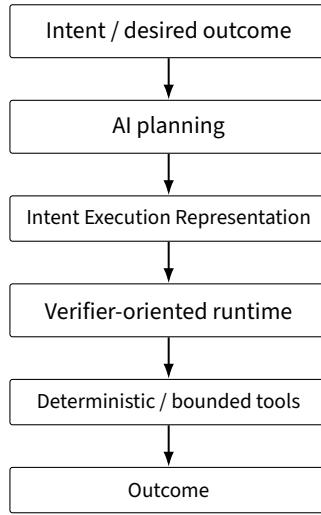


Figure 2.1: The practical architecture keeps generative freedom at the planning layer and reuses deterministic execution underneath it.

2.4 Reuse is an economic consequence, not nostalgia

Suppose a stable reusable tool costs B to design, test, and verify. Suppose full runtime synthesis carries an avoidable extra per-request cost $g > 0$ compared with invoking that reusable tool. After N equivalent uses, the avoidable synthesis overhead is Ng . In this simple accounting model, the one-time investment is amortized once the following holds.

REUSE AMORTIZATION THRESHOLD

$$N > \frac{B}{g}.$$

No empirical values are assumed here. The point is structural: whenever a stable behavior is repeated and runtime synthesis has a positive avoidable cost, repeated generation eventually loses to reuse.

This is why agentic software does not make libraries, runtimes, databases, protocols, or verified components obsolete. **It changes who composes them and when.**

2.5 The blockchain analogy: universality is not economics

There is a limited but useful analogy with blockchain architectures. A general mechanism can be conceptually powerful while still being too expensive, latent, or operationally heavy to place in the path of every computation. Distributed ledgers make this trade-off visible because consensus and replicated verification are used selectively rather than as substitutes for every local computation.

The analogy is economic, not semantic. Blockchains and agentic systems solve different problems. The shared lesson is narrower: theoretical generality does not imply that the strongest general mechanism should be used for every operation.

For agentic software, open-ended reasoning and synthesis are the expensive general mechanisms. The architecture should use them where they buy semantic leverage, then lower the result into efficient execution.

2.6 The ontological shift

The practical consequence is deeper than a new framework.

Traditional software engineering treats the application as the persistent center of gravity. Code, screens, workflows, permissions, release processes, and ownership are organized around it. Outcome-Oriented Programming makes a different object primary: the **execution** that begins with an intent and unfolds through time.

The durable engineering assets become:

- domain ontologies that give operations, outcomes, effects, entities, and evidence stable machine-readable meaning;
- tools that implement stable operations;
- Semantic Manifests and contracts that make tool behavior machine-legible;
- policies and authority models that bound permissible effects;
- formal models, proof artifacts, and small verifiers for critical invariants;
- an execution representation that can be generated cheaply and typed against the ontology;
- a runtime that binds all of these into a concrete causal execution.

This creates two independent forms of containment. **Application containment** asks whether a concrete tool is constrained by an accepted semantic contract: what state transitions and effects the implementation may validly realize. **Authority containment** asks whether this specific execution lineage is authorized to cause that operation: the role played by PIC. A tool can be behaviorally correct and still be unauthorized for the current lineage; a lineage can carry valid authority and still invoke a component whose behavioral assurance is insufficient. Production execution needs both questions answered separately.

The application does not have to vanish. It changes role. It can remain a UI, a data boundary, a tool host, a legacy workflow container, or an organizational product. What it no longer has to be is the universal unit through which every user intent, workflow, and authority relation is defined.

KEY IDEA

The ontological change is from software as a preassembled application to software as a space of verified operations from which executions are composed.

The rest of this book gives that execution a programming model, a temporal structure, an authority model, and a verification architecture.

3 Outcome-Oriented Programming

Traditional programming primarily describes procedure. Even declarative systems usually describe a desired state inside a bounded technical domain. Outcome-Oriented Programming moves the abstraction boundary outward: **the user specifies a desired result and constraints, while an agentic runtime constructs the execution capable of producing it** across available tools and systems.

The important word is *outcome*, not because every final object is known in advance, but because programming begins from conditions that should become true rather than from a pre-authored sequence of instructions. Later chapters show why some concrete outcomes can be facts of the future and must therefore be represented as predicates over an acceptable state space.

3.1 From instructions to desired state

Suppose a user says:

Get me to Rome before 09:00 tomorrow, keep the total travel cost below EUR 500, comply with company travel policy, and make sure the receipt reaches accounting.

The user has not described a procedure. The desired outcome contains state and constraints:

- arrival in Rome before a deadline;
- cost below a threshold;
- policy compliance;
- accounting receipt recorded.

The agent may search trains and flights, compare routes, consult policy, request approval, purchase a ticket, and store a receipt. Those are execution choices, not user-authored instructions.

3.2 What is being programmed?

The word *programming* still matters. An outcome cannot be handed directly to physical systems. It must be transformed into an executable representation containing operations, dependencies, conditions, error handling, evidence requirements, and validation.

The difference is who writes that representation and when. In conventional application development, programmers author the workflow ahead of time. In Outcome-Oriented Programming, an AI planner can synthesize the workflow from intent at runtime while reusing tools and constraints that programmers have already engineered.

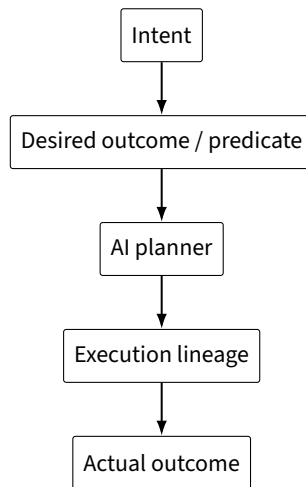


Figure 3.1: Outcome-Oriented Programming separates the desired result from the procedure used to obtain it.

💡 KEY IDEA

Outcome-Oriented Programming does not remove programming. It moves part of programming from human-authored application workflows to machine-generated execution representations while moving critical acceptance conditions into explicit runtime contracts and verifiers.

3.3 The desired separation

A mature implementation should keep three objects separate:

1. the **outcome**, which describes what must become true or what acceptable state space may be reached;
2. the **plan**, which describes how the planner proposes to make it true;
3. the **execution**, which is the concrete causal sequence that actually occurs.

Plans may change. Runtime evidence may appear later. The concrete executor may not yet exist. The outcome constraints and authority rules must nevertheless remain meaningful across those changes. This is why the programming model immediately leads to time, lineage, and continuity.

4 Intent, Goal, and Outcome

These words are often used interchangeably, but the architecture becomes clearer if they are given distinct roles.

4.1 Outcome

An **outcome** is a state of the world, or a set of admissible states, that should hold after execution. When all relevant facts are known, it can be represented directly:

```
outcome {
  arrival.city == "Rome"
  arrival.time <= "09:00"
  total_cost < 500 EUR
  travel_policy == COMPLIANT
  accounting.receipt == STORED
}
```

When future facts do not yet exist, the outcome may instead be characterized by a predicate that can only be evaluated later. The distinction is developed formally at the architectural level in the N+1 outcome chapter.

4.2 Goal

A **goal** is a planning target. An agent may decompose one outcome into multiple goals and subgoals: obtain travel options, select an eligible itinerary, secure payment authority, purchase, and archive the receipt.

Goals are useful inside the planner. *They are not necessarily the object that establishes authority.*

4.3 Intent

Intent is richer. It is an originating act by a permissioned entity or system: something is being requested, for a purpose, within a context, under constraints, and with some authority to cause effects.

In the PIC model, a permissioned entity expresses an intent within its privileges and selects the authority context that becomes the origin of a new execution lineage [1]. This gives intent two architectural consequences at once:

- on the application plane, the intent is interpreted into a desired outcome or admissible outcome predicate;
- on the authority plane, the origin selects the initial authority context C_0 that bounds what this execution may validly cause.

This is why *outcome* is the right word for the programming paradigm while *intent*

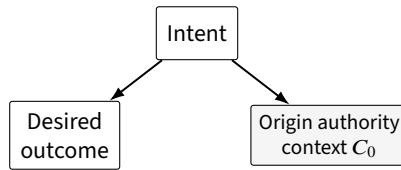


Figure 4.1: Intent originates both the desired application effect and the authority context under which it may be caused.

remains the right word for the origin of execution.

4.4 Semantic intent is a compiled interpretation

The natural-language request and the runtime’s semantic representation should not be conflated. The AI produces a candidate mapping from words to ontology-level operations, entities, constraints, and outcome predicates. That mapping is analogous to a **compilation boundary**: later verification can reason about the formal object, but *it cannot retroactively prove that the formal object captured the human’s intended meaning*. High-impact mappings can therefore require confirmation, deterministic policy checks, or governance before execution.

Design principle. Outcomes describe what should become true. Intent describes what an origin is attempting and authorized to cause. Ontology turns that intent into a machine-readable semantic coordinate, but the interpretation step remains an explicit trust boundary.

Part II

The Execution Architecture

5 From Applications to Tools

If agents are expected to compose software dynamically, they need reusable units of execution. This book calls them **tools**.

The term is intentionally plain. In security literature, *capability* already carries specific meanings associated with authority and possession. Here we need a word for application functionality that does not imply authorization. **A tool describes what software can do. Authority will be modeled separately.**

5.1 Tool versus operation

An **operation** is semantic: “purchase a ticket,” “send a message,” “reserve inventory,” “create a report.” A **tool** is a concrete implementation that can realize an operation in some environment.

That environment need not be purely software. A tool may wrap a server or cloud workload that acts machine to machine, but it may equally wrap a person who approves a request on a mobile device, an autonomous vehicle, a robot, an industrial machine, or a connected object in a home. *The architecture treats them alike*: each is an executor that the road of an execution may reach, and each is bound by the same manifest, authority, and governance rules. The common abstraction does not remove domain-specific safety controllers, real-time constraints, hardware interfaces, physical safeguards, or integration adapters; it places them behind the same rules rather than pretending they are not there.

Several tools may implement the same operation:

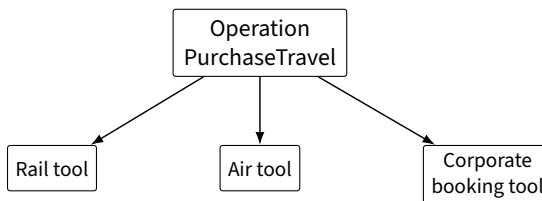


Figure 5.1: The planner reasons about operations; the runtime binds operations to concrete tools.

5.2 From API description to Semantic Manifest

A tool becomes useful to an agent when its behavior is machine-legible. An API tells a caller how to invoke software. A **Semantic Manifest** tells a planner and runtime what the tool claims to mean, what it may change, and what evidence exists for those claims.

A compact manifest can reference a stronger executable or formal contract:

```
tool PurchaseTicket {  
  ontology.operation: travel.purchase  
  inputs: itinerary, traveler, payment_method  
  preconditions: itinerary.available  
  effects: booking.created, payment.initiated  
  postconditions: booking.status == CONFIRMED  
  invariants: charged_amount <= approved_amount  
  compensation: cancel_booking  
  latency_hint: 1-5s  
  cost_hint: external_fee  
  idempotency: supported  
  assurance: contract_ref + verification_evidence  
}
```

The manifest is not itself a proof. Its semantic claims are trustworthy only to the extent justified by the associated implementation, verification, attestation, tests, refinement argument, or runtime evidence. A signed manifest can establish facts about the signed manifest and signer under the trust model; it does not establish that the implementation satisfies the manifest.

Notice what is also missing: the manifest does not say that the current execution is allowed to perform the purchase. *That is not a property of the tool. It is a property of the execution lineage and its authority context.*

5.3 Relation to semantic service descriptions

Machine-readable service semantics are not a new idea, and this book does not claim otherwise. OWL-S, for example, described Web services through semantic profiles, process models, and grounding so that software could reason about what a service does and how to invoke it [14]. The Semantic Manifest proposed here occupies related territory but is narrower in some dimensions and broader in others: it is an execution-facing contract surface for an agentic runtime, tied to ontology-level operations, effects, invariants, failure and compensation semantics, and explicit assurance evidence. The architectural claim is therefore not the invention of semantic service description; it is the use of machine-readable semantics as one layer in a verifier-oriented execution architecture that also separates application containment, authority continuity, and governance.

5.4 Why developers still matter

A reliable tool can encapsulate years of engineering knowledge: API quirks, retry behavior, idempotency, validation, data mapping, transaction boundaries, and observability. Agentic software does not make that work disappear. It changes the consumer of the abstraction.

A framework is traditionally written for another programmer. A tool in this architecture is increasingly written for a runtime and planner.

Design principle. Programmers should package stable execution as tools with explicit contracts instead of forcing agents to regenerate stable implementation.

6 Semantic Manifests and Ontological Execution

If runtime composition replaces part of pre-authored application design, the planner and the tools need a common language of meaning. **Natural-language similarity is not a sufficient execution contract.** The architecture proposed in this book therefore introduces a **semantic ontology**: a controlled machine-readable vocabulary and relation system for outcomes, operations, entities, effects, evidence, and policy-relevant concepts.

The word *ontology* is used here in a deliberately practical sense. It does not require a universal model of the world. It can be domain-scoped. A travel ontology may define concepts such as `travel.search`, `travel.purchase`, `booking.confirmed`, `cost.amount`, and `policy.compliant`; a finance ontology will use a different vocabulary. What matters is that intent, IER operations, tool manifests, evidence, and policy can refer to shared semantic coordinates.

6.1 Artifact identity is not behavior

A modern software supply chain can hash an artifact, sign a release, authenticate a publisher, and record provenance. These controls answer important questions about bytes, signer, and release path. They do not answer a different question:

What state transitions can this program cause when executed under these conditions?

A hash is a digest of bytes and becomes an integrity check only relative to a trusted expected value. A digital signature authenticates signed data relative to a key and trust model. **Neither mechanism is a semantic proof of program behavior.** Agentic execution therefore needs to carry software identity *and* behavioral semantics as separate dimensions.

6.2 A Semantic Manifest

For a tool T , define the book-level manifest abstraction as follows.

SEMANTIC MANIFEST OF A TOOL

$$M_T = (\Omega_T, \text{In}_T, \text{Pre}_T, \text{Eff}_T, \text{Post}_T, \text{Inv}_T, \text{Comp}_T, \mathcal{A}_T),$$

where Ω_T identifies the ontology operations the tool implements; In_T admissible inputs; Pre_T preconditions; Eff_T effect classes; Post_T postconditions; Inv_T invariants; Comp_T compensation semantics when defined; and $\mathcal{A}_T$ references available assurance evidence.

This tuple is an architectural proposal, not a claim about PIC. A real implementation may split these fields across schemas, contracts, policy documents, proof artifacts,

attestations, runtime monitors, and external evidence.

6.3 From prompt to semantic intent

The first AI step remains probabilistic. Given a prompt, the model proposes an interpretation containing intent, outcome predicates, constraints, and semantic operations. That proposal should not be treated as self-authenticating truth.

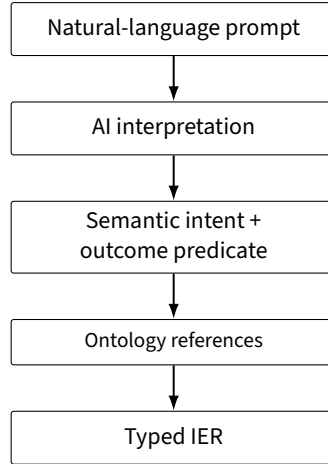


Figure 6.1: Natural language is interpreted once, then lowered into explicit semantic objects that later execution can check.

For high-risk cases, the semantic interpretation itself may require confirmation, policy validation, or a gate. Formal verification downstream cannot prove that an incorrectly interpreted human request was the request the human actually meant.

6.4 Semantic tool binding

Suppose a step of the Intent Execution Representation (IER, Chapter 7) requests ontology-level operation ω_i . The runtime should bind a tool T only if the ontology and manifest establish an acceptable semantic match. At the architectural level,

$$\text{SemanticMatch}(\omega_i, T)$$

means that the tool manifest implements the requested operation with compatible input, effect, and contract semantics under the domain’s ontology and binding rules.

This is stronger than vector similarity or a model saying “this tool looks relevant.” The AI can nominate candidates; the runtime applies a deterministic or formally specified matching rule where the domain allows it.

6.5 The two containments

The semantic layer creates a clean separation between two kinds of safety:

Application containment	containment	Constrains what a selected tool implementation can validly do relative to its semantic contract and explicit assumptions.
Authority containment	containment	Constrains what the current execution lineage may validly cause relative to its origin authority context and PIC continuity.

Neither substitutes for the other. Application containment without authority can produce a perfectly correct unauthorized action. Authority containment without application assurance can authorize a tool whose actual behavior is insufficiently constrained.

A useful book-level admissibility relation is the following.

ADMISSIBILITY OF AN EXECUTION STEP

$$\text{Admissible}_i = \text{SemanticMatch}_i \wedge \text{ApplicationAccepted}_i \\ \wedge \text{AuthorityValid}_i \wedge \text{PolicyAcceptable}_i.$$

This is an architectural composition, not a theorem already proved by PIC. PIC supplies the authority-continuity component; semantic matching and application assurance require separate models and evidence.

Design principle. Intent and outcome give execution meaning. Ontology gives that meaning a machine-readable coordinate. Semantic Manifests constrain what tools claim to implement. Formal and runtime verification determine which claims are acceptable. PIC independently constrains what the lineage may cause.

7 The Intent Execution Representation

Natural language is a poor execution format. It is ambiguous, verbose, difficult to validate, and difficult to replay deterministically. Once an agent has understood an intent, it should lower that understanding into a smaller machine-oriented representation.

This book calls that representation the **Intent Execution Representation**, or **IER**.

7.1 Not necessarily a language

IER does not need to be a human-friendly programming language. It could be a graph, an AST, a structured object, a bytecode-like form, or a constrained DSL. **Its purpose is to bridge probabilistic planning and controlled execution.**

An illustrative textual form might look like this:

```
intent travel_to_rome {
  outcome:
    arrive(city="Rome", before="09:00")
    total_cost < 500 EUR
    policy == compliant
    receipt == stored

  plan:
    options = travel.search(destination="Rome")
    eligible = policy.filter(options)
    choice = optimize(eligible, objective=min_cost)
    gate if requires_purchase_authority(choice)
    booking = travel.purchase(choice)
    accounting.store_receipt(booking.receipt)
}
```

The point is not the syntax. The point is that an AI can produce a compact semantic program instead of producing every implementation behind `search`, `filter`, `purchase`, and `store_receipt`. In a verifier-oriented runtime, those operations resolve to ontology terms and contract references rather than exist only as free-form names invented by the model.

7.2 What IER must make explicit

A useful representation should expose at least:

- desired outcome and constraints expressed as semantic predicates where possible;
- ontology references for operations, entities, effects, and evidence;
- operation dependencies;
- tool binding or tool-selection criteria tied to Semantic Manifests;
- side effects;

- checkpoints and gates;
- compensation or rollback behavior;
- terminal success and failure conditions;
- data and authority requirements relevant to execution;
- verification obligations: which tool contracts, invariants, authority conditions, and evidence predicates must hold before each step is accepted, and at what level of assurance, from a runtime check to a formal proof.

The last item is what makes the representation checkable rather than merely readable. A step that carries its own obligations can be accepted or rejected by a deterministic verifier without asking the model; where the risk justifies it, those obligations can be discharged by formal methods. Part IV (Chapters 31 and 33) develops this into proof-carrying plans.

The representation should be easy to validate before the runtime crosses an irreversible boundary.

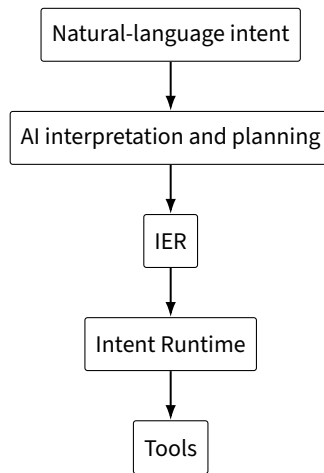


Figure 7.1: IER is the boundary between open-ended reasoning and controlled execution.

7.3 Why machine-oriented code can be better

If the primary author of the representation is an AI and the primary consumer is a runtime, human readability is no longer the only design criterion. Low ambiguity, compactness, verification, constrained generation, typed effects, and safe composition may matter more.

This does not mean humans lose visibility. It means humans may review a semantic projection of IER rather than every low-level instruction. That idea becomes important later when we discuss semantic review.

8 Execution Happens in the Future

Outcome-Oriented Programming becomes much easier to understand when execution is drawn on a time axis instead of inside an application box.

The public PIC material traces the motivating problem to stream processing. In an Apache Kafka pipeline, authority had to reach a consumer that was not yet known. Passing a bearer token through the stream appeared natural, but the future consumer might not yet exist, might receive the message after the artifact expired, or might not hold the key required to use it. Giving that consumer its own credentials solved delivery, but created a different problem: the consumer could now possess broader authority than the individual execution it was serving. Application code was left to select the correct authority [2]. Both attempts made the transport carry the authority decision, the shortcut the opening pages warned against: the stream delivered the message and was implicitly expected to vouch for what the consumer could do.

The architectural lesson is larger than Kafka. Distributed programs have always separated cause from later execution:

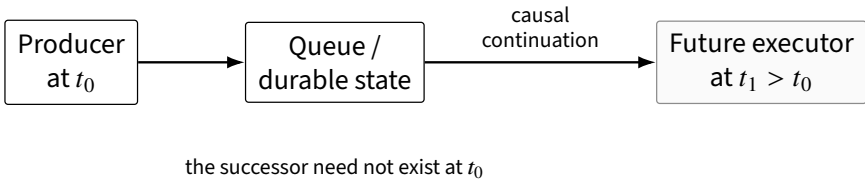


Figure 8.1: Distributed execution is a causal sequence across time, not merely a list of configured positions.

An agentic runtime pushes this pattern further. The next tool may be selected dynamically. A downstream agent may be provisioned later. A vendor may publish an offer after delegation. A risk score may change. **The concrete outcome may therefore be a fact of the future rather than an object available at planning time.**

Design principle. Programming an outcome means programming across time. The runtime must preserve meaning when the next executor, the next evidence, or the concrete next outcome does not yet exist.

8.1 Causal time is not expiration time

The temporal coordinate relevant to execution continuity is not merely a timestamp or token expiration. It is causal order:

$$s_0 \prec s_1 \prec s_2 \prec \cdots \prec s_n.$$

PIC makes this distinction explicit: two actions may involve the same principal and the same privilege and still be different occurrences because they descend from different causes [1]. Time enters authority through the execution lineage, not merely through a clock field.

Time in authority is not only expiration. It is causal continuity.

9 The N+1 Unknown Executor Problem

At step n , the concrete executor of step $n+1$ may not yet be known, selected, instantiated, or provisioned. The PIC model is explicitly designed for this case [1, 2].

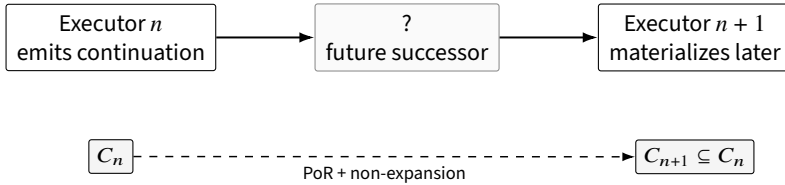


Figure 9.1: The future holder is not the invariant. The valid continuation is.

The consequence is subtle. **Authority cannot depend on pre-binding the full future chain to concrete holders.** What must survive the gap is the ability of the future receiving boundary to verify that the new state is a valid continuation of the predecessor and that authority has not expanded.

This yields three requirements stated in the public PIC material:

- authority flows from an origin inside a lineage rather than being recreated by a later workload;
- every hop proves continuity rather than relying on possession alone;
- execution is temporal and causal, not positional.

For Outcome-Oriented Programming, this means an IER should not be interpreted as a fully pre-bound topology. It is closer to a constrained execution program whose concrete executors may be resolved when the relevant future state exists.

10 The N+1 Outcome on Unknown Facts

The future uncertainty is not limited to executors. The public PIC material identifies a second case: the authority needed at step $n + 1$ may depend on a fact that did not exist when step n was authorized [2].

Suppose a user delegates a dinner-booking task with a strict budget and a conditional exception: a small increase is acceptable only if a live concert is included in the same price with no separate cover charge. At signing time, the qualifying offer may not exist. The restaurant may create it later while the agent is already executing.

The wrong abstraction is to let the agent decide that a future exception is reasonable. The PIC model instead separates a signed predicate from later runtime evidence:

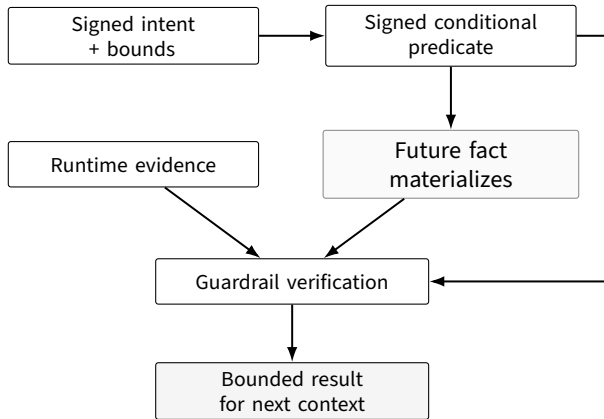


Figure 10.1: The unknown outcome is not pre-guessed. A signed case waits for evidence and closes when the fact exists.

This gives Outcome-Oriented Programming a more precise vocabulary.

Desired outcome. The state or family of states the origin wants to make true.

Outcome predicate. A condition that characterizes acceptable concrete outcomes, including constraints and signed trade-offs.

Concrete outcome. The particular state that becomes available only after runtime facts materialize.

Runtime evidence. Evidence used by a receiving or governance boundary to decide which signed case the concrete outcome satisfies.

A desired outcome therefore need not be a pre-known object. It may be an acceptable region over a future state space.

Let W denote possible world states. A minimal abstraction is an outcome predicate over that space.

OUTCOME PREDICATE

$$\Phi : W \rightarrow \{\text{true}, \text{false}\}.$$

The planner may search for a candidate $w \in W$, but the candidate becomes acceptable only when the runtime can establish the evidence required to evaluate $\Phi(w)$ under the applicable policy. This equation is a book-level architectural abstraction, not a theorem of PIC.

Design principle. The agent may discover the future. It does not get to authorize the future. Prediction proposes; evidence closes the decision.

11 The N+1 Invalid State Problem

A receiving executor sees the state it receives. It cannot inspect the complete internal history of the previous executor, prove that its memory was correctly separated, or infer whether a bug mixed two concurrent authority contexts. The PIC threat model therefore refuses to use the predecessor’s internal correctness as evidence for the next boundary [2].

The security target is not “make executor n bug-free.” No protocol can guarantee that. The target is narrower and stronger at the boundary:

Executor n must not be able to hand executor $n + 1$ an invalid authority state that nevertheless validates.

Consider two independent lineages hosted by the same long-running agent:

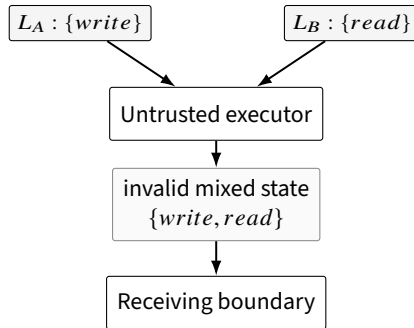


Figure 11.1: The invalid-state problem is not whether mixing can be attempted, but whether a mixed state can be accepted as a valid continuation.

PIC does not eliminate the upstream bug. Its claim is that, under a conforming receiving predicate and its stated assumptions, a state that expands the predecessor authority or misattributes authority across lineages is not valid PIC state. The receiving check remains deliberately mechanical: verify the predecessor relationship, lineage binding, required request/integrity/freshness conditions, and non-expansion [2].

11.1 Same apparent authority can encode different authorization

The invalid-state problem is deeper than a malformed set of privileges. Different causal histories can collapse to the same apparent authority view when lineage is forgotten. The PIC formal model makes this precise by representing an authority event as an occurrence inside a lineage and proving that a policy invariant under lineage cannot distinguish two occurrences with the same privilege coordinates but different lineages [1].

For example, consider three authorization instances:

$$E_1 : \text{Alice} \rightarrow \text{Bob} : \{\text{read}\},$$
$$E_2 : \text{Alice} \rightarrow \text{Bob} : \{\text{write}\},$$
$$E_3 : \text{Alice} \rightarrow \text{Bob} : \{\text{read}, \text{write}\}.$$

The set-theoretic union of the first two authority values can look identical to the authority value of the third. The authorization histories are not identical. E_1 and E_2 are independent occurrences; E_3 is one occurrence whose origin authorized the combined authority. Any policy that needs to distinguish those histories must read more than the final authority set.

This matters operationally because the independent executions may have different purposes, lifetimes, provenance, revocation conditions, or policies governing whether they may ever be combined. **Flattening them into one authority value destroys information that later policy may need.**

Design principle. Same apparent authority does not imply the same authorization. Authority values can be equal while their causal security meaning is different.

This pattern is directly relevant to AI. A model can hallucinate a tool choice. It can confuse two tasks. It can propose an invalid continuation. The architectural objective is not to trust that the model will never do so. **It is to prevent selected classes of invalid proposals from crossing a deterministic acceptance boundary.**

12 Execution Lineage

A generated plan is not yet an execution. Execution happens over time as concrete events cause later events. The architecture needs a first-class representation of that causal structure.

Execution lineage is a finite causally ordered path of execution occurrences beginning at an origin. For the core PIC model used in this book, one lineage is linear.

EXECUTION LINEAGE

$$L = \langle s_0 \prec s_1 \prec s_2 \prec \cdots \prec s_n \rangle.$$

Each s_i is an occurrence, not merely a function name. Two identical calls at different times can belong to different lineages and therefore have different meaning. An execution system may fan out from one checkpoint; in that case the branches are distinct lineage extensions that share a causal prefix. The runtime may therefore form a branching execution graph even though each individual lineage remains a linear causal history. Multi-lineage joins are treated separately as explicit composition events.

12.1 Lineage is not topology

A lineage can cross boundaries that application architecture usually treats as separate:

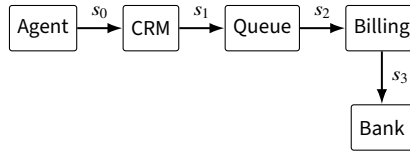


Figure 12.1: One lineage can cross agents, services, queues, and external systems.

A downstream executor may not even exist when an upstream step occurs. The PIC paper explicitly models execution as a sequence of causal steps in time rather than fixed positions in a topology [1]. That property is important for asynchronous agentic systems.

12.2 Lineage is not identity propagation

The same user identity may appear in many executions. The same service identity may process many users. **Repeating an identifier does not prove that a later action is a causal continuation of the original intent.**

Lineage answers a different question:

Which earlier occurrence caused this occurrence to exist?

That causal coordinate is the basis on which authority continuity can later be evaluated.

12.3 The unit of reasoning changes

Application-centric systems often reason about requests inside application boundaries. Outcome-oriented systems need to reason about a whole execution that may cross those boundaries. This produces one of the central shifts of the book:

💡 KEY IDEA

The natural unit of agentic execution is not the application process. It is the causal lineage created by an intent.

13 Two Planes, One Execution

The execution lineage is easier to understand if the same events are viewed through two parallel planes.

The **application plane** asks:

What is this execution doing?

The **authority plane** asks:

What is this execution allowed to cause, and why?

They are not two workflows. They are two projections of the same occurrences.

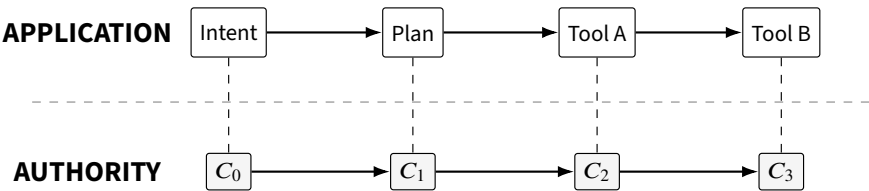


Figure 13.1: Two planes, one execution: application events and authority contexts correspond to the same lineage.

13.1 A single event, multiple coordinates

Conceptually, an execution event can be described as a tuple.

EXECUTION EVENT

$$E_i = (L, I, \omega_i, T_i, M_{T_i}, C_i, \varepsilon_i)$$

where L is the lineage, I the originating intent, ω_i the ontology-level operation, T_i the selected tool, M_{T_i} its Semantic Manifest, C_i the authority context carried by that occurrence, and ε_i the evidence observed or produced at the boundary.

The application projection focuses on $(I, \omega_i, T_i, M_{T_i}, \varepsilon_i)$. The authority projection focuses on (L, C_i) . The event is the same. Governance can read evidence from both projections without merging their semantics.

13.2 Why this matters

Security is often layered onto an application after the application model has already been chosen. The two-plane view makes authority co-equal with application execution from the beginning. Every significant action has both an operational meaning and an authority meaning.

The application plane can itself be made increasingly strict: ontology-based binding identifies the intended operation, Semantic Manifests describe the permitted behavior

of candidate tools, and formal or runtime checks determine whether a tool invocation is acceptable. This is **application containment**. The authority plane remains separate: PIC determines whether this lineage may cause the operation. This is **authority containment**.

This also makes it possible for the authority model to span software stacks without becoming coupled to their internal implementation.

Design principle. Application architecture constrains what the selected software may validly do. Authority architecture constrains whether this lineage may cause it. They meet at the same execution event but establish different properties.

14 The Application Plane

The application plane turns a desired outcome into concrete effects.

14.1 Planning

The planner decomposes outcomes into operations. It may choose among alternative strategies, run searches in parallel, request more information, or revise the plan after observing intermediate results.

The planner is allowed to be probabilistic. **The runtime should not confuse probabilistic planning with unrestricted execution.**

14.2 Tool matching

For each ontology-level operation, the runtime identifies one or more tools whose Semantic Manifests claim a compatible implementation. Matching can consider:

- ontology compatibility and operation refinement;
- input and output types;
- preconditions, postconditions, effects, and invariants;
- assurance level and available proof or validation evidence;
- latency and cost;
- location or jurisdiction;
- reliability;
- transaction and compensation support;
- data locality;
- policy constraints.

This creates a useful distinction between *planning* and *binding*. The agent can reason about the operation “send a compliant invoice” without deciding immediately which invoicing system will implement it. The runtime can require both a semantic match and a minimum assurance profile before a candidate becomes executable.

14.3 Application containment

For a tool whose implementation is related to a formal contract, the strongest useful statement is not merely that the tool has a particular name, but that its accepted execution respects a specified state-transition relation. If the tool executes as $\text{Exec}_T(w, x) = (w', y)$, then a verified implementation may establish, under its explicit assumptions, the following contract.

APPLICATION CONTAINMENT CONTRACT

$$\text{Pre}_T(w, x) \Rightarrow (\text{Post}_T(w, x, w', y) \wedge \text{Inv}_T(w, w')).$$

For opaque external services, that implication may not be formally provable; the runtime may instead rely on narrower schemas, attestations, monitored effects, or signed external evidence. The architecture should preserve this distinction rather than label all tools “verified.”

Application containment therefore means that the runtime accepts only tool behaviors justified by the assurance mechanism required for that operation. It does not mean that every real-world side effect is mathematically observable or proven.

14.4 Outcome validation

Execution is not complete merely because all planned calls returned success. The runtime should evaluate the actual state against the desired outcome.

```
validate outcome:
  trip.arrival <= 09:00
  trip.total_cost < 500 EUR
  trip.policy == compliant
  accounting.receipt == stored
```

If the final state does not satisfy the outcome, the planner may need to continue, compensate, or report partial completion.

14.5 Failure is part of the program

Dynamic composition requires explicit failure semantics. Tools should declare whether they are retryable, idempotent, compensatable, or irreversible. The IER should make boundaries visible before execution crosses them.

This is not merely operational hygiene. **A gate placed before an irreversible effect is much more meaningful than a gate placed after it.**

15 The Authority Plane

The authority plane tracks what the current execution is entitled to cause.

PIC begins from a permissioned origin. When an entity expresses an intent, a subset of the authority available to that origin becomes the authority context C_0 for a new lineage [1]. Each later step carries an authority context C_i .

When adjacent steps use the same authority vocabulary, the essential monotonic rule is non-expansion.

NON-EXPANSION OF AUTHORITY

$$C_{i+1} \subseteq C_i.$$

When the steps use different local authority vocabularies, the literal subset relation is not type-correct. Let

$$T_{i \rightarrow i+1} : \mathcal{P}(O_i \times R_i) \rightarrow \mathcal{P}(O_{i+1} \times R_{i+1})$$

be the policy translation into the successor vocabulary. The corresponding book-level condition is

$$C_{i+1} \subseteq T_{i \rightarrow i+1}(C_i),$$

with soundness depending on the translation itself [1]. In either case, **a valid continuation cannot silently acquire authority that is not justified by the predecessor** under the applicable authority mapping.

15.1 Relationship and continuity

PIC separates a local relationship from global continuity. A **Proof of Relationship** establishes that a successor step is a valid causal continuation of its immediate predecessor. **Proof-of-Continuity** is the transitive composition of those relationships together with non-expansive authority propagation across the lineage [1].

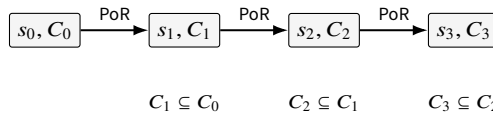


Figure 15.1: Same-vocabulary case: authority is carried by a causally linked, non-expansive execution lineage.

15.2 Authority to cause

A downstream service may perform an operation the origin could not perform directly in the same local vocabulary. The relevant question is whether the origin was authorized to *cause* the effect through the execution. PIC explicitly models this distinction and

allows policy translation between heterogeneous operation spaces [1].

That makes PIC suitable for an agentic world in which a human intent can be realized through many services with different local permission models.

The semantic ontology introduced on the application plane must not be confused with PIC's policy translation between heterogeneous authority vocabularies. The first maps application meaning so a planner and runtime can identify operations and tool semantics. The second is an authorization mapping that determines what downstream authority corresponds to upstream authority under policy. They may refer to related domain concepts, but they are different functions with different proof obligations. An unsound authority translation remains a policy error even if the application ontology is perfectly designed.

15.3 Relation to capabilities, attenuation, and execution history

Neither capability security nor monotone attenuation begins with PIC. Capability systems have a long history of coupling designation and authority, and object-capability work explains how that structure addresses the classical confused-deputy problem at invocation boundaries [15, 16, 29]. Macaroons provide a concrete credential construction with contextual caveats and monotone attenuation [18]. History-based access control likewise makes authorization depend on execution history rather than only static possession [17].

PIC's stated contribution is narrower: it makes the causal lineage of a discrete multi-hop execution an explicit authorization coordinate and requires each accepted continuation to remain causally related and non-expansive. A capability system that preserved this occurrence-sensitive continuity across the whole chain could satisfy the same temporal requirement; PIC is not presented as a replacement for capability security. Likewise, attenuation alone does not establish which execution a later use belongs to unless the construction preserves enough occurrence or lineage context to distinguish those uses [1].

15.4 Authority is not possession, and possession is not composition

An executor can hold many credentials and still be constrained in a particular lineage. The authority plane asks which authority belongs to the execution that caused the current action, not which unrelated credentials happen to be available in the process.

The same distinction applies when several valid authorities coexist. Possessing C_A and C_B does not, by itself, authorize an executor to construct $C_A \cup C_B$ for the current task. **A union of authority values is a mathematical set operation; whether those authorities are allowed to participate in one execution is a security-policy question.** If such participation is allowed, the composition must be explicit and its causal parents must remain identifiable.

This is the crucial bridge between agent planning and admissible action. A planner

may discover that two authorities would be useful together; the runtime still needs a separately justified decision that they are allowed to meet in this execution.

Design principle. Possession does not authorize composition. Composition is a security decision.

16 One Lineage, Many Stacks; One Stack, Many Lineages

The two-plane model produces a powerful symmetry.

16.1 One lineage can cross many stacks

A travel outcome may cross an assistant, a policy service, a booking provider, a payment processor, and an accounting system. These systems may be written by different organizations in different languages and operate under different identities. From the application's point of view, they are separate stacks. *From the lineage's point of view, they are consecutive occurrences caused by one origin.*

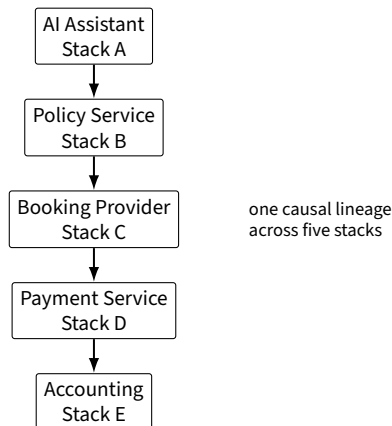


Figure 16.1: Application boundaries do not need to be authority boundaries.

PIC is deliberately about continuity of authority rather than propagation of identity. Its paper notes that each hop may be a service, workload, function, tool, or agent; what must persist is continuity of authority, not the same identity representation [1].

16.2 One stack can process many lineages

The inverse is equally important. A payment service may simultaneously process unrelated executions:

Those lineages may involve the same principal and even the same nominal privilege. PIC's temporal model distinguishes occurrences by causal lineage; the lineage coordinate separates executions that possession alone would make appear identical [1].

Design principle. Applications host execution. Lineages define causality. Authority follows the lineage, not the application.

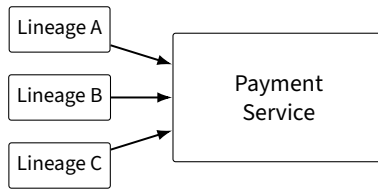


Figure 16.2: A single software process may host many independent authority lineages.

17 Tools Are Not Authority

This chapter states a distinction that the whole architecture depends on.

A tool says: the system can do X.

Authority says: this lineage may cause X.

The first is application functionality. The second is an authorization statement about a specific execution occurrence.

17.1 Why the distinction matters

Suppose an agent process has access to four tools:

- read a document;
- delete a document;
- send email;
- initiate a payment.

A user asks, “Summarize this document.” The existence of the delete, email, and payment tools must not imply that this particular execution may use them. The plan can only cross into an effect if the authority plane allows that effect as a valid continuation of the originating context.

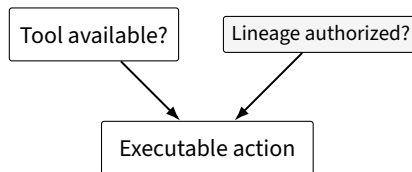


Figure 17.1: Execution requires both application ability and lineage-specific authority.

A compact conceptual rule is the following.

EXECUTABLE ACTION

$$\text{Executable}(a, L) = \text{SemanticMatch}(a, T) \wedge \text{ApplicationContained}(T, a) \\ \wedge \text{Authority}(L, a) \wedge \text{Policy}(a).$$

The exact implementation may add data constraints, risk conditions, budgets, evidence obligations, or environmental predicates. The structural point remains: semantic ability, application assurance, and lineage authority are distinct inputs. **A signed or available tool is not automatically semantically suitable; a semantically correct tool is not automatically authorized.**

17.2 Why this avoids capability terminology

In some security models, a capability is itself an authority-bearing object. That is a valuable and well-established concept, but it is not what this book means by reusable application functionality. Using *tool* keeps the application plane semantically neutral. PIC then supplies the authority dimension across the execution lineage.

Part III

Authority and Governance

18 PIC: Authority Follows the Lineage

PIC – Provenance Identity Continuity – is the authority model that makes the two-plane architecture coherent across multiple execution steps.

Its core insight is temporal. Possession tells us what authority an entity or artifact can exercise at a point. A multi-step execution also needs to know whether the authority used now is a valid continuation of the authority that originated the execution.

18.1 The origin context

At the beginning of a lineage, the origin expresses an intent within some authority. That selected authority becomes C_0 . Later contexts may be equal or narrower, but a valid PIC execution does not expand beyond the origin.

ORIGIN-BOUNDED AUTHORITY

$$C_n \subseteq C_{n-1} \subseteq \dots \subseteq C_0.$$

Within the stated PIC model, the non-expansion result follows directly: a privilege absent from the origin context cannot appear later in the same valid lineage [1].

18.2 Causality is part of authorization

Monotonic sets alone are not enough. The steps also need to be causally related. PIC therefore requires both relationship and non-expansion at each transition. Relationship is local; continuity is global [1].

This gives authority a different ontology from “the agent has a token.” **A tool call is valid because it occurs inside a lineage whose authority continues from an authorized origin.**

18.3 Why this matters for AI agents

An agent may hold multiple authority sources at once: service credentials, delegated user credentials, tool permissions, or organization-level privileges. The PIC paper highlights exactly this condition for AI agents and shows how an agent-influenced parameter can recreate the confused-deputy problem across distributed services [1].

A prompt injection that asks an agent to exfiltrate or delete data may produce a syntactically valid plan and may find a technically available tool. Under a lineage-aware authority model, *those facts are insufficient*. The action must also be a valid continuation of what the origin was entitled to cause.

Design principle. The authority question is not “What can this agent access?” It is “What may this execution, caused by this origin, continue to cause?”

19 Provenance Is Not Continuity

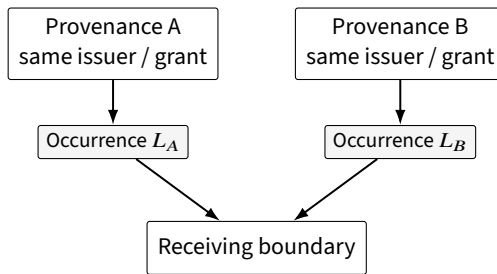
Provenance and continuity answer different questions.

Provenance. Where did this authority come from?

Continuity. Does this authority belong to this concrete execution occurrence as a valid continuation of its predecessor?

A chain of valid issuers, subjects, scopes, audiences, or holder bindings can establish a rich provenance story. It does not automatically individuate two concurrent occurrences of the same grant. The PIC formal model makes the lineage coordinate explicit precisely because identical privilege values can occur in different causal executions [1].

This distinction is especially important for long-running agents. The same agent identity, the same user, the same resource, the same tool, and even the same privilege set may be present in two independent tasks. **Equal authority values do not make the tasks interchangeable.**



the receiver must distinguish occurrences, not only grants

Figure 19.1: Provenance can be identical while execution continuity differs.

The design consequence is that a self-asserted execution identifier is not enough by itself. *An untrusted executor saying “this is lineage A” is still only an assertion from the component whose authority attribution is under examination.* The receiving predicate needs evidence that binds the presented continuation to its predecessor under the applicable construction.

Design principle. Provenance explains ancestry. Continuity establishes occurrence-sensitive continuation.

20 Possession Is Not Composition

The distinction between possession and continuity has a second consequence: **independently valid authority cannot be silently merged merely because one executor happens to hold it at the same time.**

20.1 Same authority, different lineage, different security meaning

The PIC formal model represents an event as an occurrence exercising a privilege inside a lineage. If the lineage coordinate is projected away, two distinct occurrences can collapse to the same possession view. The paper formalizes this as a projection p that forgets lineage and proves that every policy that factors only through that projection gives the same verdict to equal privilege values even when their lineages differ [1].

For an execution-level policy this yields a simple information-loss result. Let h_1 and h_2 be two execution histories and let $q(h)$ be the authority view retained after lineage information is discarded.

INFORMATION LOSS UNDER LINEAGE PROJECTION

$$q(h_1) = q(h_2) \quad \text{but} \quad P(h_1) \neq P(h_2)$$

If the projected views are equal but the intended policy requires different verdicts, then **no decision function that reads only $q(h)$ can implement P correctly for both histories.** The reason is immediate: equal inputs to that function must produce equal outputs. *The missing information is not more authority; it is the causal coordinate needed to distinguish the two cases.*

This is the mathematical core of the phrase:

Same authority. Different lineage. Different security meaning.

20.2 A temporal reading of the confused deputy

Hardy's original confused-deputy formulation concerns a deputy that applies authority in a way not justified by the request that caused the action [29]. The PIC paper explicitly recasts the underlying authority/causality mismatch on a temporal axis: an action can be spatially valid because the executor possesses the privilege, yet temporally invalid because that privilege does not belong to the execution lineage that caused the action [1].

For compactness, this book calls that pattern a **temporal confused deputy**: authority that is valid in itself is exercised inside an execution whose causal lineage does not justify its use. This is a label for the temporal reading formalized by PIC; it is not attributed to Hardy as his original terminology.

Same authority. Wrong causal lineage.

Under the stated PIC model, the mismatch cannot appear as valid behavior inside one accepted lineage: Proof of Relationship binds the hops causally and non-expansion keeps later authority origin-bounded [1]. A physical executor may still attempt the action; the claim is that the action cannot validate as a PIC continuation when the required authority does not belong to the causing lineage.

20.3 Explicit execution composition

Composition is still useful and often necessary. **The architectural requirement is that it be explicit.**

Suppose two independent lineages carry contexts (L_1, C_1) and (L_2, C_2) . An executor may physically possess both. The book does *not* define the composition as the automatic union $C_1 \cup C_2$. Instead, it models composition abstractly as a separately authorized transition.

EXPLICIT EXECUTION COMPOSITION

$$\Gamma((L_1, C_1), (L_2, C_2), e, P) \Downarrow (L_\star, C_\star),$$

where e is the evidence available at the composition boundary and P is the applicable composition policy. The judgment means that policy has explicitly approved a new execution context $L_\star$ in which selected authority from the contributing lineages may participate.

The output context must still be bounded. Let $T_{1 \rightarrow \star}$ and $T_{2 \rightarrow \star}$ translate each parent authority into the target vocabulary when such translation is required, and let A_1 and A_2 be the portions actually admitted by composition policy.

AUTHORITY BOUND OF A COMPOSED CONTEXT

$$A_1 \subseteq T_{1 \rightarrow \star}(C_1), \quad A_2 \subseteq T_{2 \rightarrow \star}(C_2), \quad C_\star \subseteq A_1 \cup A_2.$$

For parents already expressed in the target vocabulary, the corresponding T may be the identity map. **The union above is an upper bound on authority available to the composed execution, not an automatic authorization to use every element of both parents.** Policy may narrow it further, and the runtime must preserve which parent justified each admitted contribution. If $C_\star$ contains authority outside this policy-approved bound, that authority requires an explicit new origin or another separately justified authority construction; it is not supplied by composition itself.

These equations are a book-level proposal for making the composition safety obligation explicit. They are not a theorem of the current linear PIC model; a concrete protocol must define and verify the translations, selection policy, parent binding, revocation, and acceptance semantics.

The crucial properties are structural:

- L_1 and L_2 remain distinct causal parents rather than being erased by possession;
- the fact that both authorities are available does not itself authorize their combination;

- policy can restrict which portions of each context may participate;
- the composition decision can have its own lifetime, purpose, evidence, revocation, and governance conditions;
- ordinary continuation inside either parent lineage remains conceptually distinct from the new composed execution.

This is an extension of the architecture, not a claim that the current linear PIC formalization already proves arbitrary multi-lineage composition. A concrete construction would need a formal semantics for Γ , composition-policy soundness, parent-lineage preservation, revocation behavior, and the relationship between the new context and its contributing authorities.

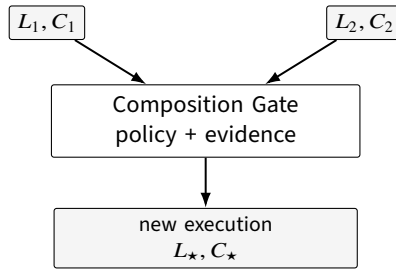


Figure 20.1: Independent authority participates in one execution only after an explicit composition decision; the parent lineages remain distinct inputs to that decision.

Design principle. Composition tells us what authorities may participate together. Continuity tells us why each authority belongs to the execution that is being continued.

20.4 Composition has its own lifetime and revocation semantics

Keeping the parent lineages distinct is not only an audit preference. It preserves policy dimensions that may differ even when the apparent authority values overlap. L_1 and L_2 may have different purposes, expiration conditions, evidence requirements, revocation states, or organizational owners. The permission to combine them can itself be narrower and shorter-lived than either parent authority.

A book-level policy abstraction is the following.

VALIDITY OF A COMPOSITION OVER TIME

$$\text{ComposeValid}(t) = \text{Valid}(L_1, t) \wedge \text{Valid}(L_2, t) \wedge \text{GrantCompose}(L_1, L_2, t) \wedge \text{Policy}(t).$$

The expression is intentionally schematic. It states a separation of decisions, not a concrete revocation protocol. A deployment may need to invalidate L_1 , invalidate L_2 , or revoke only the right to compose selected authority from them. If the two histories have already been flattened into one undifferentiated possession state, those distinctions may no longer be recoverable from that state alone.

This is another reason that explicit composition belongs at the execution level: the object

being authorized is not merely a set union but a relationship between independently caused authorities under a particular policy and time.

21 From Existing Authority to Continuity

Outcome-Oriented Programming does not require organizations to discard existing authentication and authorization infrastructure. The public PIC-X material describes an integration direction in which existing authority is validated and converted into an initial PIC context at an exchange / Trust Plane boundary [7, 8].

At the architectural level:

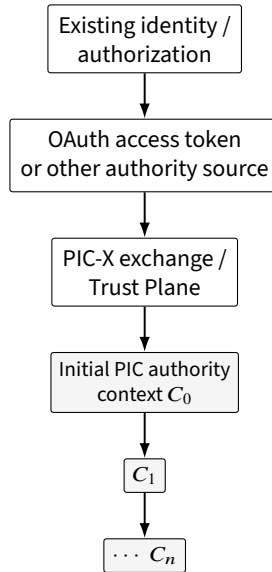


Figure 21.1: Existing authority can originate a PIC lineage through an explicit exchange boundary; later continuity remains non-expansive.

The PIC-X design uses OAuth as its first integration target: an OAuth access token is validated, mapped under an exchange profile and local policy into an initial logical PIC Context of Authority, and checkpointed into the first PIC state [8]. The high-level point is the boundary, not the wire format. The exchange is transport-independent: what the boundary verifies is the predecessor state and the authority context, not the channel over which the request arrived.

21.1 Execution context is a semantic requirement, not necessarily a carried chain

The PIC model does not require every request to transport the entire historical delegation chain. The PIC paper explicitly allows a lineage to be represented by commitments, references, hashes, signatures, or summarized proofs that let a hop establish its place in the causal chain without carrying the complete history [1].

This distinction matters when evaluating centralized exchange constructions. Consider a Token-Exchange-style service, in the broad architectural sense of OAuth 2.0 Token Exchange [20], that acts as the trusted authority-transition point. Suppose that, for every transition, it binds the new state to an explicit immediate predecessor, validates the holder binding required by the profile, enforces non-expansion $C_{n+1} \subseteq C_n$, preserves a lineage coordinate, and requires explicit policy approval for any cross-lineage composition. The service may checkpoint the accepted state and keep richer provenance separately for audit.

At that point the construction has not eliminated execution context. **It has implemented execution context at the trusted transition boundary.** The causal information may live in server state, a checkpoint reference, or a compact proof instead of being serialized as a full chain inside every token.

A schematic acceptance predicate is the following.

ACCEPTANCE AT A TRUSTED TRANSITION POINT

$$\text{Accept}_{n \rightarrow n+1} = \text{PredBound} \wedge \text{HolderBound} \wedge (C_{n+1} \subseteq C_n) \\ \wedge \text{LineageBound} \wedge \text{PolicyApproved}.$$

This equation is an architectural construction, not a normative statement about OAuth Token Exchange and not a claim that ordinary standards compliance provides these semantics automatically.

The diagnostic question is therefore concrete: *can the transition point distinguish two otherwise equal authority states that belong to different executions?* If yes because the decision reads cryptographically or otherwise trusted lineage context, it is continuity-aware in the relevant sense. If no, the projection result applies: different histories can collapse to the same possession view and any lineage-sensitive policy becomes impossible to express from that view alone [1].

Design principle. Continuity requires preservation of causal meaning, not transmission of the complete historical chain. A checkpoint may summarize history; it must not erase the lineage distinction that security policy needs.

Three separations should remain explicit:

1. the **PIC Model** defines the formal semantics and invariants;
2. the **PIC Specification** defines generic implementation-independent requirements and is not itself a protocol [9];
3. **PIC-X** is an implementation / exchange architecture that connects existing authority infrastructure to continuity; it does not define PIC [11].

The public specification material also states that current documents are independently developed public drafts rather than standards-body publications, and that no protocol specification has yet been published in the specification index [9, 10]. The book

therefore uses PIC-X as an architectural example, not as a claim of standardized interoperability.

22 Execution Gates and the Receiving Boundary

The public PIC material places the decisive security action at the **receiving boundary**: the executor proposes a continuation; the boundary decides whether the proposal is admissible under relationship, lineage, non-expansion, integrity, freshness, policy, and profile-specific checks [2].

Outcome-Oriented Programming generalizes that pattern. An **Execution Gate** is the runtime boundary at which a proposed continuation must be accepted before it becomes the next effectful execution state.

22.1 A composition gate is a security boundary

When an execution proposes to combine authority from two independent lineages, the gate is not merely validating another ordinary continuation. **It is deciding whether those authorities are allowed to participate together in a new execution context.** The inputs should therefore include the parent-lineage identities, the selected authority from each parent, applicable purpose and policy, runtime evidence, and any composition-specific constraints.

A composition gate can accept, narrow, defer, escalate, or deny the proposed combination. Critically, *the decision should not be inferred from the fact that one executor possesses all of the contributing credentials*. This is what makes composition governable: the right to use L_1 , the right to use L_2 , and the right to combine selected authority from L_1 and L_2 can be treated as distinct policy facts.

Because the core PIC model discussed in this book is linear, this multi-lineage gate is an architectural extension whose formal semantics remain an explicit proof obligation rather than an inherited PIC theorem.

22.2 A gate is not a UI dialog

The first gate of an execution is usually **plan approval**: the proposed route is accepted, by policy, by a governance agent, or by a person, before the journey begins. Later gates sit across the road. A gate may be invisible to the user. It can exist at a service boundary, queue consumer, tool adapter, transaction commit, financial action, or governance step. The essential architecture is:

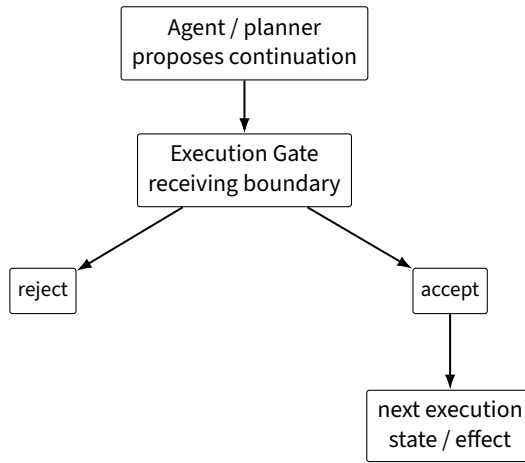


Figure 22.1: The planner proposes. The receiving boundary decides whether a proposal can become execution.

For authority continuity, the receiving boundary should not infer validity from the agent’s confidence or from possession of a credential. It verifies the conditions required by the selected profile. For broader Outcome-Oriented Programming, the gate may additionally check tool contracts, evidence predicates, business policy, risk, and commit-time outcome conditions.

A book-level acceptance shape is the following.

GATE ACCEPTANCE

$$\text{Accept}_i = \text{AppValid}_i \wedge \text{AuthValid}_i \wedge \text{Policy}_i \wedge \text{Evidence}_i.$$

Here AuthValid_i can include PIC’s single-hop relationship condition together with its non-expansion condition. The other conjuncts are architectural extensions of this book, not the PIC theorem itself.

22.3 What a gate should receive

A gate should receive structured state rather than the vague question “Is this okay?” A useful request can carry the lineage identifier or evidence, predecessor state, proposed operation, tool binding, relevant tool contract, required policy context, runtime evidence, and expected effects.

```

gate_request {
  predecessor: verified_execution_state
  proposed_operation: purchase(ticket_173)
  tool: travel_booking
  tool_contract: contract_ref
  authority_transition: candidate_context
  runtime_evidence: [...]
  outcome_predicate: ...
  policy_context: ...
}

```

The gate’s decision is therefore a structured acceptance or rejection of a state transition. This makes the boundary auditable and makes it possible to replace probabilistic judgment with deterministic or formally justified checks where the domain permits.

22.4 Residual risk becomes runtime state

Formal application containment and PIC can rule out selected invalid states under their respective models; they do not eliminate every source of operational, semantic, environmental, legal, evidence, or economic risk. Governance therefore needs explicit runtime evidence rather than a binary “verified / unverified” label.

For occurrence i , the runtime may construct an evidence vector

$$\varepsilon_i = (\text{effects, cost, latency, outcome progress, } \dots),$$

with assurance, policy state, and compliance evidence carried as additional fields when relevant.

An organization-specific risk function $\rho(\varepsilon_i, \kappa_i)$, evaluated under governance context κ_i , can produce a risk indicator or class. This function is deliberately not defined by PIC and is not claimed as a universal quantitative model. It represents the organization’s governance semantics.

When risk, missing evidence, compliance state, or irreversible effect crosses a policy decision region, **execution reaches a gate rather than relying on the planner’s confidence.** The gate can continue, constrain, request evidence, escalate, deny, or, when genuinely new authority is required, trigger an explicit new origination rather than silently expanding the current PIC continuation.

Design principle. Do not ask the agent to certify its own continuation. Let the agent propose; let the boundary verify.

23 Beyond Human-in-the-Loop

Human-in-the-loop is often described as a safety pattern in which a person checks an AI's output. That framing is too narrow for the architecture developed here.

At an Execution Gate, the system is not necessarily asking a human to review the planner's reasoning. **It is asking for a decision about whether a proposed continuation may proceed.**

23.1 The evaluator can change

Today the evaluator may be a person. Tomorrow the same gate may be handled by a deterministic policy engine. In other cases, it may be handled by specialized AI agents operating under separate authority and policy.

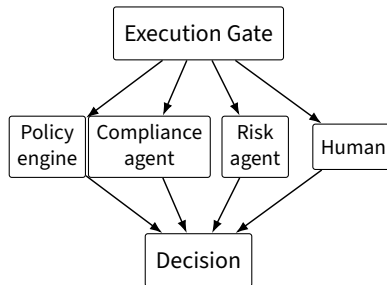


Figure 23.1: The same gate can be evaluated by policy, specialized agents, humans, or a combination.

23.2 From supervision to governance

The evolution can be understood in three stages.

Stage 1: Human supervision. The AI proposes; a human accepts or rejects.

Stage 2: Human escalation. Deterministic policy handles common cases; humans receive exceptions.

Stage 3: Governance Intelligence. Specialized systems analyze compliance, security, legal, financial, and operational evidence; humans remain at deliberately selected authority boundaries.

This leads to a useful statement:

Intelligence at every gate. Humans at selected gates.

The goal is not to remove people from accountability. *It is to stop treating manual review as the only form of control.*

24 Deferred Authority and New Origination

A critical distinction follows from PIC's non-expansion rule. Within an ordinary valid continuation,

$$C_{i+1} \subseteq C_i.$$

A gate cannot silently add a privilege that was absent from the current lineage and still call the result the same non-expansive continuation [1].

There are two different situations that must not be conflated.

24.1 Deferred decision inside origin-bounded authority

The N+1 outcome case may be unknown at signing time while still being bounded by cases established at the origin. The origin can sign a strict bound or a conditional predicate. Later, runtime evidence determines which already-authorized case applies. The guardrail-verified result then becomes the bounded context for the next step [2].

This is not arbitrary privilege creation. The future fact selects or closes a case whose admissibility was constrained before the fact existed.

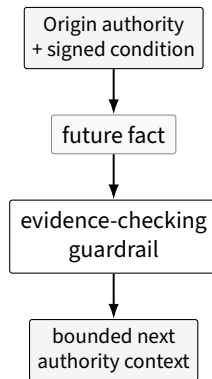


Figure 24.1: Deferred resolution can close an origin-bounded conditional case without turning the agent's prediction into authority.

24.2 Genuinely new authority

If a desired action requires authority that was never available within the originating authority boundary, that is a different event. **It requires a new act of origination** or another externally governed authority construction. The PIC model explicitly states that it constrains authority *within* a lineage but does not itself govern when an executor may legitimately open a new lineage rooted in its own privileges [1].

A human approval can therefore be architecturally important without being modeled as a magical mutation of C_i . The human may act as an authority-bearing origin for a

new decision. A policy service or specialized governance agent can do the same only to the extent that its own authority and governing rules permit it. The new origin and the causal request that led to it should remain visible.

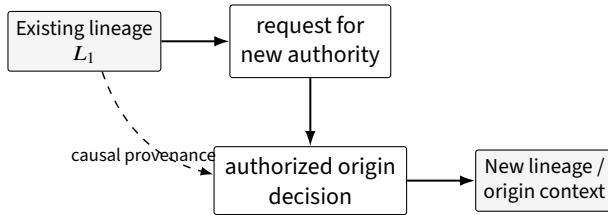


Figure 24.2: New authority should be modeled as a new authorized origin, not hidden expansion of an existing PIC continuation.

Design principle. A gate can resolve a pre-authorized condition or trigger a new authority origination. It cannot make non-expansion disappear.

25 Governance Intelligence

Once gates become first-class runtime objects, governance itself can become programmable and agentic.

The execution planner and the governance system have different jobs.

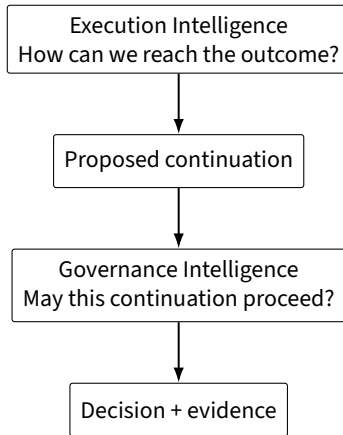


Figure 25.1: Planning intelligence and governance intelligence optimize for different objectives.

25.1 Specialized governance agents

A gate may delegate analysis to several specialists:

- a compliance agent checks rules and mandatory controls;
- a risk agent estimates exposure and reversibility;
- a security agent evaluates data and authority boundaries;
- a legal agent checks jurisdictional or contractual constraints;
- a cost agent checks budgets and economic policy;
- a human provides judgment or authority where required.

These systems need not share a single model. In fact, separation can be beneficial because execution optimization and governance optimization are different objectives.

25.2 Governance telemetry: KPI, KRI, and outcome progress

In an outcome-oriented runtime, governance is continuously fed by execution evidence. The same evidence stream can generate operational KPIs and risk KRIs: cost consumed versus budget, latency versus deadline, irreversible effects already committed, assurance

level of bound tools, policy exceptions, compliance evidence, authority narrowing, and progress toward the outcome predicate.

These metrics do not become security proofs merely because they are numerical. They are **governance observables**. Their role is to make residual risk measurable enough for a policy, specialist agent, or human to decide whether execution may continue, must be constrained, or must stop.

25.3 Governance output should be structured

A useful gate decision is more than true or false. It can contain:

```
decision {  
  verdict: APPROVE_WITH_LIMITS  
  granted_authority: purchase <= 450 EUR  
  valid_for: this booking only  
  evidence: [policy_17, risk_score_0.08]  
  conditions: [receipt_required]  
  expires: 15 minutes  
}
```

The runtime can then enforce the decision mechanically.

25.4 Humans become deliberate roots of authority

In mature systems, *the human role becomes clearer rather than smaller*. A person is not asked to inspect every routine action. The person is placed at gates where the organization intentionally wants human authority, judgment, accountability, or exception handling.

This is a better use of human attention than universal manual supervision.

26 Residual Risk: Where Security Meets Governance

Security and governance meet at a boundary that is easy to blur. **Security engineering** changes what failures remain technically possible, or changes their likelihood, impact, exposure, and observability. **Risk analysis** characterizes what remains. **Governance** decides whether that remainder is acceptable in context. **Accountability** identifies who is authorized to own that decision.

**Security constrains possibility.
Governance constrains acceptability.**

This distinction matters especially for agentic execution. A planner may propose a transition that is semantically plausible, technically executable, and even supported by a valid credential. None of those facts, by itself, answers whether the transition satisfies the relevant application invariant, belongs to the current authority lineage, or leaves an acceptable level of residual risk.

26.1 Risk is not necessarily one number

Let Ω denote a modeled space of execution and failure scenarios. Let

$$\mathcal{A} = \{a_1, a_2, \dots, a_n\}$$

be a set of attack or failure classes. Each class a corresponds to a harmful subset $H_a \subseteq \Omega$.

For a class a , an organization may track a descriptor

$$q_a = (p_a, \iota_a, x_a, u_a),$$

where p_a is an estimated likelihood, ι_a an impact measure, x_a an exposure measure, and u_a an explicit uncertainty term. A local risk model may then define

$$r_a = \rho(q_a; \kappa),$$

where κ is the organizational and operational context.

This notation is deliberately generic. It does not claim that likelihood is always known numerically, that impact has a universal unit, or that every risk class can be meaningfully added to every other one. A simple product such as $p_a \iota_a x_a$ can be useful inside a chosen model, but *it is not a universal law of risk*. For heterogeneous enterprise risk, a vector is often the cleaner abstraction.

26.2 Controls change possibility in different ways

Not every security control does the same mathematical job. A preventive or enforcing control can remove executions from the set that the system will accept. Represent that effect as an admissibility operator

$$F_c : \mathcal{P}(\Omega) \rightarrow \mathcal{P}(\Omega).$$

For controls $c_1, \dots, c_m$, the modeled set of executions still admitted after enforcement is

$$\Omega_{\text{sec}} = F_{c_m}(\dots F_{c_2}(F_{c_1}(\Omega)) \dots).$$

Other controls do not necessarily remove a scenario. They may reduce likelihood, reduce impact, reduce exposure, improve detection, shorten response time, or transfer part of the consequence. Those effects are better represented as changes to the risk descriptor or to the evidence available to governance, not as deletion from Ω .

Control effect	Architectural meaning
Prevent / enforce	Shrinks the set of executions that can be accepted.
Mitigate	Leaves a scenario possible but reduces one or more consequences or exposures.
Detect / observe	Produces evidence that can trigger response or a later gate; detection is not prevention.
Transfer / share	Changes who bears some consequence; it need not change technical possibility.
Accept	A governance decision about remaining risk; it does not itself change the technical state space.

The familiar labels *eliminated*, *reduced*, *detected*, *transferred*, *accepted*, and *unknown* are therefore useful classifications, but they need not form a clean mathematical partition. One failure class can be simultaneously reduced, monitored, insured, and conditionally accepted.

26.3 Security invariants and the proof boundary

A security invariant is a predicate over modeled executions,

$$I : \Omega \rightarrow \{\text{true}, \text{false}\}.$$

If the accepted execution set satisfies

$$\forall e \in \Omega_{\text{sec}}, \quad I(e) = \text{true},$$

then the invariant holds for that modeled set. A harmful class a is eliminated by *construction in the model* when

$$\Omega_{\text{sec}} \cap H_a = \emptyset.$$

The qualification matters. **A formal proof has a boundary.** Let B be a set of explicit assumptions and M a formal model. A proof establishes a judgment of the following shape.

PROOF BOUNDARY

$$M, B \models I,$$

read: within the model M and under the assumptions B , the invariant I holds. It does not establish the sentence “the deployed system is safe.” It establishes that the modeled executions satisfy I when the assumptions and refinement relationship actually hold.

If an assumption can fail in deployment, *the risk of that assumption failure belongs outside the theorem.* A useful conceptual decomposition is therefore a tuple, not a claim that these components are universally additive.

TOTAL RISK DECOMPOSITION

$$\mathbf{R}_{\text{total}} = (R_{\text{model}}, R_{\text{assumption}}, R_{\text{unknown}}).$$

R_{model} covers modeled harmful classes that remain possible; $R_{\text{assumption}}$ covers failure of trusted assumptions or refinement boundaries; R_{unknown} acknowledges unmodeled and epistemic uncertainty.

Design principle. A proof can exclude a modeled failure class under explicit assumptions. It does not accept the risk that remains outside the proof boundary.

26.4 Residual risk

Define the modeled residual classes as

$$\mathcal{A}_{\text{res}} = \{a \in \mathcal{A} \mid H_a \cap \Omega_{\text{sec}} \neq \emptyset\}.$$

A residual-risk representation can then be kept as the vector

$$\mathbf{R}_{\text{res}} = (\rho(q_a^{\text{sec}}; \kappa))_{a \in \mathcal{A}_{\text{res}}},$$

where q_a^{sec} represents the post-control descriptor for class a . An organization may later aggregate or rank this vector, but the architecture does not assume a universal summation rule.

This gives a precise interpretation to a phrase used throughout the book: **formal security can remove selected classes from the modeled residual-risk surface; probabilistic and operational controls can reduce others; governance must reason about what remains**, including assumptions and unknowns.

26.5 Governance decides; it does not rewrite the theorem

Let a governance function be modeled as

$$G(\mathbf{R}_{\text{res}}, \kappa, \varepsilon) \rightarrow D,$$

where ε is the available execution evidence, κ is the governance context, and

$$D \in \{\text{ACCEPT}, \text{MITIGATE}, \text{TRANSFER}, \text{AVOID}, \text{DEFER}\}.$$

The context can include business criticality, risk appetite and tolerance, asset value, data classification, time horizon, contractual constraints, compensating controls, reversibility, and any applicable policy requirements.

For an acceptance decision, a conceptual rule can be written as follows.

GOVERNANCE ACCEPTANCE OF RESIDUAL RISK

$$\begin{aligned} \text{Accept}(p, \mathbf{R}_{\text{res}}, \kappa) \iff & \text{AuthorizedRiskOwner}(p, \kappa) \wedge \text{ExplicitAccept}(p, \mathbf{R}_{\text{res}}, \kappa) \\ & \wedge \mathbf{R}_{\text{res}} \preceq \Theta(\kappa). \end{aligned}$$

Here $\Theta(\kappa)$ is an organization-specific tolerance envelope and $\preceq$ is an organization-specific acceptability relation. It need not be a scalar threshold. This equation is a proposed architectural abstraction, not a formula prescribed by a regulation or standard.

Governance may decide that more controls are required, which feeds back into the security architecture and changes Ω_{sec} or the risk descriptors. But **the act of accepting risk does not itself change a security invariant**.

Proof $\neq$ acceptance.

Acceptance $\neq$ proof.

26.6 Accountability is a separate output

A risk decision without an accountable principal is incomplete in an enterprise system. Let $\mathcal{P}$ be the set of principals or organizational roles that the governance model recognizes. At the architectural level,

$$\text{Accountability} : D \rightarrow \mathcal{P}$$

associates each decision with the principal or organizational role that is authorized to own it; where a deployment admits several accountable subjects for one decision, the codomain becomes $\mathcal{P}(\mathcal{P})$, the set of subsets of $\mathcal{P}$. The exact legal and organizational meaning of that role is outside this book and depends on the deployment context.

The complete pipeline is therefore:

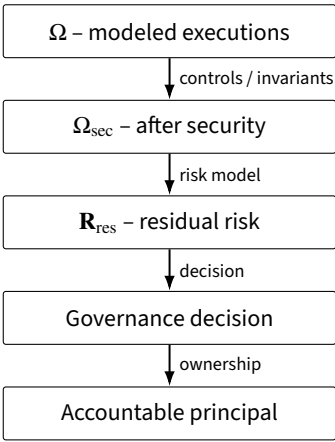


Figure 26.1: Security, risk analysis, governance, and accountability are connected but distinct transformations.

26.7 Authentication, authorization, continuity, governance

These layers answer different questions and should not be collapsed into the word “security.”

Layer	Question
Authentication	Who or what is acting, according to the selected identity and trust model?
Authorization	Which operation on which resource is permitted under the applicable policy?
Holder binding	Is the presenter bound to the key or proof required by the credential profile? DPoP is one standardized example of proof-of-possession style binding [21].
Authority continuity	Does the authority being exercised belong to the causal execution that is being continued?
Governance	Is the remaining risk acceptable under the organization’s current context and policy?
Accountability	Who is authorized to make and own that decision?

Zero-trust architectures likewise distinguish authentication and authorization and reject implicit trust based only on location or ownership [24]. The continuity property developed in PIC is not claimed to be a requirement of zero trust; it addresses the additional question of which authority instance is causally valid for a particular multi-hop execution.

26.8 Temporal confused deputy as a residual-risk class

The temporal confused-deputy pattern introduced earlier can now be placed directly into the risk model. Define a predicate and the corresponding harmful class as follows.

TEMPORAL CONFUSED DEPUTY

$$\begin{aligned}
\text{TCD}(e, A, D) &= \text{ValidAuthority}(A) \wedge \text{Possesses}(D, A) \\
&\quad \wedge \text{Executes}(D, e, A) \wedge \neg \text{CausallyValid}(A, e), \\
H_{\text{TCD}} &= \{e \in \Omega \mid \exists A, D : \text{TCD}(e, A, D)\}.
\end{aligned}$$

The important distinction is visible in the predicate itself:

$$\text{ValidAuthority}(A) \not\Rightarrow \text{CausallyValid}(A, e).$$

If a system has no lineage-sensitive acceptance invariant, the architecture cannot generally conclude that H_{TCD} is excluded merely because credentials are valid or attenuated. Under a model whose continuity invariant requires every exercised authority to be a valid continuation of the authorized predecessor, the target property becomes

$$\Omega_{\text{sec}} \cap H_{\text{TCD}} = \emptyset,$$

subject to the assumptions and refinement boundary of that model. This does not mean that PIC eliminates every confused-deputy or authorization failure. **It means that the specifically modeled lineage-mismatch class can be made invalid state under the stated continuity semantics.**

Composition preserves the same separation. *Security must prevent implicit authority mixing; governance may explicitly authorize a composition* under policy, evidence, lifetime, and revocation conditions. Revoking the permission to compose two lineages need not revoke either parent authority. The objects are distinct, so their risks and governance decisions can remain distinct.

Design principle. Security proofs reduce the set of harmful executions that remain valid under a model. Governance decides whether the risk associated with the remaining possibilities is acceptable. Neither substitutes for the other.

26.9 Framework correspondence, not compliance

This security-governance separation has recognizable correspondences with enterprise risk frameworks, but the architecture is not derived from any one regulation or standard. Appendix C gives a short, source-based correspondence to selected NIST and EU materials. That appendix is descriptive only: none of those sources is claimed to require PIC, execution lineage, Temporal Confused Deputy terminology, or Outcome-Oriented Programming.

27 Governance Is Execution Too

Once governance contains AI agents, policy services, evidence collectors, or multi-stage compliance analysis, it should not become an unmodeled oracle outside the architecture. It is execution as well.

A governance decision can therefore have its own inputs, predecessor state, evidence, tool calls, and authority. If that process is itself security-relevant, the same proposal-versus-verification discipline can be applied to it. The public PIC material describes this general shape when policy evaluation itself is governed and the guardrail process continues only as valid PIC state [2].

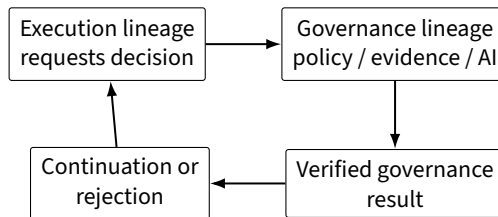


Figure 27.1: Governance can be modeled as execution rather than as an unexplained trusted answer.

This has an important consequence for specialized AI reviewers. A compliance agent can propose a judgment; the system should still separate that proposal from the deterministic rules, evidence, authority, and acceptance checks that make the judgment operative.

The aim is not infinite recursion. **The aim is to keep the trusted computing base explicit.** At some point a small verifier, policy engine, cryptographic checker, human signature, or other trusted root must decide. The architecture should make that root visible rather than hiding it inside a model response.

28 Branches, Redelivery, and Uniqueness

Continuity is not the same property as exactly-once execution.

The published Tamarin model intentionally allows sibling continuations from the same checkpoint and verifies the stated symbolic safety properties in the presence of fan-out [5]. Each accepted successor defines a distinct lineage extension sharing the same causal prefix; along each resulting path, authority remains bounded by its predecessor and origin, and authority dropped on that path cannot reappear later on it. The execution graph may therefore branch even though each lineage remains a linear causal history. *Continuity does not, by itself, say that only one successor may exist.*

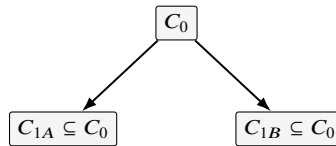


Figure 28.1: Fan-out can preserve continuity on multiple branches. Continuity alone does not imply a single successor.

This matters for queues and brokers with redelivery. If a business operation requires uniqueness, single consumption, idempotency, or a single-use commitment, that property must be designed separately or represented explicitly in the branch profile. It should not be silently attributed to PIC.

The broader lesson for Outcome-Oriented Programming is that **architectural properties must remain factored**. Causal continuity, authority non-expansion, uniqueness, idempotency, ordering, and outcome correctness are different properties. A robust runtime composes them instead of treating one successful security proof as a proof of everything.

29 Prompt Injection as an Authority Problem

Prompt injection is often framed as a content-filtering problem: prevent untrusted text from changing the agent’s instructions. Filters and model robustness are useful, but a lineage-aware architecture adds a stronger structural defense.

Suppose the origin intent is:

Summarize document D.

The execution may legitimately require reading D and producing a summary. The agent process might also possess tools that can delete files, send email, or access unrelated secrets. An indirect instruction inside D can attempt to redirect the planner toward those actions.

The application plane may still produce a candidate action. The authority plane should reject it if it is not a valid continuation of the origin authority context.

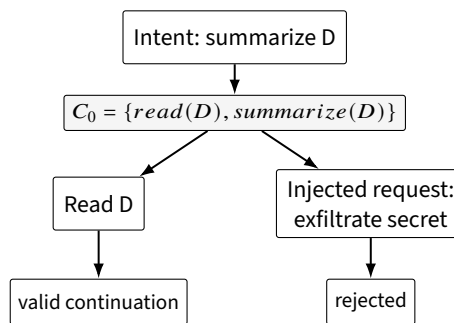


Figure 29.1: Untrusted content may influence planning, but authority remains bounded by the originating lineage.

The PIC paper uses this exact class of example: an agent asked to summarize a document may be induced to attempt deletion or exfiltration, but those actions are valid only if the origin authority context contains the corresponding authority [1].

This does not eliminate the need for model security. **It changes the failure mode.** A confused planner can propose an invalid action *without the runtime treating the proposal itself as authority*.

Semantic containment adds a second barrier. If the injected text causes the planner to invent a nonexistent operation, bind the wrong tool, or propose effects outside the selected tool’s manifest, the application plane can reject the candidate before PIC is even asked whether the lineage has authority. PIC then independently rejects authority that is not a valid continuation. The defenses are complementary: **semantic validation constrains what software behavior may be accepted; PIC constrains what the current lineage may cause.**

Part IV

The Verifiable Intent Runtime

30 The Runtime Architecture

The pieces can now be assembled into one architecture. Natural language is interpreted into explicit semantic objects; an ontology-typed IER proposes operations; Semantic Manifests and assurance evidence constrain application behavior; PIC constrains authority; and runtime evidence feeds governance decisions.

30.1 Responsibilities

A practical Intent Runtime can be decomposed into several responsibilities:

Intent interpretation. Propose goals, outcome predicates, constraints, and origin context from natural-language or system intent.

Semantic normalization. Map accepted intent, outcomes, entities, effects, and evidence requirements into the domain ontology.

Planning. Produce or revise an ontology-typed IER.

Plan approval. Accept the proposed IER, by policy, by a governance agent, or by a person, before execution begins.

Tool registry. Discover concrete tools, Semantic Manifests, contracts, and assurance evidence.

Binding. Map ontology-level operations to tools under semantic, assurance, policy, and environment constraints.

Lineage management. Assign and propagate execution occurrence relationships.

Authority evaluation. Track the authority context associated with the lineage and apply PIC continuity rules.

Gate orchestration. Pause execution, collect evidence, route decisions, and resume or terminate.

Execution. Invoke tools with retries, idempotency, transactions, and compensation where available.

Observation. Record effects, outcomes, costs, latency, decisions, causal provenance, assurance evidence, and policy/compliance state.

Governance telemetry. Derive the KPI/KRI and residual-risk evidence required by gate policy.

Outcome check. Determine whether the requested semantic outcome predicate has actually been achieved.

30.2 The runtime is not the agent

The AI planner may be replaced, upgraded, or specialized without changing the runtime's core invariants. Likewise, the runtime can execute plans generated by

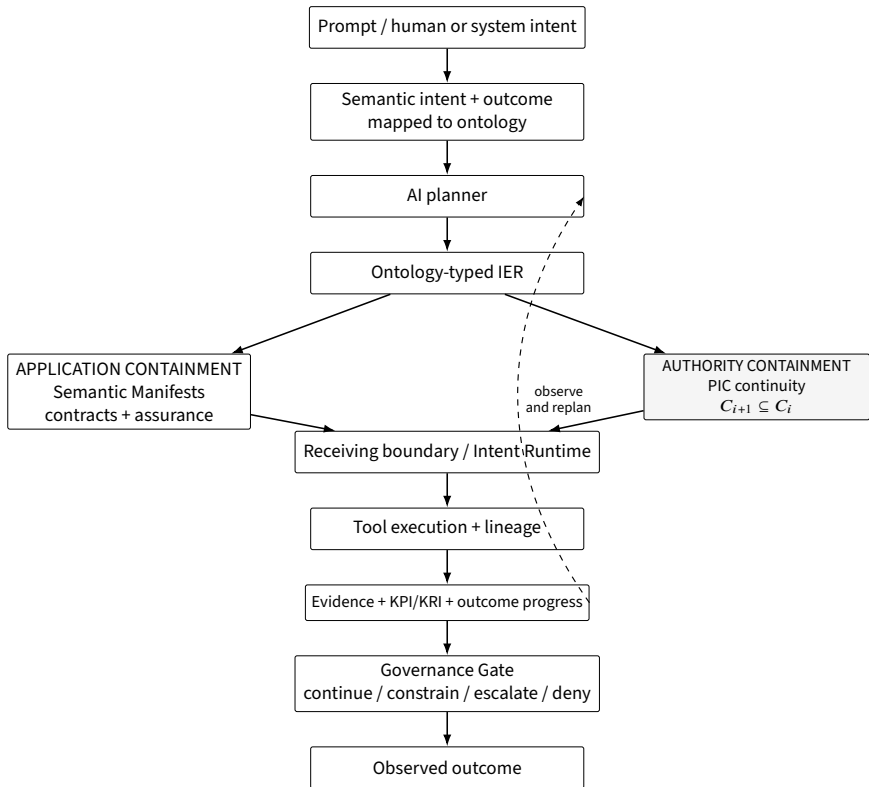


Figure 30.1: The verifiable Intent Runtime: semantic meaning is lowered into ontology-typed execution, then constrained independently by application semantics and authority continuity before governance accepts continuation.

different models.

This separation is important. **The system should not need to trust the model to enforce the model's own permissions.**

Design principle. The planner proposes execution. The runtime owns execution.

31 Formal Methods Become Architecture

If an AI planner is probabilistic, **critical execution should not depend on the planner being consistently correct**. The stronger architecture is asymmetric:

The agent may be probabilistic. The acceptance boundary should be as deterministic and checkable as the property allows.

31.1 From signed bytes to behavioral claims

Traditional software trust often starts with artifact identity: a digest can be compared with an expected digest; a digital signature can authenticate signed bytes relative to a key and trust model; a supply-chain record can describe provenance. These mechanisms are necessary, but they stop before semantics. They do not prove that a program implements `travel.purchase`, preserves a balance invariant, avoids an unlisted side effect, or refines the state machine a planner expects.

Outcome-Oriented Programming therefore asks a second trust question after “is this the expected artifact?”:

What behavioral property of this artifact has actually been established, and by what evidence?

This is not a claim that every program should be written in a theorem prover. *It is a claim about where engineering value moves when code generation becomes cheap.* More effort can move into semantic specifications, invariants, type systems, protocol semantics, model checking, refinement arguments, runtime monitors, proof artifacts, and small trusted verifiers.

31.2 An assurance ladder

The book uses the following proposed ladder to avoid the word “verified” becoming ambiguous:

Level	Meaning
A0	Artifact identity / integrity evidence: digest, signature, provenance, trusted release metadata.
A1	Machine-readable semantics: ontology references, schemas, declared effects, typed inputs and outputs.
A2	Executable contracts: checked preconditions, postconditions, invariants, state-machine or runtime-monitor obligations.
A3	Verified properties: theorem proving or model checking establishes selected properties of an abstract model or implementation fragment under stated assumptions.
A4	Refinement evidence: a justified correspondence connects the concrete implementation boundary to the abstract contract or verified model.

These levels are an architectural proposal, not an industry standard. A policy can require different assurance for different effect classes. A read-only search tool may be acceptable at A1/A2; a broadly reusable financial primitive may justify A3/A4 investment.

31.3 From test cases to proof obligations

A conventional tool is often documented by examples. A verifier-oriented tool should expose stronger semantics. A minimal mathematical contract for a tool T can be written as:

$$\text{Pre}_T(w, x)$$

for the admissible input state w and input x , and

$$\text{Post}_T(w, x, w', y)$$

for the relationship between the pre-state, post-state w' , and output y .

A planner may propose the invocation. The runtime can check types and preconditions before execution and validate observable postconditions or evidence after execution. For externally controlled services, some postconditions may remain assumptions or evidence-backed claims rather than formal proofs.

At the plan level, for actions $a_0, \dots, a_n$, an architectural validity condition can require each action to satisfy its tool contract while the parallel authority transition satisfies PIC.

STEP VALIDITY

$$\begin{aligned} \text{StepValid}_i = & \text{SemanticMatch}_i \wedge \text{ApplicationAccepted}_i \\ & \wedge \text{PoR}(s_i, s_{i+1}) \wedge (C_{i+1} \subseteq C_i) \wedge \text{Policy}_i. \end{aligned}$$

This equation is a synthesis introduced by this book. PIC proves the authority-continuity

conjunct under its formal model; semantic matching, application acceptance, and application-policy conjuncts require their own ontology, specifications, implementations, evidence, and verification arguments.

31.4 Why this matters for hallucination

An LLM can hallucinate a function name, an unsupported argument, an impossible workflow, an incorrect ontology mapping, or an unsafe transition. Formalization does not prevent the model from generating such a proposal. **It changes the acceptance question.** The plan must resolve to known semantic operations and contracts instead of becoming executable merely because the model produced plausible text.

If tools are described only by prose, the same probabilistic system that created the proposal may also be asked to judge it. If important properties are encoded as machine-checkable contracts, the verifier can reject a proposal independent of the model's confidence.

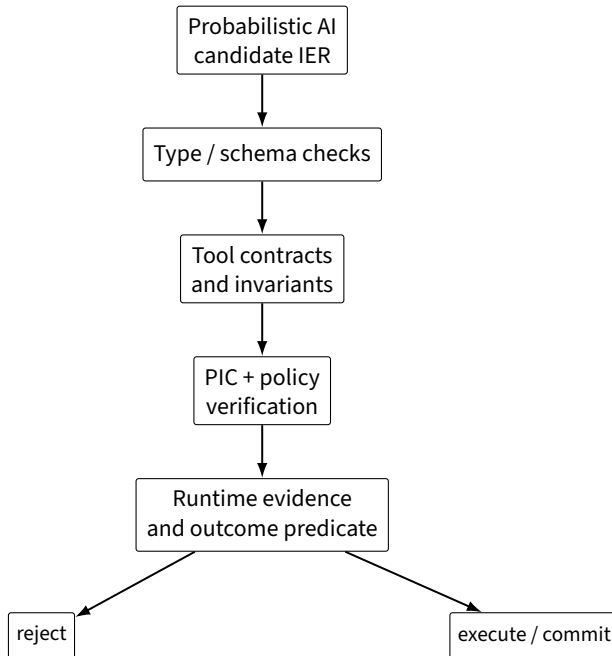


Figure 31.1: Hallucination is treated as untrusted proposal generation; acceptance moves into deterministic verification layers.

Design principle. Do not try to make the generative model the proof. Make it produce objects that a smaller, explicit verifier can check.

32 The PIC Proof Stack

Lean, Tamarin, and ProVerif as complementary verification layers.

The public PIC formal material is useful beyond PIC itself because it demonstrates how different proof tools can address different layers instead of making one verifier carry every claim [3].

Artifact	Supported role in the published PIC artifacts
Lean 4	Abstract mathematics: arbitrary finite chains, origin-bounded authority, continuity, irreversible authority drop, projection and trade-off results, heterogeneous translations, and a refinement bridge from concrete verifier acceptance to abstract PoC.
Tamarin	Stateful symbolic trace reasoning: origin-bound authority, irreversible branch-local drop, sibling branching, PoR eligibility, and key-control properties under an active Dolev–Yao network attacker.
ProVerif	Independent symbolic check of the authentication core: accepted transition implies trusted PoR eligibility; uncompromised accepted key control; plus a non-vacuity reachability check.

Table 32.1: The proof artifacts are complementary rather than interchangeable.

The published Lean development defines the core transition exactly in this shape:

```
def ValidTransition ... :=
  PoR s0 s1 /\ C1 subset_of C0

def PoC ... :=
  every adjacent transition is ValidTransition
```

The actual Lean source uses its own set notation and types; the listing above is a typography-friendly rendering of that published definition. From the chain condition, the Lean artifact proves that downstream authority is bounded by the origin and that dropped authority cannot reappear later on the same branch [4].

The published refinement module is equally important architecturally. It does not claim that a cryptographic construction is magically proven by Lean. Instead it models concrete relationship checks as predicates and makes their soundness an explicit assumption. Under the assumption that the concrete checks witness the abstract PoR relation, the theorem `concreteAcceptance_implies_PoC` proves that concrete acceptance refines to the abstract continuity model; the existing PIC safety theorem then applies [4].

That is a powerful engineering pattern:

implementation checks $\Rightarrow$ refinement bridge $\Rightarrow$ abstract invariant.

32.1 What the proof stack does not prove

The published material is explicit about its limits. Lean does not prove the cryptographic security of a concrete issuer, encoding, attestation service, or deployment. Tamarin uses symbolic cryptography and perfect parsing. ProVerif does not prove the full inductive non-expansion theorem for unbounded checkpoint chains in the published model. None of these tools proves that a human specification captures the user's complete semantic intent [3].

This discipline matters more than the presence of a theorem prover. **A mathematically perfect proof of the wrong property is still the wrong assurance claim.**

33 Formal Tool Contracts and Verifier-Oriented Execution

The PIC artifacts prove authority properties, not arbitrary application-tool correctness. Extending formal methods from authority into Outcome-Oriented Programming is therefore a proposal of this book.

The proposed direction is to make the most reusable and dangerous tools carry explicit semantics that can be checked at progressively stronger levels:

1. **schema and type level:** valid inputs, outputs, and effect categories;
2. **contract level:** preconditions, postconditions, idempotency, compensation, state transitions;
3. **property level:** invariants such as conservation, bounded amounts, monotonic state, isolation, or separation of duties;
4. **refinement level:** proof or verified correspondence between a concrete implementation boundary and the abstract contract;
5. **protocol level:** trace properties across services and adversarial message scheduling where a protocol verifier is appropriate.

Not every CRUD screen deserves a theorem. But a payment primitive used by thousands of dynamically generated workflows has very different leverage. *The more frequently a primitive is reused and the larger its blast radius, the stronger the economic case for investing in formal semantics.*

The phrase “tools become Lean” should therefore be interpreted precisely. The goal is not to rewrite every driver, UI, or external connector in Lean. The goal is that high-leverage operations acquire formal specifications and proof obligations that can be expressed and checked in tools such as Lean where appropriate; protocol interactions can be modeled in tools such as Tamarin or ProVerif; and concrete implementations must either refine those models, be wrapped by a verified boundary, or be assigned a weaker assurance level that governance can see. **The assurance must match the claim.**

33.1 The new unit of reusable trust

Today a library exposes an API. An agent-native tool should increasingly expose an API, a Semantic Manifest, and a contract that lets a runtime reason about what the tool can change. The manifest maps the implementation into the ontology; the contract defines machine-checkable behavior; assurance evidence says why the runtime should trust that correspondence.

```

tool: transfer_funds
inputs: source, destination, amount
preconditions:
  amount > 0
  source.balance >= amount
postconditions:
  source.balance' = source.balance - amount
  destination.balance' = destination.balance + amount
invariants:
  total_value_conserved
side_effects:
  financial_transfer
compensation:
  defined_by_domain_policy

```

This listing is an architectural example, not a claim that the public PIC implementation contains such a tool. Its purpose is to show the abstraction programmers would increasingly design for agents and verifiers rather than for hand-written orchestration alone.

33.2 Proof-carrying plans as a direction

A future IER can carry more than an action sequence. It can carry references to contracts, evidence obligations, authority predecessors, and proof or validation artifacts. The plan then becomes a set of obligations to discharge before commit.

The goal is not to prove arbitrary real-world truth inside Lean. **The goal is to separate what can be formally established from what must be measured, attested, signed, or approved.** External facts enter as explicit evidence assumptions; internal invariants can be machine-checked.

33.3 New proof obligations created by composition and checkpointing

Once continuity is implemented through checkpoints or independent lineages may be explicitly composed, the verifier stack acquires additional obligations that should not be hidden behind the word “verified.” Examples include:

- **checkpoint soundness:** acceptance of a compact checkpoint must imply the predecessor relationship and authority constraint claimed by the abstract model;
- **lineage non-collision:** two security-relevant executions must not become indistinguishable merely because they expose the same authority value;
- **no implicit union:** possession of several contexts must not itself construct a composed authorization;
- **composition-policy soundness:** an accepted composed context must be justified by the explicit policy and evidence for that composition event;
- **parent preservation:** the system must retain enough trusted information to

identify which independent lineages contributed to the composed execution when policy, audit, or revocation depends on that fact.

The current PIC proof artifacts establish properties of the linear continuity model; they should not be cited as proofs of an arbitrary future composition protocol. The architectural direction is instead to make these new obligations explicit so that a concrete construction can later be refined to them.

Design principle. The serious engineering problem shifts from generating glue code to defining the semantics that generated execution is not allowed to violate.

34 What Formal Verification Cannot Rescue

Formal methods strengthen a system only relative to a specification and explicit assumptions. They do not solve every uncertainty introduced by AI.

- A proof of authority continuity does not prove that an authorized action semantically serves the user's purpose.
- A verified tool contract does not prove that an external vendor told the truth unless the relevant evidence model establishes that fact.
- A theorem about an abstract model does not prove that a concrete implementation refines that model without a valid refinement argument.
- A verifier cannot recover constraints that were never specified.
- A proof that an IER satisfies a formalized outcome does not prove that the AI mapped the human's natural-language intent into the correct formal outcome. That interpretation boundary remains a semantic risk and may require confirmation or governance.
- An ontology can make meanings explicit, but a bad ontology or unsound mapping can formalize the wrong meaning consistently.
- Formal verification does not make an LLM deterministic; it limits what selected model outputs can become accepted execution.
- A proof for a single linear lineage does not automatically prove the safety of a multi-lineage composition rule; composition needs its own semantics and proof obligations.

The correct ambition is therefore not “prove the whole world.” It is to move critical uncertainty out of informal convention and into explicit interfaces between theorem, ontology, runtime evidence, trusted roots, policy, and open assumptions. Formal methods can remove selected failure modes from the residual-risk set; they cannot make residual risk globally zero.

This is also why the programmer's role becomes more intellectually demanding rather than disappearing. The high-value work moves toward formal semantics, distributed systems, protocol design, concurrency, proof engineering, policy translation, runtime verification, and the construction of small trusted computing bases.

35 Execution Economics

The original motivation for the architecture was economic as much as conceptual: avoid regenerating stable software at inference cost.

A mature runtime can make economic choices explicitly.

35.1 Choose the cheapest sufficient mechanism

For each problem, there is a hierarchy of possible mechanisms:

1. execute an existing deterministic function;
2. execute a known workflow;
3. instantiate a parameterized IER;
4. synthesize a new IER with AI planning;
5. generate a new implementation only when no adequate tool exists.

The runtime should prefer the lowest level that can correctly achieve the outcome.

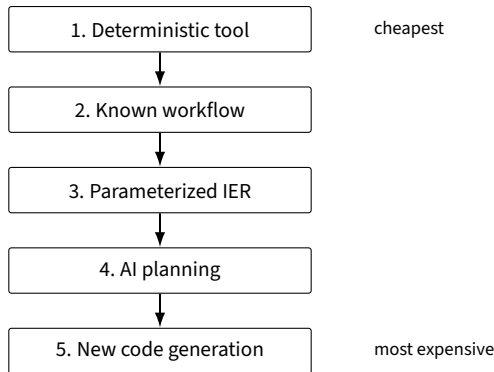


Figure 35.1: Open-ended generation should be the last resort, not the default execution mechanism.

35.2 Budget-aware planning

Cost can be part of the plan itself. A tool contract can expose expected latency, monetary cost, and resource characteristics. The planner can then optimize under constraints rather than optimize only for task completion.

```
optimize plan:
  objective = minimize(total_cost + latency_penalty)
  subject_to = {
    outcome_satisfied,
    authority_valid,
    policy_compliant,
    deadline_met
  }
```

35.3 Inference as an architectural resource

This perspective treats inference in the same way mature systems treat network calls, memory, storage, or database queries: as a powerful resource whose cost should be visible and managed.

Outcome-Oriented Programming is not an argument for more inference. **It is an argument for using inference where it buys the most semantic leverage.**

36 From Dynamic Plans to Deterministic Tools

The runtime can become more efficient over time by recognizing repeated patterns.

The first time a novel intent appears, the planner may need substantial reasoning. If the same structure appears thousands of times, it becomes wasteful to reconstruct it from scratch.

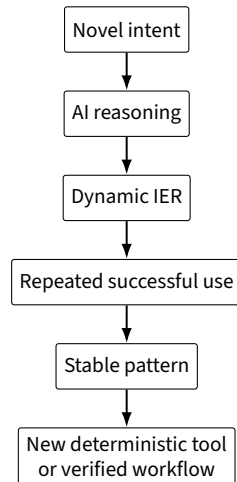


Figure 36.1: Repeated reasoning can crystallize into deterministic software.

36.1 A JIT for intentions

The analogy with just-in-time compilation is useful. The first execution may be interpreted and planned dynamically. Frequent patterns can be cached, specialized, verified, and eventually compiled into a lower-cost representation.

The system can therefore move behavior down the cost hierarchy:

reasoning → plan → workflow → tool

This is a feedback loop from intelligence to determinism.

36.2 The developer's role in crystallization

A developer may review the emerging pattern, define its contract, tighten invariants, optimize implementation, and publish it as a reusable tool. The system's runtime history becomes a source of candidates for new deterministic abstractions.

The result resembles how frameworks have always evolved: repeated application code becomes a library. The difference is that an agentic runtime can discover repetition directly from execution traces.

Design principle. When a pattern becomes stable, stop paying reasoning cost for it. Crystallize it into verified deterministic execution.

37 Semantic Review

If AI systems generate plans, workflows, and eventually implementation artifacts, reviewing raw source code becomes less sufficient and sometimes less useful.

The important question has always been behavioral: *what changed?* Traditional code review approximates that answer by showing textual differences. **Agentic systems can expose the semantic difference directly.**

37.1 From code diff to behavior diff

Instead of:

```
- if amount < 100:  
+ if amount <= 100:
```

a semantic review can say:

```
BEHAVIOR CHANGE  
  
Automatic refund threshold  
  before: amount < 100 EUR  
  after:  amount <= 100 EUR  
  
Newly reachable outcome  
  refund of exactly 100 EUR can execute automatically  
  
Authority impact  
  unchanged  
  
Affected lineages  
  refund workflow  
  
Irreversible effects  
  none added
```

The second representation is closer to the decision a reviewer actually needs to make.

37.2 Six diffs

An Outcome-Oriented system can expose at least six semantic diffs:

Intent diff. Which classes of origin requests or constraints are interpreted differently?

Outcome / ontology diff. Which desired or reachable states, ontology concepts, mappings, or predicates changed?

Tool / manifest diff. Which tool binding, declared effects, invariants, or assurance evidence changed?

Effect diff. Which external side effects were added, removed, or reordered?

Authority diff. Which actions can now be caused by which lineages, contexts, or gates?

Governance diff. Which risk thresholds, compliance conditions, KPI/KRI, or required approvals changed?

Authority diff is particularly important because generated implementation can remain functionally correct while *changing who is allowed to cause the function*. Ontology and manifest diffs are equally important because a formally correct plan can still become the wrong plan if semantic interpretation or tool meaning changes.

37.3 Machine-oriented implementation

As generated code becomes more machine-oriented, human readability may migrate upward. Humans review contracts, invariants, semantic effects, and authority transitions; machines verify the lower-level implementation against those descriptions.

This does not make source code irrelevant. It changes the primary review surface.

38 The New Role of the Programmer

The most common question about agentic software is whether programmers disappear. That question treats programming as a single activity. In reality, software engineering contains several layers of work, and AI affects them differently. Section 1.2 argued that the industry may well shrink in headcount as repetitive work is absorbed; this chapter is about what the remaining work is.

38.1 Work that moves upward

A large amount of application work consists of translating known requirements into glue code: CRUD flows, API adapters, validation plumbing, forms, routing, mapping, repetitive business orchestration. Those tasks are natural candidates for agentic composition.

The programmer's work can move toward:

- domain ontologies and semantic type systems;
- deterministic tools, Semantic Manifests, and tool contracts;
- preconditions, postconditions, effects, invariants, and formal properties;
- proof obligations, refinement boundaries, and small trusted verifiers;
- runtime architecture;
- governance mechanisms and risk telemetry;
- policy and authority translation;
- observability and semantic review;
- performance and cost optimization;
- safety boundaries and compensation behavior.

38.2 Programming software for software

Framework authors already write software whose direct user is another programmer. Outcome-Oriented Programming extends this one level further: **developers increasingly write software whose direct consumer is an agentic runtime.**

38.3 The highest-value code becomes more foundational

When application composition becomes cheap, the value of reliable primitives increases. A payment tool with precise invariants, idempotency, auditability, and compensation may be used by thousands of dynamically constructed workflows. The leverage of foundational engineering grows.

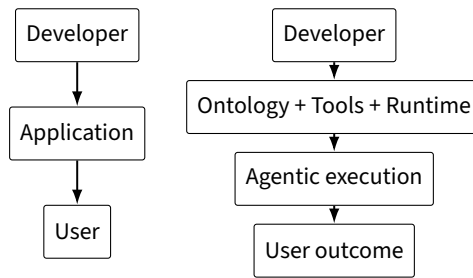


Figure 38.1: The programmer increasingly builds the execution substrate rather than every application workflow.

38.4 From CRUD gravity to computer-science gravity

When generative systems make ordinary application plumbing cheaper, engineering effort does not vanish; it migrates toward problems that are harder to synthesize safely from examples alone. The center of gravity moves toward areas such as:

- operational semantics and type systems for the IER;
- formal specification of tool state transitions;
- concurrency, partial failure, redelivery, idempotency, and distributed commit;
- authority propagation and occurrence-sensitive security;
- model checking and symbolic protocol verification;
- refinement between abstract invariants and concrete verifier behavior;
- provenance of runtime evidence and trusted computing bases;
- proof automation and proof engineering for reusable primitives.

The programmer increasingly stops being paid for remembering framework boilerplate and becomes responsible for **defining what generated execution means**. That is not less computer science. It is a return to some of its hardest subjects.

When code generation becomes abundant, precise semantics become scarce.

Part V

Beyond Applications

39 When the Application Boundary Dissolves

Part I began with a conditional limit: if an agent can synthesize a correct executable after an intent arrives, then a task-specific application need not pre-exist that request. The practical architecture did not eliminate software; it moved the durable value into tools, contracts, runtimes, policies, verifiers, and data systems.

Applications exist partly because workflows have historically needed to be assembled ahead of time. A CRM, ERP, travel portal, and expense system each package a set of screens and workflows around a bounded data and service model. An Intent Runtime can cross these boundaries dynamically and can construct a temporary composition whose lifetime is the execution rather than the product release.

A user may ask:

Find European customers with high churn risk and more than EUR 100,000 in annual revenue, show me the reasons, and let me assign selected accounts to the retention team.

The execution might cross analytics, CRM, identity, messaging, and assignment systems. The temporary interface presented to the user may exist only for this intent.

39.1 Applications become hosts

The application does not disappear immediately. It becomes one of several things:

- a host for tools;
- a data and policy boundary;
- a UI surface;
- a legacy workflow container;
- an execution node crossed by lineages.

What changes is that the application is no longer assumed to be the complete unit of user intent or authority.

39.2 Temporary applications

Generative UI can make this shift visible. Instead of navigating through five prebuilt products, a runtime can synthesize the minimal interface required for the current outcome: a filtered list, an explanation panel, and an assignment action.

The UI is a projection of the execution, not necessarily a permanent application artifact.

39.3 The new stack

The conceptual stack becomes:

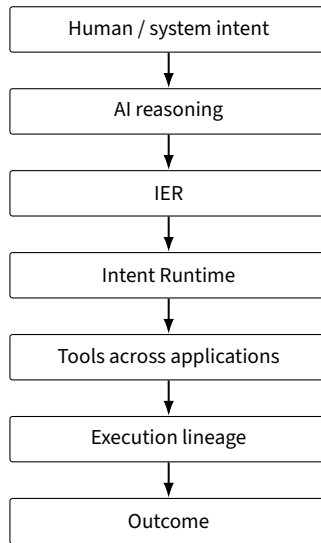


Figure 39.1: The user interacts with an intent layer that can compose execution across applications.

💡 KEY IDEA

The deepest shift is not that AI can write applications. It is that some outcomes no longer require a pre-existing application workflow at all.

40 Design Principles

The architecture can be summarized as a set of design principles.

40.1 Distinguish theoretical universality from production architecture

The ability to synthesize arbitrary software is a useful upper bound on flexibility, not a reason to regenerate stable behavior. Treat full synthesis as a fallback. Prefer the cheapest mechanism that satisfies the outcome, authority, policy, and verification requirements.

40.2 Program outcomes, not procedures

Users should be able to express desired states and constraints without specifying the implementation path when the system has enough information to plan under the stated constraints.

40.3 Keep intent distinct from outcome

Outcome is the state to achieve. Intent is the originating act that gives the execution meaning, context, and authority.

40.4 Use AI for semantic leverage

Reasoning should identify, compose, and adapt execution. Known deterministic work should remain deterministic.

40.5 Lower intent into an execution representation

Do not execute open-ended natural language directly. Translate it into semantic intent and a constrained IER that the runtime can validate and observe.

40.6 Use a shared semantic coordinate system

Map outcome predicates, operations, tool manifests, evidence, and policy-relevant concepts into a controlled domain ontology. Treat the AI's mapping as a proposal that may need independent validation.

40.7 Separate artifact identity from behavioral assurance

A hash, signature, or provenance record can identify an artifact or release path. Require stronger contracts, runtime checks, proofs, or refinement evidence when execution needs stronger behavioral assurance.

40.8 Maintain two independent containments

Application containment constrains what accepted software behavior may occur. PIC authority containment constrains what the current lineage may cause. Require both for high-impact execution.

40.9 Make tools authority-neutral

A tool describes functionality. It does not imply that the current execution has authority to use it.

40.10 Treat physical executors as tools on the same road

A person acting through a device, a server or workload, an autonomous vehicle, a robot, a machine, or a connected object are all executors an execution may reach. Bind them through the same manifests, authority rules, and gates rather than through separate authority layers; domain-specific safety controllers, real-time constraints, and hardware adapters remain, behind the same rules.

40.11 Keep transport out of the security model

Channels, queues, and protocols must be secure in their own right, but the authority of a step must never be inferred from the channel it arrived on. A security model that depends on transport cannot state, in its own terms, why a step is authorized.

40.12 Make lineage first-class

Every meaningful effect should belong to an explicit causal execution lineage.

40.13 Model application and authority as parallel planes

For every action, ask both what the action does and what lineage is authorized to cause it.

40.14 Let authority follow the lineage

Do not derive the authority of a request solely from what an executor happens to possess. Preserve causal continuity from the origin.

40.15 Make composition explicit

Possessing multiple valid authorities does not authorize their implicit union. When independent lineages must participate in one execution, authorize that meeting at an explicit composition boundary and preserve the parent lineages as distinct causal inputs.

40.16 Preserve causal meaning, not necessarily full history

A runtime need not carry an entire delegation chain in every message. Checkpoints, commitments, or trusted references may summarize continuity, provided the receiving decision still has enough trusted context to distinguish the lineage and validate the predecessor relation.

40.17 Do not hide authority expansion

If new authority is introduced, make its origin explicit through a gate, a new authority root, or a pre-established conditional rule.

40.18 Treat gates as runtime primitives

A governance decision is part of the execution architecture, not an ad hoc UI interruption. Feed gates with explicit execution evidence and governance KPI/KRI rather than model confidence alone.

40.19 Put intelligence at gates, humans where deliberately required

Policy engines, specialized governance agents, and humans can all evaluate gates. Human attention should be reserved for decisions that need human judgment, authority, or accountability.

40.20 Crystallize repetition

Repeated successful plans should migrate toward verified workflows and deterministic tools, reducing inference cost and latency.

40.21 Review semantics, not only source

Make outcome, effect, authority, and lineage changes visible as first-class review artifacts.

40.22 Optimize the whole execution economy

Treat inference, external calls, latency, money, and irreversible effects as resources the planner and runtime can optimize together.

40.23 Make causal time explicit

An execution plan is not merely topology. Model the fact that successors and concrete outcomes may materialize later.

40.24 Sign or specify predicates for unknown outcomes

When future facts do not exist yet, bind the acceptable case before execution and evaluate runtime evidence when the fact materializes.

40.25 Separate continuity from uniqueness and composition

Branching, redelivery, idempotency, exactly-once behavior, and multi-lineage composition require their own mechanisms. Do not infer uniqueness or permission to combine independent authorities merely from authority continuity.

40.26 Treat AI output as proposal, not proof

The planner may produce candidate execution. Deterministic or formally justified receiving boundaries decide which candidates become effects.

40.27 Formalize high-leverage tools

The more broadly a tool can be composed, the more valuable explicit preconditions, postconditions, invariants, and refinement evidence become.

40.28 Keep the theorem boundary visible

State which properties are formally proven, which are implementation assumptions, which depend on runtime evidence, and which remain product or policy decisions.

41 A Worked Example

Consider a business user who says:

Renew the software subscriptions we still need for next quarter, keep total spend under EUR 25,000, and do not renew anything that has not been used in the last 60 days. If a renewal creates a multi-year commitment, get approval first.

This single example contains nearly every element of the architecture.

41.1 Intent and desired outcome

The desired outcome is not “run a renewal script.” It is a set of conditions:

- necessary subscriptions remain active;
- unused subscriptions are not renewed;
- total spend remains below EUR 25,000;
- multi-year commitments require a gate;
- accounting receives the resulting commitments.

The intent originates a lineage with authority to assess and potentially renew subscriptions within some organizational context.

41.2 Semantic interpretation

Before planning, the runtime lowers the accepted request into domain semantics. For example, the outcome can reference ontology concepts for subscription activity, last-use evidence, renewal commitment, spend, accounting record, and approval-required commitment. The exact ontology is domain-specific; the important point is that later tool binding and policy do not depend only on free-form natural-language similarity.

Candidate tools advertise compatible operations through Semantic Manifests. A renewal connector may declare a `subscription.renew` operation, effects such as `commitment.created`, preconditions on quote validity, and an assurance profile. The planner can nominate that connector, but the runtime accepts it only if semantic and assurance requirements match the operation.

41.3 IER generation

The planner might produce:

```
subscriptions = software.list(active=true)
usage = telemetry.last_use(subscriptions)
needed = filter(subscriptions, usage <= 60 days)
quotes = renewal.quote(needed)
plan = optimize(quotes, total < 25000 EUR)

for item in plan:
    if item.term > 12 months:
        gate MULTI_YEAR_COMMITMENT(item)
        renewal.purchase(item)

accounting.record(plan)
validate desired_outcome
```

The IER is small because the tools already implement listing, telemetry, quoting, optimization, purchasing, and accounting integration.

41.4 Application plane and application containment

The runtime binds ontology-level operations to concrete tools whose manifests and assurance satisfy the required application contract:

Operation	Example tool binding
List subscriptions	SaaS inventory service
Read usage	telemetry warehouse
Request quote	vendor procurement connector
Purchase renewal	procurement system
Record commitment	accounting system

41.5 Authority plane

Suppose the origin context permits analysis and renewals up to twelve months, with a EUR 25,000 budget. That context travels with the lineage and may be narrowed as execution progresses.

A multi-year renewal is not silently authorized merely because the procurement tool can perform it. The execution reaches a gate.

41.6 Governance gate

The gate evidence can be evaluated by a procurement policy engine, a budget agent, and a human approver if policy requires it. If approval creates new authority for a multi-year commitment, the provenance of that authority must be explicit rather than inserted invisibly into the current context.

The same execution produces governance evidence: spend consumed versus budget, number and value of commitments, assurance level of the purchase tool, exceptions encountered, authority narrowing, policy results, latency, and progress toward the outcome. These can feed KPI/KRI and an organization-specific residual-risk decision.

A high risk or compliance exception does not cause the agent to “reason harder”; it causes a gate.

41.7 Outcome validation

At the end, the runtime checks actual subscriptions, commitments, total spend, and accounting records against the desired outcome.

The user did not write a program. The user programmed an outcome by expressing an intent. The agent proposed a semantic interpretation and generated a plan. Ontology linked that interpretation to known operations. Semantic Manifests and contracts constrained which tools could implement them. Tools performed deterministic work. The lineage tied the actions together. PIC constrained authority. Runtime evidence fed governance, and governance controlled exceptional continuation.

That is Outcome-Oriented Programming in one execution.

42 The Architectural Shift

It is useful to compare the old and new worlds directly.

APPLICATION-CENTRIC OUTCOME-CENTRIC

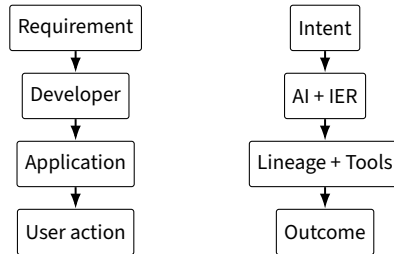


Figure 42.1: The transformation boundary moves from application development toward runtime intent execution.

The important change is not the removal of applications. **It is the removal of the assumption that every meaningful outcome must be implemented as a fixed application workflow before the user can ask for it.**

The important change is not the removal of programmers, even if their number changes (Section 1.2). **It is the movement of programming effort** toward higher-leverage domain ontologies, deterministic tools, Semantic Manifests, formal contracts, runtimes, invariants, proof obligations, and governance.

The important change is not the removal of humans from decisions. **It is the placement of humans at deliberate authority and judgment boundaries** rather than inside every execution loop.

And the important security change is not one mechanism but a separation of assurance claims. Artifact identity tells us which software artifact we received. Application containment tells us which behavioral properties of that software are justified. PIC makes causal execution lineage a first-class coordinate of authority. Governance evaluates the residual risk left outside those proven or checked properties.

43 Conclusion: Programming Outcomes

The book began with a conditional theoretical limit. If a synthesis mechanism can, for every feasible intent and environment, construct a correct executable whose result satisfies the required outcome predicate, then a task-specific application does not have to exist before the request. The application is no longer logically necessary as a pre-authored intermediate artifact.

That limit does not describe a satisfactory production system. The synthesis substrate is itself software, and unrestricted runtime generation moves design cost, uncertainty, security decisions, and governance into the hot path. The practical architecture therefore preserves the freedom of outcome-driven planning while refusing to regenerate or re-trust everything on every request.

Outcome-Oriented Programming proposes that separation explicitly: intelligence interprets and proposes; accepted intent and outcomes are mapped into a domain ontology; a compact execution representation carries ontology-level operations; reusable tools expose Semantic Manifests and assurance evidence; receiving boundaries verify selected application and authority invariants; governance evaluates residual risk; and execution is tracked as a causal lineage rather than hidden inside an application boundary.

Its strongest form is not simply intent-to-plan translation. **It is execution across an unknown future:** the concrete successor may not exist yet, runtime facts may appear after delegation, and the final acceptable outcome may be characterized by a predicate rather than known as an object at the beginning.

A human or system expresses an intent and a desired outcome. An AI planner lowers that intent into an execution representation. The runtime composes stable deterministic tools rather than regenerating their implementation. The resulting actions form an execution lineage that may cross applications, services, agents, organizations, and time.

The same lineage is viewed through two planes. The application plane describes the operations and tools that realize the outcome. The authority plane carries the context that the origin is entitled to cause. PIC binds authority to causality across the execution, so that an executor's unrelated possessions do not silently become authority for the current request.

That distinction also governs composition. Equal authority values do not imply equal authorization when they arise from different causal histories. If independently authorized lineages must meet, their combination should be a visible security decision rather than an accidental consequence of co-location inside one agent or service. The architecture therefore treats explicit execution composition as a gate with its own policy and evidence, while keeping the ordinary linear PIC continuation model conceptually intact.

Execution context should likewise be understood semantically rather than as a demand to transport unlimited history. A trusted transition point may checkpoint continuity and refer to prior state compactly. What cannot be discarded is the causal distinction itself: once two histories that policy needs to treat differently have been collapsed into the same authority view, no later authority-only decision can reconstruct the lost distinction.

Existing identity and authorization infrastructure can remain the source of initial authority. The PIC-X material provides one high-level example of an explicit exchange / Trust Plane boundary that validates existing authority and derives the first PIC context before continuity begins. The implementation mechanism is secondary to the architectural separation: origination creates authority; continuity constrains its later propagation.

Because interpretation and planning are probabilistic, the runtime should increasingly be verifier-oriented. AI generates candidate meanings and plans; ontology rules, Semantic Manifests, tool contracts, receiving predicates, evidence checks, and formal models decide whether selected properties hold. The PIC proof stack demonstrates the separation cleanly: Lean checks abstract mathematics, Tamarin checks stateful symbolic traces, and ProVerif independently checks authentication-core implications. The broader semantic-tool architecture proposed in this book extends that design principle beyond authority while keeping its new proof obligations explicit.

When execution reaches a boundary, an Execution Gate asks whether continuation is allowed. The evaluator can be a deterministic policy engine, a specialized compliance or risk agent, a human, or a combination. As governance becomes more intelligent, humans can move from universal supervision to selected points of authority and accountability.

Repeated agentic solutions can be cached, verified, and crystallized into deterministic workflows or tools. The system therefore tends to move successful behavior from expensive reasoning toward cheap execution.

The thesis is therefore larger than “AI writes software.”

- Users define outcomes.
- Intent is mapped into explicit semantics.
- Agents compose verified operations.
- Application behavior and authority are contained separately.
- Lineages preserve causality through time.
- Independent authority is composed only by explicit decision.
- Governance controls residual risk and continuation.

The resulting stack can be summarized in one diagram:

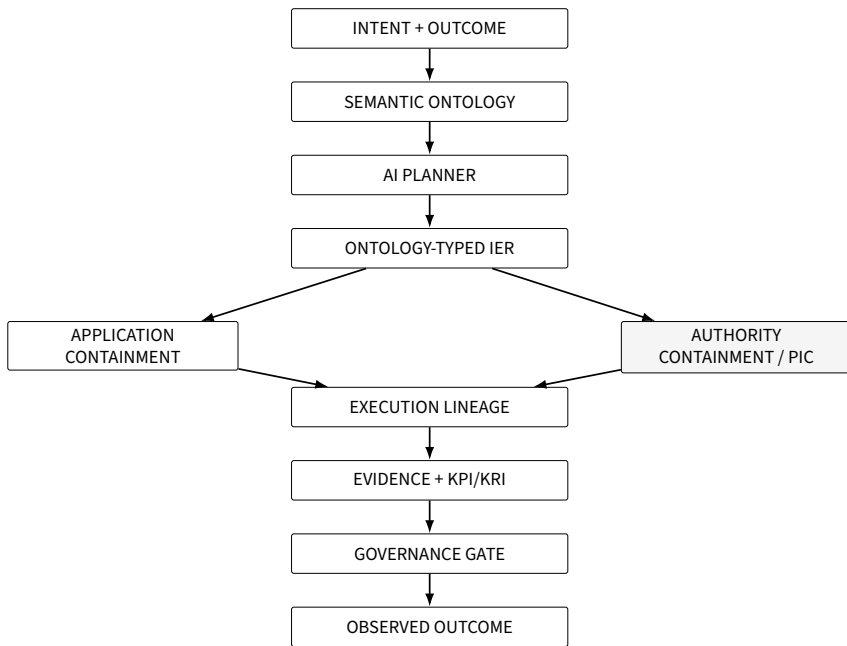


Figure 43.1: Outcome-Oriented Programming: semantic meaning is mapped to constrained execution, application behavior and authority are contained separately, and governance decides on residual risk exposed by runtime evidence.

If this architecture matures, software may become less a collection of preassembled applications and more a semantic space of reliable, constrained, and increasingly verifiable operations that intelligence composes at runtime. Applications will still exist where they are economically, organizationally, or semantically useful. What changes is that they cease to be the mandatory unit of composition, causality, and authority.

The theoretical limit was: ask for an outcome and synthesize everything. The engineering answer is stronger because it is selective: **synthesize only what must remain dynamic; reuse what is stable; prove or verify what is critical; preserve causal authority through time; and make governance part of execution.**

That is the proposed shift: from programming applications to programming outcomes.

A Legal, Rights, and AI Disclosure

A.1 Standard copyright license

Except where otherwise indicated, the original text, original diagrams, and original illustrative pseudocode and listings in this book are licensed by Nitro Agility S.r.l. under the Creative Commons Attribution 4.0 International License (CC BY 4.0). The standard CC BY 4.0 legal code governs that license and is available at <https://creativecommons.org/licenses/by/4.0/>. Nothing in this appendix replaces or modifies the Creative Commons license.

The license applies only to material for which Nitro Agility S.r.l. has the right to grant the stated license. Referenced papers, formal artifacts, software, standards, protocols, quotations, trademarks, and other third-party or separately licensed material remain subject to their respective rights and terms. No patent or trademark rights are granted except where expressly stated.

A.2 Book source repository

The LaTeX source of this book, its cover, its README, and any errata or subsequent editions are published by Nitro Agility S.r.l. at <https://github.com/nitroagility/books>, in the directory `outcome-oriented-programming`. The repository is the canonical location for the published text; the CC BY 4.0 license stated above applies to the original material it contains, subject to the same exclusions for third-party and separately licensed material.

A.3 Authorship and publication capacity

Nicola Gallo is identified as the author and original creator of the intellectual and architectural contributions attributed to him in this work. Nitro Agility S.r.l. is identified as publisher / issuer, economic copyright holder, and licensor of this edition. For this publication, Nicola Gallo acts in his stated corporate capacity as director of Nitro Agility S.r.l., in the name and on behalf of the company. Authorship does not, by itself, constitute a separate personal warranty, certification, professional undertaking, or service commitment to readers or implementers.

A.4 Artificial-intelligence assistance

Generative artificial-intelligence systems were used as editorial and production tools to accelerate drafting, restructuring, language refinement, consistency checking, formatting, and related preparation of this book. The conceptual architecture, selection and development of the original contributions, technical positions, and final editorial decisions are attributed to Nicola Gallo. AI systems are not identified as authors or rights holders of this publication, and their use is not represented as independent verification of any technical, mathematical, legal, security, safety, or compliance claim.

A.5 Technical scope and implementation responsibility

This publication is provided for research, educational, architectural, and technical discussion. It is not a certification of any product, protocol, implementation, or deployment and does not constitute legal, regulatory, compliance, cybersecurity, financial, or engineering-certification advice. References to laws, regulations, standards, guidance, and industry frameworks are descriptive, selective, and non-exhaustive; they are not a legal interpretation, compliance mapping, conformity assessment, or representation that any architecture or implementation satisfies an applicable requirement. Applicability and current obligations depend on the facts, jurisdiction, role, and law governing a particular deployment.

No statement in this book should be read as a promise that Outcome-Oriented Programming, PIC, PIC-X, Semantic Manifests, formal verification, execution lineages, Execution Gates, Governance Intelligence, or any combination of the techniques described will eliminate security risk, compliance risk, operational risk, model risk, financial risk, or residual risk. Mathematical and formal-verification statements apply only to the properties, models, assumptions, abstractions, and artifacts expressly stated. A theorem about a model is not, by itself, a certification of a real-world implementation.

While reasonable care has been taken in preparing this book, no representation or warranty is made as to the accuracy or completeness of its contents, including its mathematical, formal, and technical statements, and, to the maximum extent permitted by applicable law, no liability is accepted for errors or omissions. Where the book restates formally verified results, the versioned artifacts identified in Appendix B are the primary technical reference for the exact properties encoded and checked there; the book's renderings are expository and do not extend the scope of those artifacts, models, or assumptions.

Any person or organization that implements, adapts, deploys, integrates, evaluates, commercializes, or relies on material described in this publication remains responsible for its own architecture, implementation, validation, testing, cybersecurity, privacy, regulatory and contractual compliance, operational controls, risk assessment, governance, and deployment decisions.

Economic, market, organizational, employment, and technology-adoption passages express the author's analysis and strategic hypotheses as of the publication date. They are not guarantees or representations of future results and should not be read as individualized investment, employment, procurement, management, or financial advice. Actual outcomes may differ materially as technology, competition, regulation, adoption, market structure, and other conditions evolve.

A.6 Third-party rights and implementation

Third-party patents, trademarks, copyrights, and other proprietary rights remain with their respective holders. No representation or warranty is made that every possible implementation, product, service, protocol, deployment, or combination based on concepts described in this book is unrestricted by third-party rights. Publication of

a concept should not be interpreted as a representation that the concept is patented, patentable, unpatented, or unrestricted by third-party rights.

Any person or organization implementing, commercializing, distributing, deploying, integrating, or otherwise using concepts described in this work is responsible for assessing the intellectual-property rights applicable to its own implementation and for obtaining any licenses or permissions that may be required. The CC BY 4.0 license applicable to this publication does not grant third-party patent or trademark rights.

A.7 Attribution and rights-review channel

Reasonable care has been taken to identify and attribute third-party material. If a rights holder, author, contributor, or other person believes that material in this publication incorporates protected work, requires additional attribution or acknowledgment, or contains an inaccurate rights attribution, they may contact Nitro Agility S.r.l. at info@nitroagility.com and identify the material, its location in the book, the asserted right or contribution, and the correction requested. Nitro Agility S.r.l. may review good-faith notices and, where appropriate, correct attribution, add acknowledgment, or amend or remove material in a future edition or distribution.

This review channel is intended to facilitate good-faith correction. Contacting Nitro Agility S.r.l. is not stated as a mandatory precondition to the exercise of any legal right or remedy. The existence of the channel, receipt or review of a notice, or a voluntary correction does not by itself constitute an admission of infringement, liability, ownership, validity, or enforceability.

B Source and Claim Discipline

This book intentionally separates claim classes and keeps them distinct from system-level warranties or certifications.

B.1 PIC public resources

The public PIC project and theoretical work cited throughout this book are available at:

- Official site: <https://www.pic-protocol.org>
- Public source organization: <https://github.com/pic-protocol>
- Formal-model portal: <https://www.pic-protocol.org/formal-model>
- Formal-artifact repository: <https://github.com/ngallo/pic-model> (directory `draft/0.1/pic-model-math/`)
- Theoretical paper: <https://arxiv.org/abs/2607.08906>

B.2 Formal-artifact snapshot

To make the proof references in this edition reproducible even if public repositories later evolve, the reference snapshot is the artifact set published in <https://github.com/ngallo/pic-model>, directory `draft/0.1/pic-model-math/`. The Lean development is multi-file, so every proof module is fingerprinted individually. SHA-256 fingerprints of the snapshot files:

```
pic-lean/Main.lean
a19997a35c22c5257c374ddf6027f1f95fb506c4d4942c0b34bc7fd5d337cb26
pic-lean/PICVerification.lean (module index)
56de4db44a2c87417d7f7df2417239124b942fb16a76f780be8f4371372cbb81
pic-lean/PICVerification/Basic.lean
935c7ad8846d3ba16a8b57b7e5c1f92955e572aa40941933057c694d925699e7
pic-lean/PICVerification/Chain.lean
967c2211b6e0213f1ff3802d53ed6c6d3131922293b4e0e6c0234be81bf57327
pic-lean/PICVerification/ConfusedDeputy.lean
dd24ab7a73ac556ff91f397fdd5f81c7ef279be239c203d1d52fa58248444db8
pic-lean/PICVerification/Possession.lean
cee8ea469070636f0474cdac882ba55378100524bf59da972e9e054a409c60e8
pic-lean/PICVerification/Projection.lean
c402fcc8d066e569d58e8da41afd7499f8f0929bea84af3343b614fea67bd54b
pic-lean/PICVerification/Refinement.lean
e044383929c18a3854dbf75be2340fe5ea9d6296addec8cd868100747351dca6
pic-lean/PICVerification/TradeOff.lean
53855c622ae852f210a13cc729f9b26c34816cb156ee0b3712f7865bf71e8487
pic-lean/PICVerification/Translation.lean
653ee971434ef69ad56d44571d7bbada758f0142c246dafb65e726c70d464c7
pic-tamarin/PICContinuity.spthy
0396faf0e2a35f50a0a9183ee85063dd24797b4370342b998d901ffa22509b7b
pic-proverif/pic_continuity.pv
43ca9b2dc59a85543dbdc4d5143c56fe9f71593209a7eb444b7a005061662b8
```

These hashes identify bytes; they are not themselves proofs of correctness.

B.3 Claim classes

PIC claims are limited to properties supported by the public PIC formal model, specification material, and proof artifacts. The mathematical core treats execution as a causal lineage of steps carrying non-expansive authority contexts; the published Lean development formalizes abstract theorems, while the published Tamarin and ProVerif artifacts address complementary message-level properties. Those artifacts state their own assumptions and limits.

PIC-X claims are limited to the high-level architecture described in the public implementation material. PIC-X is treated here as an exchange and Trust Plane component that can derive an initial PIC authority context from existing authority infrastructure, with OAuth as an initial integration target. PIC-X does not define PIC, and this book does not treat its current implementation profile as a standards-body interoperability guarantee.

Outcome-Oriented Programming claims are the architectural thesis of this book. Concepts such as the Intent Execution Representation, the Intent Runtime, semantic ontologies, Semantic Manifests, formal tool contracts, application containment, verifier-oriented execution, and governance intelligence are presented as a proposed software architecture. Where this book extrapolates from PIC's verified separation between proposal and acceptance, that extrapolation is identified as a design proposal rather than a property already proven by PIC.

Market and economic claims appear mainly in Chapter 1: the arbitrage windows, the compression of consulting and internal teams, the restructuring of skills, and digital sovereignty as a parallel opportunity. They are the author's reading of patterns that economics has already described (the smiling curve, the conservation of attractive profits) applied to models and agentic execution. They are forward-looking scenario analysis and interpretation: they are not formally verified results, they are not necessary conditions for the architecture of this book to be valid, and they do not imply that any particular industry, profession, organization, or role will necessarily experience the outcome described. Alternative trajectories remain possible.

For avoidance of ambiguity, "PIC" in this book refers to Provenance Identity Continuity as defined in the PIC materials cited herein, and "PIC-X" refers to the corresponding exchange / Trust Plane work attributed in this book to Nicola Gallo.

Composition claims require an additional distinction. The core PIC model used in this book represents each lineage as a finite linear causal path with non-expansive authority. Multiple sibling continuations may share a prefix, producing a branching execution graph made of distinct lineage extensions; that does not turn one lineage into a DAG. When this book discusses joining two independently authorized lineages, it treats that join as an explicit new authorization / composition event, not as an ordinary PIC continuation and not as a theorem already established by the current linear PIC formalization. A concrete multi-lineage composition protocol requires its own specification, policy semantics, authority bound, and verification argument.

Formal-verification claims are property-specific. A cryptographic digest can support

byte-level integrity checking relative to a trusted expected digest, and a valid digital signature can authenticate signed bytes relative to a signing key and trust model. Neither mechanism, by itself, proves what the software will do. A proof establishes only the stated property of the stated model or implementation under explicit assumptions. This book therefore distinguishes artifact integrity, semantic specification, runtime evidence, model verification, and implementation refinement instead of treating “verified” as a single undifferentiated label.

The opening claim that a task-specific application can disappear is explicitly a **conditional theoretical limit**. It assumes a synthesizer capable of producing a correct executable for every feasible intent and environment. The book does not claim that present-day AI satisfies that assumption. The logical consequence is narrower: under that assumption, a task-specific application need not pre-exist the request. The synthesis and execution substrate remains software.

The purpose of this discipline is to keep the boundary between theorem, implementation claim, architectural proposal, and open problem visible.

C Enterprise Risk Framework Correspondence

This appendix is intentionally short. It is a descriptive correspondence between the architectural model in Chapter 26 and selected public primary sources. It is not a compliance checklist, legal interpretation, certification, or statement that any cited framework requires Outcome-Oriented Programming, PIC, execution lineage, or the terminology used in this book.

C.1 NIST Cybersecurity Framework 2.0

The NIST Cybersecurity Framework (CSF) 2.0 provides a taxonomy of high-level cybersecurity outcomes and expressly does not prescribe how those outcomes must be achieved [22]. Its six Functions are Govern, Identify, Protect, Detect, Respond, and Recover. NIST's CSF resources also place risk appetite and risk tolerance inside the Govern function [22].

A useful architectural correspondence is therefore: security controls and invariants primarily contribute to protection and prevention; residual-risk characterization contributes to identification and governance; monitoring contributes evidence to detection and governance; response and recovery handle realized adverse events. This is a correspondence, not a normative mapping.

C.2 NIST Risk Management Framework

NIST SP 800-37 Rev. 2 organizes the Risk Management Framework around Prepare, Categorize, Select, Implement, Assess, Authorize, and Monitor, and describes authorization as a risk-based decision by a senior official about operation of a system [23].

That vocabulary creates an important terminological warning for this book: RMF *Authorize* is a management risk decision; it is not the same concept as protocol-level authorization of an API call, token, tool invocation, or execution continuation. The distinction mirrors the separation developed here between authorization mechanics and governance acceptance.

C.3 NIST Zero Trust Architecture

NIST SP 800-207 describes zero trust as removing implicit trust based solely on network location or ownership and treats authentication and authorization of subject and device as discrete functions before a session to a resource is established [24].

PIC's execution-lineage continuity is presented in this book as an additional architectural coordinate for multi-hop causality. This book does not claim that SP 800-207 requires that property or that conformance to zero-trust guidance implies PIC continuity.

C.4 NIS2 and ENISA guidance

NIS2 Article 20 requires, for entities within its scope, management bodies to approve cybersecurity risk-management measures and oversee their implementation. Article 21 requires appropriate and proportionate technical, operational, and organizational measures to manage cybersecurity risk [25].

ENISA's 2025 technical implementation guidance for cybersecurity risk-management measures gives a more operational example: it describes documented risk assessment and treatment and states, within the scope of that guidance, that residual risks are to be accepted by management bodies or other accountable persons with authority to manage risk [26].

The correspondence to this book is limited: regulation and guidance define risk-management and governance responsibilities; this architecture proposes technical mechanisms for constraining selected execution and authority failure classes. NIS2 and the cited guidance are not claimed to require PIC, Temporal Confused Deputy terminology, or execution-lineage protocols.

C.5 DORA

For financial entities within its scope, DORA assigns the management body responsibility for the ICT risk-management framework and includes determination of an appropriate ICT risk-tolerance level among its responsibilities; it also requires a documented ICT risk-management framework [27].

That is conceptually compatible with the book's separation between technical controls, residual-risk evidence, governance decisions, and accountable ownership. It is not evidence that DORA mandates the particular runtime, gate, ontology, or authority-continuity mechanisms proposed here.

C.6 EU AI Act

For high-risk AI systems within its scope, Article 9 of the EU AI Act establishes a continuous lifecycle risk-management process. Article 14 addresses human oversight, including risks that persist despite other requirements, and Article 15 addresses accuracy, robustness, and cybersecurity [28].

The book's Execution Gates and verifier-oriented runtime can be read as architectural patterns for placing technical checks and human or organizational decisions at explicit boundaries. They are not presented as a compliance recipe, and the applicability of the AI Act depends on the legal classification and use of the actual system.

C.7 What the correspondence does not establish

None of the sources above is cited as proving that an implementation is secure or compliant. None is claimed to standardize authority continuity, explicit multi-lineage composition, or Temporal Confused Deputy. The useful separation is narrower:

Architecture layer	Enterprise-risk interpretation
Security engineering	Constrain, prevent, detect, or mitigate technical failure scenarios.
Risk analysis	Characterize harmful possibilities and uncertainty that remain.
Governance	Decide treatment or acceptance under organizational context and tolerance.
Accountability	Identify the authorized owner of the decision.
Continuous evidence	Reassess assumptions, controls, exposure, and decisions over time.

Design principle. Regulation and risk frameworks can define obligations, responsibilities, and decision processes. Security architecture defines technical mechanisms. A correspondence between the two is not a claim that one uniquely implements the other.

Glossary

- Actual outcome** The state observed after an execution, evaluated against the desired outcome.
- Application** A preassembled software composition that packages functionality, workflow, data access, and often user interaction. In this book it remains useful, but is not assumed to be the universal unit of intent, execution, or authority.
- Application containment** The application-plane discipline in which an accepted tool invocation is constrained by its semantic contract and the assurance evidence required for that operation. It is distinct from authority containment.
- Application plane** The projection of an execution concerned with operations, tools, data, and effects.
- Assurance level** A book-level classification of what evidence supports a tool claim, ranging from artifact identity/integrity through semantic contracts and formally verified properties to implementation refinement.
- Authority containment** The PIC-based constraint that a lineage may only exercise authority that remains a valid causal continuation of its origin authority context.
- Authority context** The set or semantic scope of authority carried by a lineage occurrence.
- Authority plane** The projection of an execution concerned with origin authority, policy, causal continuity, and permissible effects.
- Causal time** The ordering induced by execution causality ($s_0 \prec s_1 \prec \dots$), distinct from clock expiration time.
- Composition Gate** An Execution Gate that decides whether independently authorized lineages may participate together in a new execution context and under what constraints.
- Deferred authority** Origin-bounded conditional authority whose concrete applicable case is resolved later from runtime evidence.
- Desired outcome** The state and constraints that execution is intended to make true.
- Execution Gate** A runtime checkpoint at which continuation requires an explicit governance decision.
- Execution lineage** A finite linear causal path of execution occurrences originating from an authority origin. A fan-out creates distinct lineage extensions with a shared prefix; the overall execution graph may branch while each lineage remains linear.
- Explicit execution composition** A separately authorized event in which selected, policy-bounded authority from independent lineages is permitted to participate

in a new execution context while the contributing lineages remain distinguishable. In this book it is an architectural extension, not an ordinary linear PIC continuation.

Governance Intelligence Policy engines, specialized AI agents, humans, or combinations that evaluate Execution Gates.

Governance decision An organizational decision about treatment or continuation based on risk, policy, evidence, and authority; it is distinct from a security proof.

IER Intent Execution Representation: a compact machine-oriented representation of the plan to realize an intent.

Intent An originating request or act that carries purpose, context, constraints, and an authority relation to the effects it seeks to cause.

Intent Runtime The runtime layer that interprets IER, binds tools, tracks lineage, evaluates authority, orchestrates gates, executes, observes, and validates outcomes.

Lineage projection A mapping that forgets the lineage coordinate and retains only an authority / possession view; distinct executions may collide under this projection.

New origination A new authority-creating event required when an action needs authority not available inside the current non-expansive lineage or an explicitly authorized composition bound.

Ontology A controlled machine-readable vocabulary and relation system used to give shared meaning to outcomes, operations, entities, effects, evidence, and policy-relevant concepts. In this book it is domain-scoped and architectural, not assumed to be a universal world model.

Operation A semantic action such as purchase, search, notify, or record.

Outcome-Oriented Programming A programming model in which desired outcomes are specified while an agentic system constructs the execution needed to realize them.

Outcome predicate A machine-evaluable condition describing the set of acceptable concrete outcomes, potentially evaluated only when runtime facts exist.

Plan approval The first Execution Gate of an execution: acceptance of the proposed plan, by policy, by a governance agent, or by a person, before any effect is caused.

PIC Provenance Identity Continuity, the model in which authority is propagated as a causally linked, non-expansive property of an execution lineage. Public resources: official site, formal model, GitHub, and theoretical paper.

Proof of Relationship (PoR) Evidence that a successor step is a valid causal continuation of its immediate predecessor within the same lineage.

Proof-of-Continuity (PoC) The transitive continuity property of a lineage whose

adjacent steps preserve causal relationship and non-expansive authority under the applicable authority vocabulary or sound policy translation.

Receiving boundary The trusted acceptance boundary that verifies whether a proposed successor state is admissible under the applicable continuity, policy, evidence, integrity, and freshness rules.

Residual risk The risk associated with harmful scenarios that remain possible after the modeled controls and invariants, together with relevant assumption and epistemic uncertainty. The book does not assume that residual risk is universally scalar or additive.

Risk owner In this book, a generic term for a principal or organizational role authorized by the applicable governance model to own a risk decision. The exact legal meaning is deployment-specific.

Security invariant A property required to hold for every execution accepted by a stated model under explicit assumptions.

Semantic Manifest A machine-readable description linking a tool to ontology-level operations, effects, contracts, invariants, compensation semantics, and assurance evidence. A manifest is a claim surface; its trust depends on the evidence connecting it to the implementation.

Temporal confused deputy The book's compact label for the temporal authority/causality mismatch formalized in PIC: authority valid in itself is used in an execution whose causal lineage does not justify that use.

Tool A deterministic or bounded component that can realize an operation. It may be software acting machine to machine (a server, a workload, a function) or a physical executor reached through software (a person acting through a device, an autonomous vehicle, a robot, a machine, a connected object). Tool availability does not itself imply authority.

Transport The protocols, queues, and channels that carry an execution from one step to the next. A transport must protect what travels on it, but application and authority decisions must not depend on the channel a message arrived on.

Tool contract A machine-readable description of a tool's inputs, preconditions, postconditions, effects, invariants, failure semantics, costs, and related execution properties.

Verifier-oriented execution An architectural discipline in which probabilistic components propose execution objects while smaller deterministic or formally justified components decide whether those objects satisfy explicit acceptance conditions.

References

- [1] Nicola Gallo. *Proof-of-Continuity: A Temporal Model for Authority Propagation in Distributed Systems and AI Agents*. 9 July 2026. arXiv:2607.08906. <https://arxiv.org/abs/2607.08906>. Formal-model portal: <https://www.pic-protocol.org/formal-model>.
- [2] PIC Protocol. *Why PIC; The Threat Model; The N+1 Unknown Executor Problem; The N+1 Outcome on Unknown Facts; The N+1 Invalid State Problem*. Public documentation drafts, 2026. <https://www.pic-protocol.org/>.
- [3] PIC Protocol. *PIC Formal Model: Lean, Tamarin, and ProVerif proof artifacts*. Public documentation and source artifacts, 2026. <https://www.pic-protocol.org/formal-model>. Artifact repository: <https://github.com/ngallo/pic-model>.
- [4] Nicola Gallo. *PIC Lean Formal Proof*. Lean 4 formalization of the PIC mathematical model and refinement bridge, 2026. <https://github.com/ngallo/pic-model> (draft/0.1/pic-model-math/pic-lean). Formal-model portal: <https://www.pic-protocol.org/formal-model>.
- [5] Nicola Gallo. *PIC Tamarin Symbolic Proof*. Stateful symbolic trace model of PIC continuity, 2026. <https://github.com/ngallo/pic-model> (draft/0.1/pic-model-math/pic-tamarin). Formal-model portal: <https://www.pic-protocol.org/formal-model>.
- [6] Nicola Gallo. *PIC ProVerif Symbolic Proof*. Symbolic verification of the PIC authentication core, 2026. <https://github.com/ngallo/pic-model> (draft/0.1/pic-model-math/pic-proverif). Formal-model portal: <https://www.pic-protocol.org/formal-model>.
- [7] Nicola Gallo. *Designing PIC-X: From Specification to Architecture*. 1 August 2026.
- [8] Nicola Gallo. *Designing PIC-X: Deriving an Initial PIC Context of Authority*. 1 August 2026.
- [9] PIC Protocol. *PIC Specification*. Independently developed public draft, 2026. <https://www.pic-protocol.org/>.
- [10] PIC Protocol. *PIC Protocols*. Public draft documentation, 2026. <https://www.pic-protocol.org/>.
- [11] PIC Protocol. *PIC Implementations*. Public documentation, 2026. <https://www.pic-protocol.org/>.
- [12] Stan Shih. *Me-Too Is Not My Style: Challenge Difficulties, Break Through Bottlenecks, Create Values*. Acer Foundation, Taipei, 1996. Discusses the “smiling curve” of value distribution along a value chain, a framework Shih had introduced in 1992.

- [13] Clayton M. Christensen and Michael E. Raynor. *The Innovator's Solution: Creating and Sustaining Successful Growth*. Harvard Business School Press, 2003. Chapter 6 states the law of conservation of attractive profits.
- [14] World Wide Web Consortium (W3C). *OWL-S: Semantic Markup for Web Services*. W3C Member Submission, 2004. <https://www.w3.org/Submission/OWL-S/>.
- [15] J. B. Dennis and E. C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [16] M. S. Miller, K.-P. Yee, and J. S. Shapiro. *Capability Myths Demolished*. Technical Report SRL2003-02, Systems Research Laboratory, Johns Hopkins University, 2003.
- [17] M. Abadi and C. Fournet. Access Control Based on Execution History. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2003.
- [18] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.
- [19] D. Hardt, editor. *The OAuth 2.0 Authorization Framework*. RFC 6749, Internet Engineering Task Force, October 2012.
- [20] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore. *OAuth 2.0 Token Exchange*. RFC 8693, Internet Engineering Task Force, January 2020.
- [21] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, and D. Waite. *OAuth 2.0 Demonstrating Proof of Possession (DPoP)*. RFC 9449, Internet Engineering Task Force, September 2023.
- [22] National Institute of Standards and Technology. *The NIST Cybersecurity Framework (CSF) 2.0*. NIST CSWP 29, February 2024. <https://doi.org/10.6028/NIST.CSWP.29>.
- [23] National Institute of Standards and Technology. *Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy*. NIST SP 800-37 Rev. 2, December 2018. <https://doi.org/10.6028/NIST.SP.800-37r2>.
- [24] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. *Zero Trust Architecture*. NIST SP 800-207, August 2020. <https://doi.org/10.6028/NIST.SP.800-207>.
- [25] European Parliament and Council of the European Union. *Directive (EU) 2022/2555 (NIS2)*, Articles 20–21. 14 December 2022. <https://eur-lex.europa.eu/eli/dir/2022/2555/oj>.
- [26] European Union Agency for Cybersecurity (ENISA). *Technical Implementation Guidance on Cybersecurity Risk Management Measures*, version 1.0. June 2025.

https://www.enisa.europa.eu/sites/default/files/2025-06/ENISA_Technical_implementation_guidance_on_cybersecurity_risk_management_measures_version_1.0.pdf.

- [27] European Parliament and Council of the European Union. *Regulation (EU) 2022/2554 on Digital Operational Resilience for the Financial Sector (DORA)*, Articles 5–6. 14 December 2022. <https://eur-lex.europa.eu/eli/reg/2022/2554/oj>.
- [28] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, Articles 9, 14, and 15. 13 June 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- [29] Norman Hardy. The Confused Deputy: Or Why Capabilities Might Have Been Invented. *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

About the Author

Nicola Gallo is the author and original creator of the architectural thesis presented in this book and of the PIC and PIC-X contributions attributed to him herein. He works on security, distributed execution, authority propagation, formal reasoning, and agentic software architecture. His work on Provenance Identity Continuity (PIC) develops a temporal model for preserving the causal relationship between originating authority and later execution across distributed systems and AI-agent tool chains. Public PIC resources are available at <https://www.pic-protocol.org>, <https://www.pic-protocol.org/formal-model>, <https://github.com/pic-protocol>, and <https://arxiv.org/abs/2607.08906>. For this publication, he acts in his capacity as director of Nitro Agility S.r.l., in the name and on behalf of the company.